

**PENERAPAN ALGORITMA MACHINE LEARNING DALAM
PERAMALAN PERMINTAAN PRODUK PADA COFFEE
SHOP BERBASIS DATA HISTORIS TRANSAKSI**

SKRIPSI

Digunakan Sebagai Syarat Maju Ujian Diploma IV
Politeknik Negeri Malang

Oleh:

Doni Wahyu Kurniawan NIM. 2241720015



**PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG**

2026

HALAMAN PENGESAHAN



PENERAPAN ALGORITMA MACHINE LEARNING DALAM PERAMALAN PERMINTAAN PRODUK PADA COFFEE SHOP BERBASIS DATA HISTORIS TRANSAKSI

Disusun oleh:

DONI WAHYU KURNIAWAN

NIM. 2241720015

Proposal Skripsi ini telah diseminarkan pada **Kamis, 22 Januari 2026**.

Disetujui oleh:

1. Pembimbing : Endah Septa Sintiya. SPd., MKom.
NIP. 199401312022032007
2. Pembahas I : Ir. Rudy Ariyanto, ST., M.Cs.
NIP. 197111101999031002
3. Pembahas II : Hendra Pradibta, SE., M.Sc.
NIP. 198305212006041003

Mengetahui

Ketua Jurusan
Teknologi Informasi

Mungka Astinigrum, ST., M.Kom.
NIP. 197710302005012001

Koordinator Program Studi
D4 Teknik Informatika

Dr. Ely Setyo Astuti, S.T., M.T.
NIP. 197605152009122001

DAFTAR ISI

HALAMAN PENGESAHAN.....	II
DAFTAR ISI.....	III
DAFTAR GAMBAR	VI
DAFTAR TABEL	VII
BAB I PENDAHULUAN	1
1.1 LATAR BELAKANG	1
1.2 RUMUSAN MASALAH	4
1.3 BATASAN MASALAH	5
1.4 TUJUAN	5
1.5 MANFAAT.....	5
BAB II LANDASAN TEORI.....	7
2.1 STUDI LITERATUR	7
2.2 DASAR TEORI	10
2.2.1 Peramalan Permintaan.....	10
2.2.2 Machine Learning untuk Peramalan	13
2.2.3 Random Forest	15
2.2.4 Extreme Gradient Boosting (XGBoost)	15
2.2.5 Validasi Silang Data Deret Waktu (<i>Time Series Cross-Validation</i>)	19
2.2.6 Evaluasi Kinerja Model Peramalan.....	20
2.2.7 FastAPI.....	21
2.2.8 PostgreSQL	22
2.2.9 React.....	22
BAB III METODE PENELITIAN	23
3.1 ANALISA KEBUTUHAN SISTEM.....	23
3.1.1 Kebutuhan Fungsional	23
3.2 PENDEKATAN PENGEMBANGAN SISTEM.....	25
3.3 PENGUMPULAN DATA	26
3.4 PENGOLAHAN DATA.....	28

3.4.1 Transformasi Data	28
3.4.2 Analisa Data Eksploratif	29
3.4.3 Rekayasa Fitur (Feature Engineering)	33
3.4.4 Pra-pemrosesan Data (<i>Data Preprocessing</i>)	34
3.5 PELATIHAN MODEL	35
3.6 EVALUASI MODEL	37
3.7 PENGEMBANGAN SISTEM PERANGKAT LUNAK	38
3.7.1 Arsitektur Sistem	38
3.7.2 Desain Basis Data (<i>Database</i>)	39
3.7.3 Implementasi API	40
3.7.4 Implementasi Antarmuka	40
3.8 PENGUJIAN SISTEM	40
3.8.1 Pengujian Model	40
3.8.2 Pengujian Fungsionalitas	45
3.8.3 System Usability Scale (SUS)	41
3.9 DEPLOYMENT DAN MAINTENANCE	42
BAB IV ANALISIS DAN PERANCANGAN SISTEM	44
4.1 KEBUTUHAN SISTEM	44
4.1.1 Deskripsi Sistem	44
4.1.2 Analisis Kebutuhan Fungsional	45
4.1.3 Analisis Kebutuhan Non-Fungsional	46
4.2 DESIGN	48
4.2.1 <i>Use Case Diagram</i>	49
4.2.2 <i>Physical Data Model</i>	50
4.2.3 Arsitektur Diagram	53
4.2.4 Desain Antarmuka	54
4.2.5 <i>Deployment Diagram</i>	58
4.3 PENGEMBANGAN SISTEM	59
4.3.1 Pengumpulan dan Persiapan Data	59
4.3.2 Analisis Data Eksploratif	65
4.3.3 Rekayasa dan Seleksi Fitur	72
4.3.4 Integrasi dan Pengembangan Sistem	71

4.4 PENGUJIAN	72
4.4.1 Pengujian <i>System Usability Scale</i> (SUS)	73
4.4.2 Pengujian Model	72
4.5 PEMELIHARAAN	74
BAB V IMPLEMENTASI DAN PENGUJIAN	76
5.1 IMPLEMENTASI PROSES SISTEM.....	76
5.1.1 Proses Pelatihan Model peramalan	78
5.1.2 Proses Perhitungan Kebutuhan Barang Baku	85
5.2 IMPLEMENTASI ANTARMUKA	84
5.2.1 Halaman <i>Login</i>	84
5.2.2 Halaman <i>Dashboard</i>	85
5.2.3 Halaman Peramalan	85
5.2.4 Halaman Kebutuhan Bahan Baku	86
5.2.5 Halaman Pengaturan	87
BAB VI HASIL DAN PEMBAHASAN	89
6.1 HASIL PENGUJIAN	89
6.1.1 Hasil Pengujian <i>System Usability Scale</i> (SUS).....	94
6.1.2 Hasil Pengujian Model.....	89
6.2 PEMBAHASAN.....	96
BAB VII KESIMPULAN DAN SARAN	101
7.1 KESIMPULAN.....	101
7.2 SARAN.....	101
DAFTAR PUSTAKA	102

DAFTAR GAMBAR

Gambar 3.1 Alur <i>Iterative & Incremental Development</i>	26
Gambar 3.2 <i>Flowchart</i> Alur Pengolahan Data.....	28
Gambar 3.3 <i>Flowchart</i> Analisis Data Eksploratif.....	30
Gambar 3. 4 <i>Time Series Cross-Validation</i> dengan <i>Expanding Window</i>	35
Gambar 3.5 <i>Flowchart</i> Logika <i>Two-Level Modeling</i>	36
Gambar 3.6 Arsitektur <i>Three-Tier System</i>	38
Gambar 4.1 <i>Use Case Diagram</i>	50
Gambar 4.14 <i>Physical Data Model</i> Aplikasi Peramalan	52
Gambar 4.15 <i>Physical Data Model</i> transaksi PoS	52
Gambar 4.16 Arsitektur Diagram.....	53
Gambar 4.17 Desain Antarmuka <i>Login</i>	55
Gambar 4.18 Desain Antarmuka <i>Dashboard</i>	55
Gambar 4.19 Desain Antarmuka Peramalan	56
Gambar 4.20 Desain Antarmuka Kebutuhan Bahan Baku.....	57
Gambar 4.21 Desain Antarmuka Analitik.....	57
Gambar 4.22 Desain Antarmuka Pengaturan.....	58
Gambar 4.23 <i>Deployment Diagram</i>	59
Gambar 4.24 <i>Stats Dataset</i>	65
Gambar 4.25 Matriks Korelasi Antar-Fitur.....	66
Gambar 4.26 Analisis RF & XGBoost (<i>Gain</i>) <i>Feature Importance</i>	67
Gambar 4.27 Analisis <i>XGBoost Feature Importance (Permutation)</i>	68
Gambar 5.1 Implementasi Halaman <i>Login</i>	84
Gambar 5. 2 Implementasi Halaman <i>Dashboard</i>	85
Gambar 5.3 Implementasi Halaman Peramalan.....	86
Gambar 5.4 Implementasi Halaman Kebutuhan Bahan Baku	87
Gambar 5.5 Implementasi Halaman Pengaturan.....	88

DAFTAR TABEL

Tabel 3.1 Struktur Data <i>Raw</i> dari Sistem <i>PoS</i>	27
Tabel 3. 2 Struktur Data Final (Agregasi Harian).....	27
Tabel 3.3 Daftar Pernyataan Kuesioner System Usability Scale (SUS)	41
Tabel 4.1 Deskripsi Sistem	44
Tabel 4.2 Analisis Kebutuhan Fungsional	46
Tabel 4.3 Analisis Kebutuhan Non-Fungsional	47
Tabel 4.4 Kebutuhan Perangkat Lunak	47
Tabel 4.5 Kebutuhan perangkat Keras	48
Tabel 5. 1 Proses Pelatihan Model Peramalan	86
Tabel 5.2 Kode <i>Data Cleaning</i>	87
Tabel 5.3 Fungsi <i>Feature Engineering</i>	88
Tabel 5.4 Fungsi <i>Pipeline</i> Pelatihan (<i>default</i> xgboost)	90
Tabel 5.5 Peramalan <i>Batch</i> -Rekursif (<i>default</i> xgboost).....	90
Tabel 5.6 Kode Perhitungan Kebutuhan Bahan Baku	92
Tabel 5.7 Kode Prosesor <i>Bill of Material</i> (BOM)	94

BAB I PENDAHULUAN

1.1 Latar Belakang

Industri *Food and Beverage* (F&B) menghadapi paradoks pertumbuhan: ekspansi bisnis yang pesat beriringan dengan inefisiensi lingkungan yang masif. Pada tahun 2022, sektor layanan makanan dilaporkan mengakumulasi 290 juta ton limbah makanan secara global. Angka ini merepresentasikan hambatan struktural dalam pencapaian *Sustainable Development Goals* (SDG) 12.3 mengenai konsumsi dan produksi yang bertanggung jawab (United Nations Environment Programme, 2024). Di Indonesia, urgensi ini divalidasi oleh kajian yang mengidentifikasi bahwa titik krusial timbulan *Food Loss and Waste* (FLW) di Indonesia terjadi pada tahap konsumsi, di mana sektor HOREKA (Hotel, Restoran, Katering) berkontribusi sebesar 20% dari total limbah pangan non-rumah tangga (Bappenas, 2021). Fenomena ini memicu kerugian ganda: pemborosan material yang mendegradasi profitabilitas bisnis sekaligus memperburuk jejak karbon industri.

Kompleksitas manajemen inventaris teramplifikasi pada industri kedai kopi, di mana karakteristik bahan baku memiliki tingkat kerentanan dan *perishability* yang tinggi (Chae dkk., 2024; Hulaini dkk., 2025). Dalam iklim kompetisi yang semakin saturasi, margin keuntungan pelaku usaha tergerus tidak hanya oleh persaingan harga, tetapi juga oleh inefisiensi operasional internal yang tidak terdeteksi (González Morales & Caverio Rubio, 2023). Deviasi minor dalam perencanaan stok menghasilkan dampak ganda: kerugian material akibat pembusukan bahan baku (*overstock*) dan hilangnya kesempatan penjualan saat kehabisan stok (*stockout*) pada periode permintaan tinggi. Ketergantungan pada metode perencanaan konvensional yang berbasis intuisi manajerial terbukti rentan terhadap bias kognitif dan subjektivitas, menginduksi pengadaan bahan baku yang tidak akurat.

Mengatasi keterbatasan perencanaan konvensional tersebut, *Machine Learning* (ML) menawarkan pendekatan analitik prediktif yang lebih *robust*. Berbeda dengan teknik statistik klasik seperti ARIMA yang mensyaratkan stasioneritas data, algoritma ML seperti *Random Forest* (RF) dan *Extreme Gradient Boosting* (XGBoost) mampu menangkap pola non-linear kompleks tanpa asumsi

parametrik yang ketat (Schmidt dkk., 2022). Keunggulan teknis ML terletak pada kapabilitas mengolah variabel eksternal multidimensi, seperti hari libur, promosi, tren musiman yang sering luput dari peramalan manual. Studi empiris menunjukkan ML umumnya mengungguli metode *baseline* perusahaan dalam akurasi prediksi permintaan restoran (Chae dkk., 2024). Presisi peramalan berbasis ML memungkinkan optimalisasi pengadaan yang mereduksi *waste* secara signifikan tanpa mengorbankan tingkat layanan pelanggan (Rodrigues dkk., 2024).

Berbagai penelitian dalam lima tahun terakhir telah mengeksplorasi kapabilitas *machine learning* untuk mengatasi inefisiensi inventaris di industri F&B. Schmidt dkk. (2022) membuktikan bahwa algoritma *Random Forest* dan *XGBoost* mampu menangkap pola non-linear penjualan yang gagal dideteksi metode statistik klasik. Temuan ini diperkuat oleh Chae dkk. (2024) yang menunjukkan bahwa model ML secara konsisten mengungguli metode *baseline* rata-rata bergerak dalam memprediksi permintaan restoran. Dari aspek eksternal, Shah dkk. (2025) berhasil meningkatkan akurasi peramalan dengan mengintegrasikan variabel kalender dan tren musiman ke dalam model prediktif. Implementasi praktisnya dibuktikan oleh Rodrigues dkk. (2024), di mana optimalisasi pengadaan berbasis prediksi ML terbukti signifikan mereduksi volume limbah makanan. Sementara itu, Nanda Rosya (2024) menyoroti bahwa penerapan algoritma serupa pada UMKM lokal mampu memitigasi risiko *overstock* yang membebani modal kerja.

Husgendam Coffee, sebagai representasi kedai kopi skala menengah di Kota Batu, menghadapi tantangan fluktuasi permintaan harian dengan variabilitas tinggi namun masih mengandalkan sistem pengadaan manual berbasis estimasi kasar. Meskipun memiliki data historis transaksi yang tersimpan sistematis pada sistem *Point-of-Sale* (PoS), aset data ini belum dioptimalkan sebagai instrumen pengambilan keputusan strategis, fungsinya terbatas pada arsip administratif dan pelaporan keuangan. Observasi awal mengindikasikan ketidakakuratan prediksi yang bermuara pada dua risiko operasional: (1) *overstock* bahan baku *perishable* yang berkontribusi pada limbah organik dan pemborosan modal kerja, serta (2) *stockout* pada menu populer yang menurunkan kepuasan pelanggan dan menghilangkan potensi *revenue*. Kesenjangan antara ketersediaan data bervolume

besar dan praktik manajerial konvensional menciptakan peluang signifikan untuk intervensi teknologi berbasis ML.

Penelitian ini secara spesifik berfokus pada peramalan permintaan (*demand forecasting*) yang secara konseptual berbeda dengan peramalan penjualan (*sales forecasting*), meskipun data yang digunakan bersumber dari catatan transaksi penjualan historis sistem PoS. Perbedaan mendasar antara kedua pendekatan tersebut terletak pada entitas yang direpresentasikan, permintaan (*demand*) merepresentasikan kuantitas produk yang diinginkan atau diminta oleh pelanggan pada tingkat harga tertentu, terlepas dari ketersediaan stok, sedangkan penjualan (*sales*) hanya mencerminkan kuantitas yang benar-benar terealisasi dalam transaksi dan dibatasi oleh ketersediaan inventaris (Birkmaier dkk., 2024).

Dalam konteks manajemen inventaris industri F&B, memahami permintaan aktual memiliki nilai strategis yang lebih tinggi dibandingkan sekadar menganalisis pola penjualan historis. Hal ini disebabkan oleh fenomena *censored demand* atau *latent demand* (permintaan terpendam) yang terjadi ketika data penjualan pada periode *stockout* gagal menangkap permintaan pasar yang sesungguhnya. Sebagai ilustrasi, apabila menu "Black Ice" mengalami *stockout* pada hari Sabtu sore dengan penjualan tercatat sebesar 20 cup, angka yang dibatasi oleh ketersediaan stok, maka nilai tersebut tidak merepresentasikan permintaan aktual yang berpotensi mencapai 35-40 cup jika inventaris tersedia memadai. Kondisi ini mengindikasikan adanya kesenjangan antara *realized sales* dan *true market demand* yang tidak tertangkap dalam sistem pencatatan transaksi.

Dengan mengadopsi pendekatan peramalan permintaan, sistem yang dikembangkan dirancang untuk mengestimasi kebutuhan pasar yang sebenarnya (*true demand*), bukan sekadar mereplikasi pola penjualan masa lalu yang mungkin telah mengalami distorsi akibat keterbatasan stok. Pendekatan ini memungkinkan pembangkitan rekomendasi pengadaan yang lebih akurat dan responsif terhadap dinamika permintaan pasar, sehingga dapat meminimalkan risiko *stockout* sekaligus mengoptimalkan penggunaan inventaris untuk mengurangi *food waste*.

Merespons kesenjangan antara ketersediaan data historis dan kebutuhan efisiensi operasional tersebut, penelitian ini difokuskan pada pengembangan model prediksi permintaan harian menggunakan pendekatan ML. Melalui komparasi

performa algoritma *Random Forest* dan *XGBoost*, penelitian ini berupaya menghasilkan rekomendasi pengadaan bahan baku yang presisi. Pendekatan ini diharapkan dapat menjadi solusi kuantitatif untuk meminimalkan risiko *overstock* dan *stockout*, sekaligus mengoptimalkan manajemen inventaris Husgendam Coffee yang selama ini masih berbasis intuisi.

Pemilihan *Random Forest* dan *XGBoost* didasarkan pada tiga pertimbangan teknis: kemampuan menangani *non-linearity* tanpa *feature engineering* kompleks, *robustness* terhadap *outlier* yang umum pada data transaksi F&B, dan interpretabilitas model yang mendukung justifikasi keputusan bisnis (Alt, 2021; González Morales & Caverio Rubio, 2023). Secara praktis, penelitian ini diharapkan menjadi *blueprint* implementasi sistem prediksi berbasis ML untuk UMKM F&B dengan karakteristik serupa, berkontribusi pada transformasi digital yang *sustainable* dan *data-driven*. Secara akademis, penelitian ini memperkaya literatur tentang aplikasi ML pada konteks bisnis skala menengah di Indonesia, khususnya dalam industri *coffee shop* yang mengalami pertumbuhan pesat namun masih minim adopsi teknologi analitik prediktif.

Berdasarkan latar belakang permasalahan tersebut, penulis menyusun skripsi dengan judul "Penerapan Algoritma *Machine Learning* dalam Peramalan Permintaan Produk pada *Coffee Shop* Berbasis Data Historis Transaksi". Melalui penelitian ini, diharapkan dapat menghadirkan solusi prediktif yang aplikatif untuk meminimalkan inefisiensi inventaris di Husgendam Coffee, sekaligus memberikan kontribusi nyata dalam literatur akademik mengenai transformasi manajemen operasional UMKM berbasis *data-driven decision making*.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah yang dapat diambil adalah sebagai berikut:

1. Apakah algoritma *XGBoost* dapat diterapkan untuk memprediksi permintaan menu harian di Husgendam Coffee dengan tingkat akurasi yang optimal?
2. Apakah penerapan model prediksi berbasis *machine learning* dapat mengurangi tingkat ketidaktepatan perencanaan stok secara signifikan

jika dibandingkan dengan metode konvensional (intuisi & perkiraan kasar) yang saat ini diterapkan?

3. Bagaimana tingkat *usability* (kegunaan) sistem aplikasi peramalan yang dibangun bagi pengguna operasional di Husgendam Coffee berdasarkan pengukuran *System Usability Scale* (SUS) dengan target skor minimal kategori *acceptable* (>70)?

1.3 Batasan Masalah

Adapun batasan dalam penelitian ini meliputi

1. Objek penelitian adalah Husgendam Coffee dengan *dataset* transaksi periode 1 Januari 2022 hingga 25 Mei 2026 yang terdiri dari 72,771 data item terjual.
2. Luaran sistem difokuskan pada rekomendasi kebutuhan produksi (*Gross Requirement*) berdasarkan prediksi permintaan, belum memperhitungkan posisi stok gudang (*Net Requirement*).
3. Data resep diasumsikan statis selama periode penelitian dan konversi kebutuhan bahan baku tidak memperhitungkan sisa stok di gudang
4. Hasil peramalan difokuskan untuk memperkirakan kebutuhan produksi dan tidak mencakup aspek harga bahan, pengadaan, maupun distribusi logistik.

1.4 Tujuan

Tujuan dari penelitian ini adalah sebagai berikut:

1. Menghasilkan model *machine learning* yang mampu memprediksi permintaan menu harian dengan memanfaatkan pola data historis.
2. Membangun modul konversi otomatis yang menerjemahkan prediksi penjualan menjadi rekomendasi kuantitas bahan baku.
3. Menganalisis efektivitas model dalam mengurangi tingkat kesalahan prediksi dibandingkan metode konvensional.

1.5 Manfaat

Manfaat dari penelitian ini adalah sebagai berikut:

1. Manfaat bagi mitra

- a. Memberikan kemudahan dalam penentuan jumlah stok dengan menyediakan acuan berbasis data bagi manajemen Husgendam Coffee, guna mendukung pengambilan keputusan yang lebih terukur.
 - b. Membantu mengurangi kehilangan potensi pendapatan akibat kehabisan stok pada momen permintaan tinggi
 - c. Menyediakan alat bantu pengambil keputusan yang objektif untuk perencanaan stok
2. Manfaat bagi penulis
- a. Mengimplementasikan ilmu dan teori yang telah dipelajari selama perkuliahan, khususnya dalam bidang kecerdasan buatan dan pengembangan sistem, ke dalam penyelesaian masalah nyata di industri.

BAB II LANDASAN TEORI

2.1 Studi Literatur

Beberapa penelitian yang dijadikan rujukan dalam penelitian ini adalah sebagai berikut:

Schmidt dkk. (2022) dalam penelitiannya yang berjudul "*Machine Learning Based Restaurant Sales Forecasting*" membuktikan efektivitas *machine learning* dalam meramalkan penjualan restoran. Menggunakan *dataset* PoS selama tiga tahun (2016-2019), penelitian tersebut menguji lebih dari 20 model, termasuk *Linear Models*, *Ensemble Methods*, dan *Recurrent Neural Networks*. Hasil penelitian menunjukkan bahwa model RNN seperti *Long Short-Term Memory* menghasilkan *Symmetric Mean Absolute Percentage Error* (sMAPE) sebesar 19,5 persen untuk peramalan multi-hari, sementara model linear seperti *Ridge Regression* menghasilkan performa terbaik untuk peramalan satu hari dengan sMAPE 19,6 persen. Temuan penting dari penelitian ini adalah bahwa model *ensemble methods* seperti *Random Forest* menunjukkan performa yang *robust* dengan kemampuan menangkap pola non-linear yang gagal dideteksi oleh metode statistik klasik.

Panda & Mohanty (2023) dalam penelitian "*Time Series Forecasting and Modeling of Food Demand Supply Chain Based on Regressors Analysis*" melakukan perbandingan komprehensif tujuh algoritma regresi. Menggunakan fitur *lag* dan *Exponentially Weighted Moving Average*, penelitian ini membandingkan algoritma seperti *Random Forest*, *XGBoost*, *CatBoost*, *LightGBM*, dan LSTM. Hasilnya mengungkapkan bahwa model LSTM menunjukkan superioritas dibandingkan algoritma lain dengan *Mean Absolute Percentage Error* (MAPE) 6,56 persen. Namun demikian, penelitian ini juga mencatat bahwa *tree-based ensemble methods* seperti *Random Forest* dan *XGBoost* menunjukkan *trade-off* yang lebih baik antara akurasi dan interpretabilitas model, faktor yang krusial untuk pengambilan keputusan dalam bisnis.

Studi spesifik pada gerai F&B dilakukan oleh Groene & Zakharov (2024) dengan judul "*Introduction of AI-based Sales Forecasting: How to Drive Digital Transformation in Food and Beverage Outlets*". Penelitian ini menunjukkan bahwa penggunaan *XGBoost* mampu mengurangi *Root Mean Squared Error* (RMSE)

sebesar 22-33 persen dan *Mean Absolute Error* (MAE) sebesar 19-31 persen dibandingkan peramalan heuristik manual. Model ini mengintegrasikan data penjualan internal dengan faktor eksternal seperti cuaca, hari libur, dan *event*. Kontribusi signifikan penelitian ini adalah demonstrasi implementasi praktis yang membuktikan bahwa adopsi ML *forecasting* dapat langsung diterjemahkan menjadi *cost reduction* dan *service level improvement* pada operasional F&B.

Mitra dkk. (2022) dalam "*A Comparative Study of Demand Forecasting Models for a Multi-Channel Retail Company*" membandingkan metode ML untuk peramalan permintaan ritel menggunakan data Walmart. Penelitian ini mengungkapkan bahwa *XGBoost* mengungguli *Random Forest standalone* dengan R^2 Score 0,9547, sedangkan model hibrida RF-*XGBoost*-LR mencapai skor yang sedikit lebih tinggi yaitu 0,9551. Temuan ini mengindikasikan bahwa meskipun *XGBoost* secara umum lebih superior, peningkatan margin terhadap *Random Forest* relatif kecil pada beberapa konteks, yang menjustifikasi eksplorasi kedua algoritma dalam penelitian ini.

Nasseri dkk. (2023) membandingkan metode *tree-based ensemble* dan *deep learning* untuk produk *perishable*. Hasil penelitian menunjukkan bahwa empat model *tree-based ensemble* (*Extra Trees*, *Random Forest*, *Gradient Boosting*, *XGBoost*) menunjukkan performa yang sebanding dengan LSTM, di mana *XGBoost* menunjukkan sedikit keunggulan dengan R^2 berkisar 0,54-0,60 pada *dataset* yang kompleks. Penelitian ini memperkuat argumen bahwa untuk *dataset* dengan ukuran terbatas dan karakteristik *non-sequential* yang dominan, metode *tree-based* dapat menjadi alternatif efektif terhadap *deep learning* dengan keuntungan tambahan berupa *lower computational cost* dan *higher interpretability*.

Mustapha & Sithole (2025) dalam "*Forecasting Retail Sales Using Machine Learning Models*" menunjukkan bahwa *XGBoost* muncul sebagai algoritma dengan performa terbaik di semua metrik (RMSE, MAE, R^2), diikuti oleh LSTM, *Random Forest*, ARIMA, dan *Linear Regression*, khususnya pada *dataset* dengan musiman dan efek promosi. Penelitian ini memberikan bukti terbaru yang konsisten dengan tren literatur bahwa *XGBoost* merupakan *state-of-the-art* untuk masalah *tabular forecasting*.

Terkait pengurangan limbah pangan, Rodrigues dkk. (2024) membuktikan bahwa optimalisasi pengadaan berbasis prediksi ML dapat mereduksi volume limbah makanan sebesar 14-52 persen dibandingkan estimasi manual. Hal senada diungkapkan Turker (2025), di mana integrasi model ML (*Lasso Regression* dan *XGBoost*) di kantin kampus berhasil mengurangi *food waste* sebesar 28 persen. Kedua penelitian ini menegaskan bahwa akurasi *forecasting* bukan sekadar metrik akademik, melainkan dampak langsung ke keberlanjutan dan kondisi finansial.

Dalam konteks UMKM di Indonesia, studi masih terbatas. Nanda Rosya (2024) menerapkan algoritma *machine learning* untuk optimalisasi stok pada UMKM lokal dan menyoroti bahwa penerapan algoritma serupa mampu memitigasi risiko *overstock* yang membebani modal kerja. Namun demikian, penelitian ini tidak secara spesifik mengeksplorasi *robustness* model terhadap perubahan struktur seperti *rebranding* atau perubahan skala operasional. Mutiarachim dkk. (2025) menggunakan algoritma *Decision Tree* ID3 untuk memprediksi keberhasilan transformasi digital UMKM dengan akurasi 89,19 persen, meskipun penelitian ini lebih berfokus pada faktor kesiapan digital daripada peramalan permintaan kuantitatif.

Berdasarkan tinjauan literatur di atas, teridentifikasi empat kesenjangan penelitian yang menjadi motivasi penelitian ini. Pertama, mayoritas penelitian *forecasting* berbasis ML (Groene & Zakharov, 2024; Mustapha & Sithole, 2025; Schmidt dkk., 2022) fokus pada rantai restoran besar dengan infrastruktur IT matang, sementara karakteristik operasional UMKM Indonesia yang menghadapi *resource constraints* dan *operational volatility* belum dieksplorasi secara mendalam, terutama dalam konteks *structural break* akibat *rebranding* yang dialami Husgendam Coffee. Kedua, studi-studi terdahulu (Rodrigues dkk., 2024; Turker, 2025) fokus pada prediksi penjualan menu sebagai *end-output* tanpa menerjemahkan peramalan tersebut ke *gross requirement* bahan baku, padahal *translation* dari prediksi unit penjualan ke kebutuhan bahan baku spesifik merupakan langkah kritical untuk pengambilan keputusan pada UMKM F&B yang selama ini belum dibahas dalam literatur. Ketiga, sebagian besar penelitian menggunakan *dataset* dengan *historical period* yang relatif stabil tanpa gangguan besar, sehingga kondisi *rebranding* dengan perubahan nama bisnis, *repositioning*

pasar, dan menu *restructuring* yang menciptakan *structural break* dalam *time series* data menjadi *test case* menarik untuk mengevaluasi kemampuan beradaptasi model ML terhadap kondisi *non-stationary*. Penelitian ini berupaya mengisi kesenjangan tersebut dengan mengimplementasikan dan membandingkan *Random Forest* dan *XGBoost* pada konteks UMKM *coffee shop* dengan karakteristik permintaan yang fluktuatif, mengintegrasikan *output* peramalan dengan konversi resep untuk menghasilkan rekomendasi pengadaan yang dapat ditindaklanjuti, mengevaluasi performa model pada *dataset* yang mengandung *structural break*, dan menyediakan *framework* implementasi yang dapat diaplikasikan untuk UMKM F&B dengan batasan sumber daya serupa.

2.2 Dasar Teori

2.2.1 Peramalan Permintaan

2.2.1.1 Definisi dan Komponen

Peramalan permintaan merupakan salah satu aspek paling kritis dalam manajemen rantai pasok, karena menentukan keputusan terkait pengadaan, perencanaan produksi, dan manajemen inventaris (Allam dkk., 2025). *Time series forecasting* melibatkan analisis data yang berevolusi sepanjang waktu dan memanfaatkan model statistik untuk membuat prediksi tentang pola dan tren masa depan (Kramar & Alchakov, 2023).

Data *time series* dalam *forecasting* terdiri dari empat komponen fundamental (Hao & Liu, 2024). Pertama, tren merepresentasikan pola jangka panjang yang menunjukkan kecenderungan naik atau turun, seperti pertumbuhan *customer base* pada *coffee shop*. Kedua, musiman adalah pola berulang dalam periode tetap yang terjadi pada interval yang sudah ditentukan, misalnya *weekend spike* atau *lunch hour peak* pada industri F&B (Kramar & Alchakov, 2023). Ketiga, siklus merupakan fluktuasi jangka panjang tanpa periode tetap yang biasanya terkait dengan siklus ekonomi, berbeda dengan musiman yang memiliki durasi yang sudah ditentukan (Hao & Liu, 2024). Keempat, *irregular* atau *noise* adalah variasi *random* yang tidak dapat diprediksi, disebabkan oleh *events* tidak terduga seperti cuaca ekstrem atau gangguan operasional (Kramar & Alchakov, 2023).

2.2.1.2 Karakteristik Permintaan di Industri F&B

Permintaan di kedai kopi memiliki karakteristik unik yang membedakannya dari retail atau *manufacturing* tradisional, sehingga memerlukan pendekatan *forecasting* yang spesifik. Karakteristik pertama adalah *high volatility*, di mana fluktuasi permintaan harian dapat sangat signifikan. Perbedaan antara pola *weekday* dan *weekend* sangat mencolok, demikian pula variasi dalam satu hari yang dipengaruhi oleh *breakfast rush*, *lunch peak*, dan *afternoon lull*. Volatilitas ini membuat metode *simple averaging* tidak efektif karena gagal menangkap pola dinamik (Birkmaier dkk., 2024).

Karakteristik kedua adalah *multiple seasonality*, yaitu keberadaan pola musiman pada berbagai tingkat temporal secara simultan. Pada level *intra-day*, terdapat *morning rush hour*, *lunch time peak*, dan *afternoon surge* yang menciptakan fluktuasi permintaan sepanjang hari operasional. Pada level *intra-week*, terdapat pola konsisten dimana *weekend* menunjukkan volume lebih tinggi daripada *weekday*, dengan transisi bertahap menuju akhir pekan. Pada level *intra-year*, terdapat variasi musiman seperti peningkatan penjualan pada musim liburan, periode ujian untuk kedai kopi dekat kampus, atau penurunan selama periode bulan puasa di area tertentu (Birkmaier dkk., 2024).

Karakteristik ketiga adalah efek promosi, di mana diskon, promo *buy-one-get-one*, atau penawaran spesial menyebabkan lonjakan permintaan yang bersifat *non-linear* dan sulit diprediksi dengan metode konvensional. Efek dari promosi tidak hanya terjadi pada hari promosi tersebut, tetapi juga menciptakan *pre-promotion dip* dan *post-promotion slump*. Karakteristik keempat adalah sensitivitas terhadap faktor eksternal seperti cuaca, *event* lokal yang meningkatkan *footfall traffic*, serta aksi kompetitor yang dapat menggerus *market share* (Birkmaier dkk., 2024).

Karakteristik kelima yang spesifik adalah batasan ketahanan (*perishability constraints*), yaitu bahan baku kedai kopi seperti susu segar, *pastry*, dan *produce* memiliki masa simpan terbatas sehingga *overstock* langsung berujung pada *waste* (Birkmaier dkk., 2024). Berbeda dengan retail yang dapat menahan *inventory* lebih lama, F&B menghadapi *trade-off* tajam antara *stockout cost* (*lost sales* dan *customer dissatisfaction*) dan *obsolescence cost* (*food waste* dan *capital tied up*).

Karakteristik ini membuat akurasi *forecasting* menjadi faktor kritikal untuk keuntungan operasional.

2.2.1.3 Pendekatan Peramalan

Metode *forecasting* dapat dikategorikan ke dalam tiga pendekatan utama. Pendekatan kualitatif mengandalkan penilaian dan opini ahli, digunakan ketika data historis tidak tersedia tetapi rentan terhadap bias subjektif dan sulit diukur (Zellner dkk., 2021). Pendekatan kuantitatif tradisional menggunakan teknik statistik seperti *Moving Average*, *Exponential Smoothing*, dan ARIMA untuk menganalisis data historis. *Moving Average* memperhalus fluktuasi *random* tetapi cenderung *lag* terhadap perubahan tren. *Exponential Smoothing* memberikan bobot yang lebih besar pada observasi terbaru, dengan varian *Holt-Winters* yang mampu menangani data musiman. ARIMA sangat baik untuk deret waktu teratur dengan kemampuan menangani tren dan musiman melalui *differencing* (Kontopoulou dkk., 2023).

Keterbatasan mendasar dari metode tradisional adalah asumsi linearitas dan stasioneritas yang sering kali tidak terpenuhi pada data F&B di dunia nyata. ARIMA memerlukan data yang stasioner, yang tidak selalu dapat dicapai pada data dengan *structural breaks* seperti *rebranding*. Metode ini juga kesulitan dalam mengintegrasikan beberapa variabel eksternal secara simultan serta memerlukan penyetelan parameter yang ekstensif, yang menjadi hambatan adopsi bagi UMKM dengan sumber daya teknis terbatas (Kontopoulou dkk., 2023).

Pendekatan *Machine Learning* menawarkan pergeseran paradigma dengan beberapa keunggulan mendasar. Pertama, algoritma ML seperti *Random Forest* dan *XGBoost* bersifat non-linear secara inheren, sehingga mampu menangkap interaksi kompleks tanpa spesifikasi manual. Kedua, ML memfasilitasi *feature engineering* yang lebih canggih, di mana variabel eksternal (seperti indikator kalender, cuaca, aktivitas pemasaran) dapat diintegrasikan sebagai fitur *input*. Model *three-based* secara alami mampu menangani tipe data campuran tanpa *preprocessing* yang ekstensif (Groene & Zakharov, 2024; Schmidt dkk., 2022). Ketiga, model ML lebih *robust* terhadap *outlier* dan data hilang, *Random Forest* menggunakan mekanisme voting dari *multiple trees* dan dapat menangani nilai hilang melalui *surrogate splits* tanpa penurunan akurasi yang signifikan.

Schmidt dkk. (2022) membuktikan bahwa metode *ensemble* menghasilkan akurasi 15-20 persen lebih tinggi dibandingkan peramalan tradisional. Groene & Zakharov (2024) mendemonstrasikan bahwa *XGBoost* mengurangi kesalahan prediksi sebesar 22-33 persen dalam RMSE dan 19-31 persen dalam MAE. (Rodrigues dkk., 2024) membuktikan bahwa optimasi pengadaan berbasis ML dapat mengurangi *food waste* hingga 52 persen, yang secara langsung menerjemahkan peningkatan akurasi menjadi nilai bisnis yang nyata.

2.2.2 Machine Learning untuk Peramalan

Machine Learning didefinisikan sebagai bidang studi yang memberi komputer kemampuan untuk belajar tanpa diprogram secara eksplisit. Dalam konteks *forecasting*, ML dapat didefinisikan sebagai aplikasi algoritma dan teknik komputasi yang memungkinkan sistem untuk secara otomatis belajar pola dari data historis dan menggunakan pola tersebut untuk membuat prediksi terhadap data baru di masa depan (Douaioui dkk., 2024).

Salah satu pendekatan utama dalam ML adalah regresi, yaitu teknik kuantitatif yang digunakan untuk menentukan hubungan kausal antara variabel dependen (target) dan variabel independen (fitur atau prediktor) (Mitra dkk., 2022). Berbeda dengan klasifikasi yang memprediksi kategori diskrit, algoritma regresi bertujuan untuk memprediksi nilai *output* yang bersifat kontinu atau numerik (Cholissodin Imam dkk., 2020; Tuays Almuqati dkk., 2024). Teknik ini, seperti regresi linier, memungkinkan sistem untuk membangun model persamaan melalui kombinasi linier dari fitur-fitur masukan guna meminimalkan selisih antara nilai aktual dan hasil prediksi,. Dalam praktiknya, metode regresi sering dianggap lebih sistematis dan dapat diandalkan dibandingkan metode deret waktu tradisional karena fleksibilitasnya dalam mempertimbangkan berbagai variabel yang mempengaruhi hasil (Mitra dkk., 2022).

Proses pembelajaran dalam ML fokus pada pengembangan sistem yang mampu belajar secara mandiri tanpa harus berulang kali diprogram oleh manusia. ML membutuhkan data sebagai bahan belajar pada tahap pelatihan (*training*) sebelum digunakan untuk pengujian (*testing*) yang dapat menghasilkan *output* yang optimal (Cholissodin Imam dkk., 2020). Dalam konteks *demand forecasting*, data historis transaksi penjualan digunakan sebagai *training* data di mana model belajar

hubungan antara *input features* (tanggal, hari, fitur temporal) dan target variabel (jumlah penjualan), kemudian menggunakan pola belajar tersebut untuk memprediksi penjualan di masa depan.

2.2.2.1 *Supervised Learning* untuk Peramalan Permintaan

Machine Learning diklasifikasikan menjadi empat kategori utama: *supervised learning*, *unsupervised learning*, *semi-supervised learning*, dan *reinforcement learning* (Tuays Almuqati dkk., 2024). Penelitian ini secara spesifik menggunakan *supervised learning*, sehingga pembahasan akan difokuskan pada kategori ini.

Supervised learning atau pembelajaran terbimbing adalah paradigma pembelajaran mesin yang mengasumsikan fungsi dari data pelatihan berlabel. Dalam *supervised learning*, setiap sampel data terdiri dari *input features* (variabel independen) dan *corresponding output* atau target (variabel dependen). Algoritma *supervised learning* belajar untuk memetakan *input* ke *output* berdasarkan *labeled examples*, kemudian menggunakan *mapping function* tersebut untuk memprediksi *output* dari *input* baru yang belum pernah dilihat sebelumnya (Tuays Almuqati dkk., 2024).

Supervised learning dipilih untuk peramalan permintaan karena beberapa alasan mendasar. Pertama, peramalan permintaan pada hakikatnya merupakan masalah *supervised learning* di mana kita memiliki data historis berlabel: setiap rekaman memiliki fitur *input* (tanggal, hari, fitur *lag*) dan target yang sesuai (penjualan aktual pada tanggal tersebut). Kedua, struktur data deret waktu dapat ditransformasi menjadi format *supervised learning* melalui teknik *windowing* atau pembuatan *lag*, di mana nilai-nilai observasi masa lalu menjadi fitur untuk memprediksi nilai di masa depan. Ketiga, *supervised learning* telah terbukti secara empiris lebih unggul untuk tugas peramalan dibandingkan pendekatan *i* atau *reinforcement learning*, sebagaimana didemonstrasikan dalam berbagai studi (Panda & Mohanty, 2023; Schmidt dkk., 2022).

Supervised learning bekerja dalam dua kategori berbeda: *regression* dan *classification*. *Regression* bekerja pada data kontinu yang mendefinisikan hubungan antara variabel dependen dan independen. Variabel dependen disebut dengan variabel target yang sifatnya bergantung pada kumpulan data *input features*.

Variabel independen (*features*) digunakan untuk memprediksi nilai variabel dependen. Model regresi umumnya digunakan untuk memprediksi nilai numerik kontinu seperti harga rumah, harga saham, volume penjualan, atau dalam konteks penelitian ini, jumlah unit menu yang terjual per hari (Tuays Almuqati dkk., 2024).

Klasifikasi, di sisi lain, bekerja pada data kategori untuk membagi data ke dalam *discrete classes* atau *categories*. Teknik klasifikasi dapat berupa *binary classification* (membagi data dalam dua kelas, biasanya 0 dan 1) atau *multiclass classification* (membagi data menjadi lebih dari dua kelas). Contoh aplikasi *classification* adalah *spam detection*, *sentiment analysis*, atau *customer churn prediction*. Dalam konteks *forecasting*, *classification* dapat digunakan untuk memprediksi kategori *demand* (*low*, *medium*, *high*) namun *approach* ini *less common* dibandingkan *regression* karena hilangnya informasi *granular* tentang *magnitude actual demand*.

Penelitian ini menggunakan pendekatan regresi karena *output* yang dibutuhkan adalah *continuous numerical values* (jumlah unit penjualan per menu per hari), bukan *categorical predictions*. *Regression models* memungkinkan pengguna untuk mendapatkan perkiraan jumlah yang tepat yang dapat langsung diterjemahkan ke jumlah pengadaan, berbeda dengan klasifikasi yang hanya memberikan kategori luas.

2.2.3 Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting atau *XGBoost* merupakan algoritma *machine learning* berbasis *ensemble learning* yang menggunakan teknik *gradient boosting*. *XGBoost* dikembangkan oleh Chen & Guestrin (2016) sebagai implementasi yang dioptimalkan dari *gradient boosted decision trees*. *XGBoost* bekerja berdasarkan prinsip *boosting*, yaitu membangun model secara iteratif dimana setiap model baru berusaha memperbaiki kesalahan yang dibuat oleh model sebelumnya.

Dalam *XGBoost*, proses *training* dimulai dengan model awal (biasanya prediksi konstan), kemudian secara iteratif menambahkan pohon baru yang memprediksi *residual error* (kesalahan) dari model sebelumnya. Setiap pohon baru di-*train* untuk memperbaiki kesalahan yang masih tersisa, bukan untuk memprediksi target *original*. Pendekatan ini membuat *XGBoost* sangat *powerful* karena fokus pada area dimana model masih lemah.

Fungsi objektif *XGBoost* terdiri dari *loss function* dan *regularization* term yang dapat diformulasikan sebagai berikut:

$$Obj(\theta) = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (6)$$

dimana:

- $L(y_i, \hat{y}_i)$ adalah loss function yang mengukur perbedaan antara nilai prediksi ($\hat{y}_i$) dan nilai aktual (y_i). Untuk regresi biasanya menggunakan squared error: $L = (y_i - \hat{y}_i)^2$. Untuk klasifikasi binary menggunakan logistic loss (log loss).
- $\Omega(f_k)$ adalah regularization term untuk mengontrol kompleksitas model dan mencegah *overfitting*. Regularisasi ini menghukum model yang terlalu kompleks.
- K adalah jumlah total pohon dalam ensemble.
- n adalah jumlah sampel data dalam training set.

Regularization term didefinisikan sebagai:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (7)$$

- T adalah jumlah daun (leaves) dalam pohon. Semakin banyak daun, semakin kompleks pohon tersebut.
- w_j adalah skor (*weight*) pada daun ke- j , yang merepresentasikan prediksi *output* untuk sampel yang jatuh ke daun tersebut.
- γ (gamma) adalah parameter regularisasi yang mengontrol jumlah daun. Nilai γ yang lebih besar membuat model lebih konservatif dalam menambah daun baru.
- λ (lambda) adalah parameter L2 regularization pada weights. Nilai λ yang lebih besar membuat weight cenderung lebih kecil (mendekati nol), mengurangi *overfitting*.

Regularization term ini membuat *XGBoost* berbeda dari *gradient boosting* tradisional. γT menghukum complexity (jumlah leaves), sedangkan $\frac{1}{2} \lambda \sum w_j^2$ menghukum magnitude dari weights (L2 regularization seperti Ridge regression).

Proses *training XGBoost* menggunakan *gradient descent* untuk meminimalkan fungsi objektif secara *iteratif*. Pada setiap iterasi t , model baru f_t ditambahkan untuk memperbaiki prediksi:

$$\widehat{y}_i^{(t)} = \widehat{y}_i^{(t-1)} + \eta \cdot f_t(x_i) \quad (8)$$

dimana:

- $\widehat{y}_i^{(t)}$ adalah prediksi pada iterasi ke- t untuk sampel i .
- $\widehat{y}_i^{(t-1)}$ adalah prediksi kumulatif dari semua pohon sebelumnya (iterasi 0 hingga $t-1$).
- $f_t(x_i)$ adalah prediksi dari pohon baru yang ditambahkan pada iterasi t untuk sampel i . Pohon ini di-train untuk memprediksi residual (kesalahan) dari model sebelumnya.
- η (eta) adalah learning rate yang mengontrol kontribusi setiap pohon baru. Nilai η yang kecil (misalnya 0.01 - 0.3) membuat learning lebih lambat tetapi lebih *robust* terhadap *overfitting*. Ini adalah bentuk shrinkage yang membuat setiap pohon berkontribusi sedikit demi sedikit, bukan langsung kontribusi penuh.

Proses iteratif ini dapat dijelaskan sebagai berikut

1. Iterasi 0: Mulai dengan prediksi awal $\widehat{y}^{(0)}$ (biasanya nilai konstan seperti mean untuk regresi atau log odds untuk klasifikasi).
2. Iterasi 1: Hitung residual $r_1 = y - \widehat{y}^{(0)}$. Train pohon pertama f_1 untuk memprediksi residual ini. Update prediksi: $\widehat{y}^{(1)} = \widehat{y}^{(0)} + \eta \cdot f_1(x)$.
3. Iterasi 2: Hitung residual baru $r_2 = y - \widehat{y}^{(1)}$. Train pohon kedua f_2 untuk memprediksi residual ini. Update prediksi: $\widehat{y}^{(2)} = \widehat{y}^{(1)} + \eta \cdot f_2(x)$.

4. Iterasi t : Proses terus berlanjut hingga mencapai jumlah pohon yang ditentukan ($n_estimators$) atau *improvement* dalam *loss function* sudah tidak signifikan (*early stopping*).

Setiap pohon baru fokus pada memperbaiki kesalahan yang masih tersisa dari *ensemble* sebelumnya. Ini membuat *XGBoost* sangat efektif dalam menangkap pola kompleks dalam data. Salah satu inovasi kunci *XGBoost* adalah algoritma untuk menemukan *split* terbaik. Untuk setiap *node* dalam pohon, *XGBoost* mengevaluasi semua *possible splits* untuk semua fitur yang dipertimbangkan. Gain untuk melakukan *split* dihitung sebagai:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (9)$$

dimana:

- G_L dan G_R adalah sum of gradients untuk left dan right child setelah split.
- H_L dan H_R adalah sum of hessians (second order gradients) untuk left dan right child.
- γ adalah regularization parameter yang menghukum penambahan split baru.

Split hanya dilakukan jika $Gain > 0$, artinya benefit dari split lebih besar dari cost regularization.

XGBoost memiliki banyak *hyperparameter* yang dapat di-tune untuk mengoptimalkan performa:

1. *Tree-Spesific Parameters*:

- *max_depth*: Kedalaman maksimum setiap pohon. Membatasi *depth* mencegah *overfitting*. *Default* biasanya 6. *Range*: 3-10.
- *Min_child_weight*: Minimum sum of instance weight (hessian) dalam *child node*. Semakin besar nilai ini, semakin konservatif algoritma. *Default* 1.
- *gamma*: Minimum *loss reduction* untuk membuat *split* baru. Semakin besar γ , semakin konservatif. *Default* 0.

2. *Boosting Parameters*

- *n_estimators*: Jumlah *boosting rounds* (jumlah pohon yang akan dibangun). Lebih banyak pohon umumnya lebih baik tetapi meningkatkan risiko *overfitting* dan *computational cost*. *Range*: 100-1000.
- *learning_rate (eta)*: *Shrinkage* untuk mencegah *overfitting*. Setiap pohon berkontribusi $\eta \times$ prediksi. Default 0.3. *Range*: 0.01-0.3. Nilai lebih kecil membutuhkan lebih banyak pohon tetapi lebih robust.

3. *Sampling Parameters*

- *subsample*: Rasio *subsample* dari *training* data untuk setiap pohon. Nilai 0.8 berarti 80% data dipilih *random* untuk setiap *tree*. Default 1.0. *Range*: 0.5-1.0. Nilai < 1 mencegah *overfitting*.
- *colsample_bytree*: Rasio fitur yang di-*sample* untuk setiap pohon. Nilai 0.8 berarti 80% fitur dipilih *random*. Default 1.0. *Range*: 0.5-1.0.
- *colsample_bylevel*: Rasio fitur yang di-*sample* untuk setiap level (*depth*) dalam pohon. Default 1.0.

4. *Regularization Parameters*

- *lambda (reg_lambda)*: L2 *regularization term on weights*. Semakin besar λ , semakin smooth model (weights mendekati nol). Default 1.
- *alpha (reg_alpha)*: L1 *regularization term on weights*. Dapat membuat *weights* benar-benar nol (*feature selection*). Default 0.

2.2.4 Validasi Silang Data Deret Waktu (*Time Series Cross-Validation*)

Validasi silang konvensional (*k-fold cross-validation*) tidak dapat diterapkan secara langsung pada data deret waktu karena melanggar asumsi temporal *ordering*. Pada *k-fold* standar, data diacak (*shuffled*) sebelum dibagi ke dalam *fold*, yang menyebabkan data masa depan dapat masuk ke *training set* sementara data masa lalu masuk ke *test set*. Kondisi ini menghasilkan *data leakage* yang menyebabkan evaluasi performa model menjadi tidak realistis. Untuk mengatasi keterbatasan tersebut, dikembangkan strategi validasi silang khusus untuk deret waktu yang mempertahankan urutan kronologis data. Terdapat dua pendekatan utama yang umum digunakan.

Pertama, *sliding window approach* membagi data menjadi *training window* dengan ukuran tetap yang bergeser maju seiring waktu. Pada setiap *fold*, *training set* mencakup periode waktu yang sama panjangnya, dan *test set* adalah periode

berikutnya. Pendekatan ini cocok untuk situasi di mana pola lama dianggap kurang relevan.

Kedua, *expanding window approach* memulai dengan *training set* minimal kemudian secara progresif menambahkan data baru ke *training set* pada setiap *fold* berikutnya. *Training window* terus berkembang sementara *test window* tetap berukuran konsisten. Pendekatan ini sesuai untuk situasi di mana seluruh data historis dianggap informatif dan mengakomodasi kemungkinan adanya *structural breaks* dalam data.

Pemilihan antara kedua pendekatan bergantung pada karakteristik data dan asumsi tentang stasioneritas pola. Untuk data dengan *structural breaks* seperti *rebranding*, *expanding window* lebih sesuai karena memungkinkan model belajar dari perubahan pola secara *incremental*.

2.2.5 Evaluasi Kinerja Model Peramalan

Untuk mengukur tingkat akurasi model prediksi, digunakan metrik evaluasi yang membandingkan nilai aktual dengan nilai prediksi.

1. Mean Absolute Error (MAE)

MAE mengukur rata-rata selisih absolut antara nilai prediksi dan nilai aktual. Metrik ini memberikan gambaran besarnya kesalahan prediksi dalam satuan yang sama dengan data asli, sehingga mudah dipahami secara langsung.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (10)$$

2. Root Mean Squared Error (RMSE)

RMSE mengukur akar rata-rata kuadrat kesalahan. Metrik ini memberikan bobot lebih besar pada kesalahan yang bernilai besar (*outlier*), sehingga cocok digunakan jika kesalahan prediksi yang ekstrem sangat dihindari.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (11)$$

3. Mean Absolute Percentage Error (MAPE)

Weighted Mean Absolute Percentage Error (wMAPE) merupakan pengembangan dari MAPE yang dirancang khusus untuk mengatasi masalah pembagian dengan nol (division by zero) yang terjadi ketika nilai aktual (y_i) bernilai nol. Metrik ini mengukur total kesalahan absolut dibagi dengan total permintaan aktual. wMAPE memberikan gambaran persentase kesalahan yang dibobotkan oleh volume penjualan, sehingga memberikan evaluasi yang lebih stabil untuk data yang jarang (sparse) atau memiliki fluktuasi tinggi.

$$wMAPE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n y_i} \times 100\% \quad (12)$$

4. Koefisien Determinasi (R^2)

R^2 mengukur seberapa baik variasi data dapat dijelaskan oleh model. Nilai R^2 berkisar antara 0 hingga 1, di mana nilai mendekati 1 menunjukkan model yang sangat baik dalam menyesuaikan data.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (13)$$

dimana $\bar{y}$ adalah rata-rata nilai aktual.

2.2.6 FastAPI

FastAPI dipilih sebagai kerangka kerja backend karena arsitekturnya yang mendukung *asynchronous programming* (*async/await*), yang krusial untuk menangani *concurrent requests* saat model melakukan inferensi data secara *real-time*. Selain itu, integrasi *native* FastAPI dengan ekosistem Python memfasilitasi *deployment model Machine Learning* (scikit-learn/XGBoost) tanpa memerlukan lapisan translasi bahasa tambahan, serta menyediakan validasi tipe data otomatis melalui Pydantic yang meminimalkan *runtime error*.

2.2.7 PostgreSQL

PostgreSQL digunakan dalam penelitian ini karena kapabilitasnya dalam menangani integritas data relasional sekaligus fleksibilitas penyimpanan data semi-terstruktur (JSONB). Fitur ini esensial untuk menyimpan *metadata* transaksi F&B yang dinamis. Lebih lanjut, skalabilitas PostgreSQL dalam menangani *time-series* data memungkinkan penyimpanan riwayat transaksi jangka panjang yang menjadi basis data latih model peramalan tanpa degradasi performa *query*.

2.2.8 React

React diimplementasikan untuk pembangunan antarmuka pengguna (*dashboard*) guna memvisualisasikan hasil peramalan secara interaktif. Pendekatan *component-based architecture* dan *Virtual DOM* pada React memungkinkan pembaruan grafik peramalan dan tabel rekomendasi stok secara efisien tanpa memuat ulang halaman keseluruhan, memberikan pengalaman pengguna yang responsif saat melakukan analisis data stok harian.

BAB III METODE PENELITIAN

3.1 Analisa Kebutuhan Sistem

Analisis kebutuhan sistem merupakan tahapan fundamental dalam pengembangan perangkat lunak yang bertujuan untuk mengidentifikasi, mendokumentasikan, dan memvalidasi kebutuhan pengguna serta sistem secara komprehensif. Pada penelitian ini, analisis kebutuhan dilakukan melalui observasi langsung terhadap operasional Husgendam Coffee, wawancara dengan pemilik kafe, serta kajian terhadap data historis transaksi dari sistem POS yang telah beroperasi sejak Januari 2022.

Hasil analisis mengidentifikasi beberapa permasalahan kritis pada sistem pengelolaan inventaris yang berlaku saat ini. Pertama, terdapat *food waste* dari total pengadaan bahan baku akibat kelebihan stok yang disebabkan oleh ketidakakuratan estimasi permintaan. Kedua, kejadian kehabisan stok (*stockout*) terjadi rata-rata 2-3 kali per minggu untuk item dengan permintaan tinggi, yang berdampak pada hilangnya potensi pendapatan dan penurunan kepuasan pelanggan. Ketiga, *rebranding* yang dilakukan pada Mei 2025 menyebabkan perubahan struktural pada pola permintaan beberapa menu, sehingga pendekatan peramalan konvensional menjadi kurang efektif.

Berdasarkan permasalahan tersebut, dikembangkan sistem peramalan permintaan menu berbasis *website* yang memanfaatkan algoritma *machine learning* untuk memprediksi permintaan menu harian.

3.1.1 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan fungsi-fungsi yang harus dapat dilakukan oleh sistem untuk memenuhi tujuan pengembangan. Berdasarkan analisis permasalahan dan ekspektasi pengguna, kebutuhan fungsional sistem dirumuskan sebagai berikut:

1. Integrasi Data Transaksi Historis. Sistem harus mampu mengintegrasikan data historis transaksi dari sistem POS yang mencakup informasi tanggal transaksi, nama menu, dan kuantitas terjual. Data ditransformasi dari format tingkat transaksi (*transaction-level*) menjadi agregasi harian (*daily aggregation*) untuk memfasilitasi analisis temporal.

2. Ekstraksi Fitur Otomatis. Sistem mengimplementasikan modul *feature engineering* yang secara otomatis mengekstraksi fitur *time-series* dari data transaksi. Fitur dikelompokkan ke dalam lima kategori: (1) *differencing* yang menangkap perubahan dan akselerasi permintaan, (2) *lag* untuk menangkap nilai historis, (3) *rolling statistics* untuk menangkap tren dan volatilitas jangka pendek, (4) *exponential weighted moving average* untuk menangkap level permintaan yang adaptif terhadap perubahan, dan (5) rasio musiman untuk mendeteksi pola mingguan.
3. Strategi Validasi Silang Berbasis Deret Waktu. Sistem menerapkan *time-based cross-validation* dengan pendekatan *expanding window* minimal 3 *fold* untuk memastikan evaluasi yang robust dan mencegah kebocoran data (*data leakage*) dengan memperhatikan urutan temporal.
4. Pelatihan Model dengan Strategi Dua Tingkat. Sistem melatih model dengan pendekatan adaptif, yaitu *global model* untuk item dengan historis terbatas dan *item-specific model* untuk menu dengan data historis memadai, disertai mekanisme pembatasan (*bounding mechanism*) untuk mencegah prediksi ekstrem.
5. *Model Inference* untuk Prediksi Multi-Menu. Sistem membangun dan menyimpan model *machine learning* yang telah dilatih sehingga dapat digunakan kembali untuk menghasilkan prediksi permintaan seluruh menu tanpa perlu melakukan proses pelatihan ulang setiap kali prediksi dibutuhkan. Setelah model selesai dilatih dan dievaluasi, model disimpan dalam bentuk artefak model (*model artifact*) dan digunakan sebagai model *inference*. Melalui mekanisme ini, pengguna dapat menjalankan proses prediksi berkali-kali menggunakan data terbaru yang tersedia, sementara model yang sama tetap digunakan hingga dilakukan proses *retraining* secara eksplisit. Pendekatan ini meningkatkan efisiensi komputasi, mempercepat waktu respons sistem, serta memungkinkan prediksi permintaan seluruh menu secara simultan tanpa harus mengulangi proses pelatihan yang relatif mahal dari sisi waktu dan sumber daya.
6. Konversi Otomatis ke Rekomendasi Bahan Baku. Sistem menerjemahkan hasil peramalan permintaan menjadi rekomendasi kebutuhan bahan baku

berdasarkan *bill of materials* untuk setiap menu dengan konversi otomatis dari unit penjualan ke unit pengadaan.

7. Evaluasi Model dengan Metrik Ganda. Sistem menyediakan evaluasi komprehensif menggunakan R^2 , MAE, RMSE, dan wMAPE untuk memastikan kualitas prediksi dari berbagai perspektif.
8. Visualisasi Peramalan dan Diagnostik. Sistem menyediakan dasbor visualisasi yang mencakup plot deret waktu (*time series plot*) perbandingan permintaan aktual dengan prediksi, metrik kinerja, analisis distribusi kesalahan, dan agregat peramalan harian.

3.2 Pendekatan Pengembangan Sistem

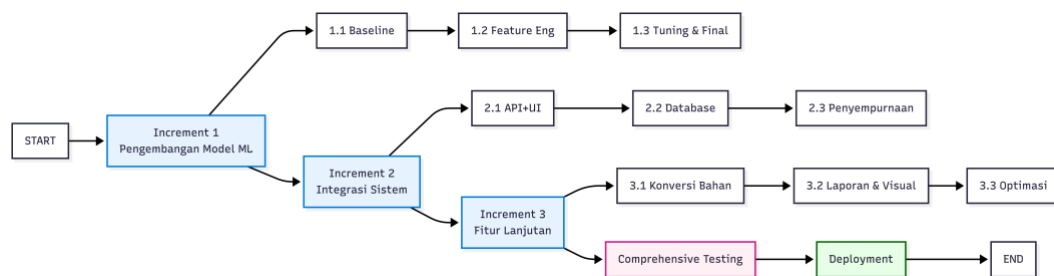
Penelitian ini menggunakan pendekatan *Iterative and Incremental Development* dalam mengembangkan sistem peramalan permintaan menu berbasis *machine learning* untuk Husgendam Coffee. Pemilihan metodologi ini didasarkan pada karakteristik proyek yang menggabungkan dua domain berbeda: pengembangan model *machine learning* yang bersifat eksperimental dan memerlukan iterasi berkelanjutan, serta pengembangan perangkat lunak yang membutuhkan pendekatan terstruktur dan sistematis.

Metodologi *Iterative and Incremental Development* memungkinkan pengembangan sistem dilakukan secara bertahap (*incremental*) dimana setiap tahap menambahkan fungsionalitas baru, sekaligus memungkinkan perbaikan kualitas (*iterative*) melalui siklus pengujian dan penyempurnaan. Pendekatan ini sangat sesuai untuk konteks penelitian ini karena beberapa alasan. Pertama, pengembangan model *machine learning* memerlukan eksperimen berulang dalam *feature engineering*, pemilihan algoritma, dan *hyperparameter tuning* dimana performa model tidak dapat diprediksi sebelum pelatihan dilakukan. Kedua, sistem yang dikembangkan bukan hanya model prediksi tetapi merupakan aplikasi web lengkap dengan arsitektur *three-tier* yang mencakup antarmuka pengguna, *backend API*, dan *database*. Ketiga, pendekatan iteratif memungkinkan deteksi masalah lebih awal sehingga mengurangi risiko kegagalan pada tahap akhir. Keempat, setiap *increment* menghasilkan *prototype* yang dapat divalidasi dengan *stakeholder* untuk memastikan sistem sesuai kebutuhan bisnis.

Pengembangan dilakukan dalam tiga *increment* utama, di mana setiap *increment* dibagi menjadi beberapa iterasi untuk penyempurnaan progresif:

1. *Increment 1: Pengembangan Model Machine Learning*
 - a. Iterasi 1.1: Pembuatan *baseline model* dengan fitur sederhana
 - b. Iterasi 1.2: Penerapan *feature engineering* lanjutan
 - c. Iterasi 1.3: Proses *hyperparameter tuning* dan finalisasi model
2. *Increment 2: Integrasi Sistem*
 - a. Iterasi 2.1: Pengembangan struktur API dan UI
 - b. Iterasi 2.2: Integrasi sistem dengan *database*
 - c. Iterasi 2.3: Penyempurnaan UI dan pengujian fungsionalitas dasar
3. *Increment 3: Fitur Lanjutan*
 - a. Iterasi 3.1: Implementasi fitur konversi kebutuhan bahan baku
 - b. Iterasi 3.2: Pengembangan fitur ekspor laporan dan visualisasi data lanjutan
 - c. Iterasi 3.3: Optimisasi sistem dan persiapan lingkungan produksi

Setelah seluruh iterasi selesai, tahap akhir mencakup pengujian menyeluruh (*comprehensive testing*) dan *deployment* sistem. Alur tahapan pengembangan secara sekuensial ditunjukkan pada Gambar 3.1.



Gambar 3.1 Alur *Iterative & Incremental Development*

3.3 Pengumpulan Data

Pengumpulan data dilakukan dengan memperoleh data historis transaksi dari sistem PoS Husgendam Coffee yang telah beroperasi sejak Januari 2022. Data diekspor dari sistem PoS ke dalam format CSV untuk mempermudah proses pengolahan dan analisis. Data yang diperoleh memiliki struktur transaksi individual

dengan granularitas level item per transaksi dimana setiap baris merepresentasikan satu item yang dibeli dalam satu transaksi.

Struktur data *raw* dari sistem PoS ditunjukkan pada Tabel 3.1. *Dataset raw* terdiri dari 6 atribut yang merepresentasikan informasi transaksi individual.

Tabel 3.1 Struktur Data *Raw* dari Sistem *PoS*

Nama Variabel	Tipe Data	Deskripsi
Tanggal	String (DD/MM/YY HH.MM)	<i>Timestamp</i> transaksi yang mencakup tanggal dan waktu pembelian
Struk	String	Nomor struk unik untuk identifikasi setiap transaksi
Kategori	String	Kategori Produk
SKU	String	<i>Stock Keeping Unit code</i> sebagai ID unit produk
Barang	String	Nama item/menu yang dijual
Kuantitas	Float	Jumlah unit item dalam transaksi

Setelah proses transformasi, data memiliki struktur yang ditunjukkan pada Tabel 3.2. *Dataset final* mencakup periode operasional penuh *coffee shop* yang direpresentasikan dalam format agregasi harian per item.

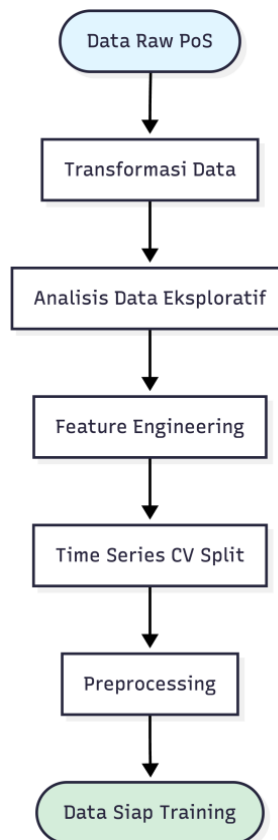
Tabel 3. 2 Struktur Data Final (Agregasi Harian)

Nama Variabel	Tipe Data	Deskripsi
Date	Date (YYYY/MM/DD)	Tanggal transaksi yang telah dikonversi ke format tanggal tanpa komponen waktu
Item	String	Nama menu setelah <i>cleaning</i> , <i>consolidation</i> , dan <i>filtering</i>

Quantity_Sold	Float	Total jumlah unit terjual untuk item tersebut pada tanggal tersebut (hasil agregasi dari semua transaksi)
---------------	-------	---

3.4 Pengolahan Data

Pengolahan data dilakukan melalui serangkaian tahapan sistematis untuk mengubah data *raw* menjadi format yang siap digunakan untuk pelatihan model. Tahapan pengolahan ditunjukkan pada Gambar 3.2.



Gambar 3.2 *Flowchart* Alur Pengolahan Data

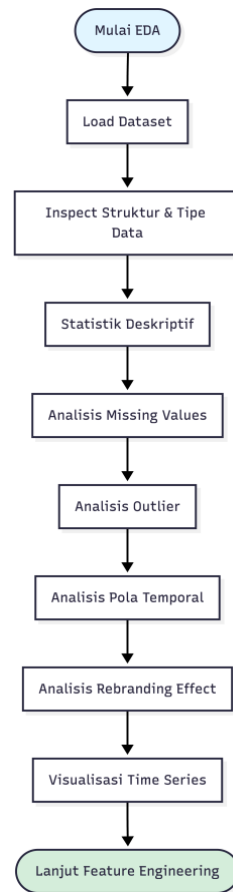
3.4.1 Transformasi Data

Transformasi data mengubah data *raw transaction-level* menjadi *daily aggregation*. Proses transformasi meliputi *parsing* tanggal ke format *datetime*, *cleaning* dan standardisasi nama item untuk mengatasi *inconsistencies* dalam

penamaan, *consolidation variants* untuk menggabungkan variasi penulisan menu yang sama, *filtering add-ons* dengan *prefix add* karena bukan menu utama, *filtering discontinued items* seperti produk yang tidak lagi dijual, *extract date component* untuk menghilangkan *time information*, *grouping by date* dan item, *sum quantity per group* untuk mendapatkan total penjualan harian, dan *sorting* berdasarkan *date* untuk memastikan *temporal ordering*. Hasil transformasi menghasilkan *dataset* dalam format *sparse representation* yang efisien dimana kombinasi *date-item* dengan *zero sales* tidak disimpan secara eksplisit.

3.4.2 Analisa Data Eksploratif

Analisis Data Eksploratif (*Exploratory Data Analysis*) merupakan proses awal analisis data yang bertujuan untuk memahami struktur, pola temporal, dan karakteristik data transaksi sebelum dilakukan *feature engineering* dan *modeling*. Proses ini membantu mengidentifikasi *trends*, *seasonality*, *anomalies*, dan *structural breaks* dalam *time series* yang krusial untuk merancang strategi *forecasting* yang tepat. Langkah-langkah yang dilakukan pada tahap ini ditunjukkan pada Gambar 3.2.



Gambar 3.3 *Flowchart* Analisis Data Eksploratif

Tahap pertama dalam EDA adalah memuat *dataset* dan melakukan inspeksi awal terhadap struktur data. Inspeksi ini mencakup pemeriksaan tipe data untuk setiap kolom, verifikasi keutuhan data transaksi, identifikasi jumlah *unique items* dalam menu operasional, dan validasi rentang temporal data sesuai periode operasional *coffee shop*. Pemahaman struktur ini penting untuk memastikan data telah ditransformasi dengan benar dari format *transaction-level* ke *daily aggregation*.

Selanjutnya dilakukan perhitungan statistik deskriptif untuk variabel *Quantity_Sold*, yang mencakup *measures of central tendency* (rata-rata dan median), *measures of spread* (*standard deviation*, *min*, *max*, *quartiles*), dan distribusi data. Statistik ini dihitung secara *overall* untuk seluruh *dataset*, dan juga per-item untuk memahami karakteristik permintaan masing-masing menu. Sebagai contoh, beberapa item seperti "*Americano Hot*" mungkin memiliki rata-rata penjualan 15 unit/hari dengan std 5, sementara spesial item seperti "*Black Ice*"

setelah *rebranding* memiliki rata-rata penjualan 30 unit/hari dengan std 10, menunjukkan volatilitas yang berbeda.

Pemeriksaan *missing values* dilakukan untuk mengidentifikasi apakah ada gap dalam *time series* data. Untuk data yang sudah ditransformasi ke *daily aggregation*, *missing values* dapat muncul dalam dua bentuk: (1) *Implicit missing* - kombinasi *date-item* yang tidak ada record karena 0 penjualan, dan (2) *Explicit missing* - nilai NaN dalam kolom *Quantity_Sold*. Strategi penanganan disesuaikan dengan konteks: untuk *implicit missing* (0 penjualan), data dibiarkan dalam *sparse format*; untuk *explicit* NaN yang tidak wajar, dilakukan investigasi dan konsultasi dengan mitra.

Analisis *outlier* merupakan langkah kritis dalam peramalan permintaan karena *outlier* bisa mengindikasikan anomali data. Deteksi dilakukan menggunakan metode *Interquartile Range* (IQR) yang diterapkan per menu secara independen. Formula yang digunakan:

$$IQR = Q_3 - Q_1 \quad (14)$$

Keterangan:

- *IQR* = *Interquartile Range* (rentang interkuartil)
- Q_1 = Kuartil pertama (persentil ke-25), nilai dimana 25% data berada di bawahnya
- Q_3 = Kuartil ketiga (persentil ke-75), nilai dimana 75% data berada di bawahnya

Setelah *IQR* dihitung, batas *outlier* ditentukan dengan:

$$LowerBound = Q_1 - 1.5 \times IQR \quad (15)$$

$$UpperBound = Q_3 + 1.5 \times IQR \quad (16)$$

Keterangan:

- *Lower Bound* = Batas bawah, nilai minimum yang masih dianggap wajar
- *Upper Bound* = Batas atas, nilai maksimum yang masih dianggap wajar

- Konstanta 1.5 = Standard *multiplier* dalam metode IQR untuk deteksi *outlier moderate*

Nilai yang berada di luar rentang (*range*) ini dianggap sebagai *potential outliers*. Namun, dalam konteks peramalan permintaan industri F&B, tidak semua *outliers* dihapus karena data tersebut dapat merepresentasikan fenomena asli (*genuine phenomena*) seperti kegiatan promosi, lonjakan akhir pekan, atau lonjakan pasca-*rebranding*. Oleh karena itu, penelitian ini menerapkan pendekatan konservatif dengan melakukan pemangkasan (*clipping*) pada kuantil ekstrem (*extreme quantiles*) 0.05 dan 0.95. Langkah ini bertujuan untuk menjaga stabilitas pada proses *training* tanpa menghapus data sepenuhnya.

Analisis pola temporal merupakan inti dari Analisis Data Eksploratif (EDA) untuk peramalan deret waktu (*time series forecasting*). Analisis ini bertujuan mengidentifikasi berbagai pola yang akan menjadi dasar bagi *feature engineering*. Pola yang dianalisis mencakup musiman mingguan untuk melihat apakah akhir pekan memiliki permintaan lebih tinggi dibandingkan hari kerja, musiman bulanan terkait efek gaji pada awal dan akhir bulan, efek hari libur berupa lonjakan permintaan sebelum dan sesudah hari libur, efek Ramadan terkait perubahan pola konsumsi selama bulan puasa, serta tren jangka panjang untuk melihat pertumbuhan atau penurunan penjualan secara keseluruhan. Visualisasi deret waktu menggunakan *line plots* dan *seasonal decomposition* digunakan untuk mengidentifikasi pola-pola ini secara visual.

Analisis khusus dilakukan untuk mengevaluasi dampak *rebranding* yang terjadi pada Mei 2025. Peristiwa ini menyebabkan perubahan struktural yang signifikan dalam data deret waktu untuk beberapa menu. Analisis dilakukan dengan membandingkan statistik permintaan pra-*rebranding* (sebelum 01-05-2025) dan pasca-*rebranding* (sejak 01-05-2025). Untuk setiap menu, dihitung faktor lonjakan yang merepresentasikan perubahan tingkat permintaan. Sebagai contoh, menu "Black Ice" mengalami *surge factor* sekitar 3.0 (permintaan meningkat 3 kali lipat), sementara menu lain mungkin mengalami perubahan yang lebih moderat atau bahkan penurunan. Pemahaman ini krusial untuk merancang fitur *rebranding* yang akan digunakan dalam model.

Tahap akhir EDA adalah visualisasi komprehensif dari data *time series*. Visualisasi meliputi plot agregat penjualan keseluruhan, plot deret waktu tingkat item (*item-level time series plots*) untuk item terlaris, *heatmaps* yang menunjukkan pola penjualan berdasarkan hari dan bulan, serta histogram distribusi. Visualisasi ini tidak hanya membantu memvalidasi temuan dari analisis statistik, tetapi juga memberikan intuisi visual mengenai karakteristik data yang akan dimodelkan.

3.4.3 Rekayasa Fitur (Feature Engineering)

Feature engineering merupakan tahap ekstraksi fitur-fitur informatif dari data mentah untuk meningkatkan kemampuan prediktif model. Berdasarkan karakteristik data transaksi kedai kopi dan studi literatur terkait, penelitian ini mengekstraksi fitur dari enam kategori utama.

Kategori pertama adalah fitur kalender yang merepresentasikan informasi temporal seperti hari dalam minggu, minggu dalam tahun, dan kuartal. Fitur ini menangkap pola musiman mingguan dan bulanan. Transformasi siklikal menggunakan fungsi trigonometri diterapkan untuk merepresentasikan sifat siklikal dari variabel temporal.

Kategori kedua adalah fitur *event* yang menandai kejadian khusus seperti akhir pekan, periode gaji, hari libur nasional, dan periode Ramadan. Fitur-fitur ini menangkap deviasi permintaan dari pola normal akibat *event* tertentu.

Kategori ketiga adalah fitur *lag* yang merepresentasikan nilai permintaan pada periode sebelumnya ($t-1$, $t-7$, $t-14$) untuk menangkap pola *autoregressive* dalam data.

Kategori keempat adalah fitur *rolling statistics* yang menghitung *moving average* dan *moving standard deviation* dengan berbagai *window sizes* untuk menangkap tren jangka pendek dan volatilitas permintaan.

Kategori kelima adalah fitur momentum yang mengukur *rate of change* permintaan melalui *first-order difference* untuk mengidentifikasi akselerasi atau deselerasi tren.

Kategori keenam adalah fitur *rebranding* yang secara khusus dirancang untuk menangkap dampak *structural break* akibat *rebranding* pada Mei 2025. Fitur ini mencakup indikator biner pra/pasca-*rebranding* dan fungsi *decay* untuk memodelkan penurunan bertahap efek lonjakan permintaan.

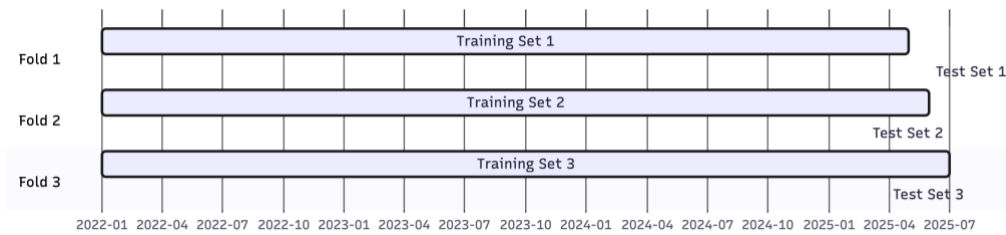
Seleksi fitur final dilakukan melalui analisis *feature importance* dari model *tree-based* dan evaluasi dampak setiap fitur terhadap metrik performa model. Fitur yang tidak meningkatkan performa atau memiliki korelasi tinggi dengan fitur lain akan dieliminasi.

3.4.4 Pra-pemrosesan Data (*Data Preprocessing*)

Pra-pemrosesan data adalah langkah krusial untuk membersihkan dan mempersiapkan data agar siap diolah dalam proses pelatihan model. Pada tahap ini, *dataset* dibagi menjadi dua komponen utama: data fitur (X) dan target (y). Fitur (X) merupakan variabel independen yang mencakup fitur hasil *feature engineering* yang telah diekstraksi pada tahap sebelumnya. Sementara itu, target (y) adalah *Quantity_Sold* yang merepresentasikan jumlah unit menu yang terjual. Strategi pembagian data yang digunakan adalah validasi silang deret waktu (*time series cross-validation*) dengan pendekatan jendela ekspansi (*expanding window approach*). Berbeda dengan metode klasifikasi standar yang menggunakan pengacakan (*random shuffling*), peramalan deret waktu memerlukan strategi pemisahan yang memperhatikan urutan waktu (*temporal ordering*) untuk menghindari kebocoran data (*data leakage*). Strategi *expanding window* memastikan bahwa model selalu dilatih menggunakan data masa lalu dan diuji pada data masa depan. Pendekatan ini dipilih secara khusus untuk konteks peramalan permintaan F&B karena tiga alasan utama (1) *Pattern consumption* di kedai dapat berevolusi seiring waktu (perubahan preferensi pelanggan, tren musiman yang bergeser), sehingga model perlu terus diperbarui dengan data terbaru, (2) Peristiwa *rebranding* pada Mei 2025 menyebabkan *structural break*, dimana model perlu belajar dari data pra dan pasca-*rebranding* secara *incremental*, (3) Mensimulasikan skenario pelatihan ulang realistis dimana model di-retrain secara berkala (misalnya bulanan) dengan data historis yang terus bertambah.

Sebagaimana diilustrasikan pada Gambar 3.3, penelitian ini menggunakan skema *3-fold cross-validation*. Pada *Fold 1*, model dilatih menggunakan data dari awal periode hingga batas waktu tertentu (*cutoff 1*) dan diuji pada periode berikutnya (*test period 1*). *Fold 2* memperluas jendela pelatihan hingga *cutoff 2* yang mencakup periode pengujian sebelumnya, kemudian diuji pada *test period 2*.

Proses ini berlanjut hingga *Fold 3*. Pendekatan ini memberikan evaluasi yang *robust* karena menguji model pada beberapa periode waktu yang berbeda.



Gambar 3. 4 Time Series Cross-Validation dengan Expanding Window

Setiap *fold* dalam validasi silang menghasilkan metrik kinerja yang kemudian diagregasi untuk mendapatkan rata-rata kinerja (*mean performance*) dan varians antar *fold*. Varians yang rendah mengindikasikan bahwa model memiliki kinerja yang konsisten dan tidak terlalu sensitif terhadap periode pelatihan tertentu.

Penelitian ini menggunakan 3-fold *time series cross-validation* dengan pertimbangan sebagai berikut. Pertama, *dataset* mencakup periode Januari 2022 hingga September 2025 (sekitar 44 bulan), sehingga pembagian menjadi 3 *fold* menghasilkan *test period* sekitar 4-5 bulan per *fold* yang cukup representatif untuk mengevaluasi performa model pada berbagai kondisi temporal. Kedua, jumlah *fold* yang terlalu banyak akan menghasilkan *training set* yang terlalu kecil pada *fold* awal, mengurangi kemampuan model menangkap pola historis. Ketiga, 3 *fold* memungkinkan evaluasi performa model pada periode *pra-rebranding*, transisi, dan pasca-*rebranding* secara terpisah.

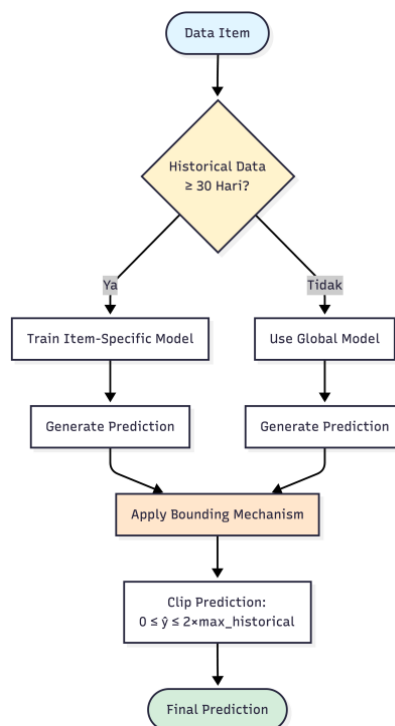
Langkah *preprocessing* tambahan meliputi *handling missing values* dengan strategi $\text{NaN} \rightarrow 0$ (*missing quantity interpreted* sebagai 0 penjualan), *handling infinite values* dengan strategi $\infty \rightarrow 0$ (hasil dari pembagian oleh 0 di beberapa fitur *rolling*), *outlier clipping* pada kuantil 0.05 dan 0.95 per menu untuk mencegah *extreme values* mendominasi pelatihan tanpa menghilangkan *genuine demand spikes*, dan tidak dilakukan *feature scaling* karena *tree-based models invariant* terhadap *feature scales* dan tidak memerlukan normalisasi seperti *neural networks* atau *linear models*.

3.5 Pelatihan Model

Proses pelatihan model dimulai dengan penentuan *hyperparameter* yang optimal. Pada *XGBoost*, *hyperparameter* di konfigurasi dengan *n_estimators* yang

sebanding dengan *Random Forest*, *learning_rate* yang seimbang antara kecepatan konvergensi dan stabilitas model, serta *max_depth* yang lebih rendah dibandingkan *Random Forest* mengingat karakteristik pohon *boosted* yang lebih baik pada kedalaman dangkal. Parameter regularisasi lainnya mencakup *subsample* dan *colsample_bytree* untuk menerapkan *stochastic gradient boosting*, *min_child_weight* serta *gamma* sebagai penguat regularisasi (terutama untuk mengatasi *structural break* dalam data), dan *lambda* untuk regularisasi L2. Nilai pasti *hyperparameter* ditentukan melalui proses pencarian seperti *grid search* atau *random search* pada tahap implementasi.

Seperti digambarkan pada Gambar 3.5, pendekatan *two-level modeling* dirancang khusus untuk mengatasi variasi ketersediaan data historis antar menu. Pendekatan ini mempertimbangkan heterogenitas data, di mana beberapa menu memiliki riwayat penjualan yang panjang dan stabil, sementara menu lainnya baru ditambahkan atau hanya memiliki data penjualan yang sporadis.



Gambar 3.5 Flowchart Logika Two-Level Modeling

Logika pengambilan keputusan model dimulai dengan evaluasi ketersediaan data historis setiap item menu, di mana ambang batas 30 hari ditetapkan berdasarkan kebutuhan minimal observasi untuk menangkap pola musiman mingguan dan bulanan dalam peramalan deret waktu; item dengan data historis

kurang dari ambang batas tersebut akan menggunakan model global yang dilatih pada pola agregat seluruh menu untuk mengatasi keterbatasan data individu, sedangkan item dengan data historis cukup akan dilatih dengan model khusus per item guna menangkap karakteristik permintaan uniknya, kemudian setelah prediksi dihasilkan, diterapkan mekanisme *bounding* dengan batas bawah 0 untuk mencegah nilai negatif dan batas atas sebesar dua kali nilai maksimum historis agar tetap memberikan ruang pertumbuhan permintaan tanpa menghasilkan prediksi yang tidak realistis, sehingga menjaga keandalan dan kelayakan bisnis dari hasil ramalan secara keseluruhan..

Model yang telah dilatih disimpan untuk evaluasi dan selanjutnya akan dibandingkan performanya berdasarkan *multiple metrics* untuk memilih model terbaik.

3.6 Evaluasi Model

Sistem perlu menyediakan mekanisme evaluasi yang komprehensif menggunakan berbagai metrik untuk mengukur kinerja model, yaitu:

a. *Mean Absolute Error* (MAE)

Pengukuran MAE merujuk pada persamaan (10). MAE menghitung rata-rata kesalahan absolut antara nilai prediksi dan nilai aktual. Alasan utama MAE adalah karena hasilnya disajikan dalam satuan unit yang sama dengan variabel target, sehingga sangat mudah diinterpretasikan untuk kepentingan bisnis untuk memahami besaran kesalahan rata-rata secara riil.

b. *Root Mean Squared Error* (RMSE)

Pengukuran MAE merujuk pada persamaan (11). RMSE mengukur akar dari rata-rata kuadrat kesalahan. Karakteristik utama RMSE adalah memberikan bobot penalti yang lebih tinggi pada kesalahan prediksi yang besar. Hal ini sangat penting dalam operasional bisnis untuk mendeteksi adanya penyimpangan atau *outlier* signifikan yang bisa berakibat fatal jika tidak terdeteksi.

c. *Weighted Mean Absolute Percentage Error* (wMAPE)

Pengukuran wMAPE merujuk pada persamaan (12). Metrik ini merupakan pengembangan dari MAPE standar untuk mengatasi kondisi di mana data aktual

bernilai nol. Dalam data penjualan harian, nilai nol sering muncul, penggunaan wMAPE (atau penambahan konstanta kecil c pada rumus) memastikan nilai evaluasi tetap terdefinisi dan akurat meskipun terdapat transaksi yang kosong. wMAPE memberikan gambaran dalam dimensi persentase, sehingga lebih intuitif bagi pemilik usaha untuk memahami sejauh mana penyimpangan prediksi terhadap total volume bisnis

d. Koefisien Determinasi (R^2)

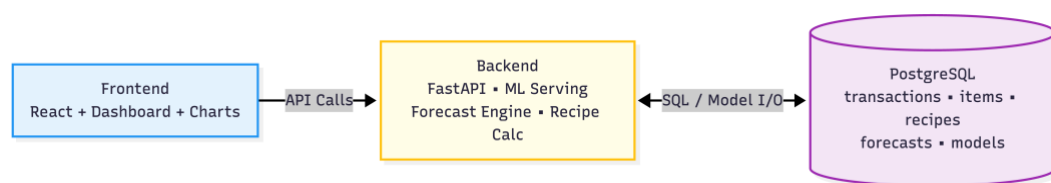
Pengukuran R^2 merujuk pada persamaan (13). Metrik ini digunakan untuk mengukur proporsi varian pada data aktual yang dapat dijelaskan oleh model prediksi. Nilai ini memberikan gambaran seberapa baik model menyesuaikan diri dengan data (*goodness of fit*). R^2 menunjukkan seberapa baik model dapat menjelaskan varians pada data aktual, sehingga pelaku analisis dapat mengetahui apakah model tersebut sudah "tepat", mengalami *underfit*, atau *overfit* terhadap pola data

3.7 Pengembangan Sistem Perangkat Lunak

Pengembangan sistem mencakup implementasi komponen-komponen perangkat lunak yang mengintegrasikan model *machine learning* ke dalam aplikasi web yang lengkap dan fungsional.

3.7.1 Arsitektur Sistem

Sistem dirancang menggunakan arsitektur *three-tier* yang memisahkan *concerns* berdasarkan *layers* untuk *maintainability* dan *scalability*. Arsitektur ditunjukkan pada Gambar 3.6.



Gambar 3.6 Arsitektur *Three-Tier System*

Lapisan Presentasi dibangun menggunakan *framework React* untuk menghasilkan antarmuka pengguna yang responsif, interaktif, dan ramah pengguna. Lapisan ini bertanggung jawab untuk menampilkan berbagai visualisasi data penting seperti grafik *time series* yang menggambarkan tren penjualan, metrik kinerja model peramalan, serta hasil prediksi permintaan menu beserta kebutuhan

bahan baku yang dihasilkan. Selain itu, lapisan presentasi juga menangani interaksi pengguna secara langsung, seperti pemilihan rentang tanggal untuk peramalan, *trigger* proses prediksi, dan penyajian *insight* bisnis yang mudah dipahami. Komunikasi dengan lapisan aplikasi dilakukan secara mulus melalui panggilan HTTP REST API dengan format pertukaran data JSON.

Lapisan Aplikasi dikembangkan dengan memanfaatkan *framework* FastAPI yang mendukung pemrosesan permintaan secara *asinkronus* dan berkinerja tinggi. Lapisan ini menjadi inti dari seluruh logika bisnis sistem, di mana terdapat beberapa komponen utama yang bekerja secara terintegrasi. Komponen tersebut meliputi penyajian model *machine learning* untuk melakukan inferensi secara cepat, *pipeline* rekayasa fitur yang menyiapkan data transaksi mentah menjadi *input* yang sesuai untuk model, peramalan yang menerapkan strategi pemodelan dua tingkat, serta kalkulator resep yang secara akurat menerjemahkan hasil prediksi permintaan menu menjadi kebutuhan bahan baku berdasarkan daftar bahan setiap item menu.

Lapisan Data menggunakan PostgreSQL sebagai sistem manajemen basis data relasional yang andal dan terstruktur. Lapisan ini menyimpan berbagai entitas penting yang menjadi fondasi operasional sistem, antara lain tabel transaksi yang mencatat data penjualan historis, tabel item sebagai master data menu, tabel *forecasts* untuk menyimpan hasil prediksi lengkap dengan interval kepercayaan dan informasi versi model, tabel bahan baku sebagai referensi master bahan, tabel resep yang memetakan hubungan antara menu dan komposisi bahan beserta rasio kuantitasnya, serta tabel model yang mencatat *metadata* penting seperti *hyperparameter*, tanggal pelatihan, dan metrik performa model. Struktur data yang terorganisasi dengan baik ini memastikan akses dan integritas informasi yang optimal bagi seluruh sistem.

3.7.2 Desain Basis Data (*Database*)

Desain Basis Data (*Database*) Skema basis data dirancang menggunakan normalisasi untuk menjaga integritas data sekaligus mempertahankan performa kueri (query). Tabel-tabel utama mencakup transactions dengan indeks komposit (composite index) pada (date, item_id) untuk mempercepat kueri deret waktu (time series), forecasts dengan indeks parsial untuk data peramalan terbaru, serta recipes

dengan batasan unik (unique constraint) pada (item_id, ingredient_id) untuk mencegah duplikasi entri data.

3.7.3 Implementasi API

Endpoint API dirancang mengikuti prinsip-prinsip RESTful: POST /api/transactions untuk mengunggah data historis, POST /api/forecast/generate untuk memicu proses peramalan dengan parameter horizon waktu dan pemilihan model, GET /api/forecast/results untuk mengambil hasil prediksi dengan penyaringan berdasarkan rentang tanggal atau *item*, GET /api/ingredients/requirements untuk mengonversi hasil peramalan menjadi daftar pengadaan bahan, serta GET /api/model/metrics untuk memantau kinerja model. Setiap *endpoint* dilengkapi dengan autentikasi menggunakan token JWT, validasi permintaan (*request validation*), dan penanganan kesalahan (*error handling*).

3.7.4 Implementasi Antarmuka

Antarmuka pengguna (*user interface*) dirancang dengan fokus pada visualisasi plot deret waktu untuk membandingkan permintaan aktual vs prediksi, dasbor kinerja yang menampilkan metrik (R^2 , MAE, RMSE, wMAPE) di seluruh *fold* validasi, serta tabel kebutuhan bahan baku yang dapat diekspor. Prinsip desain yang diterapkan meliputi kesederhanaan (*simplicity*) untuk kurva pembelajaran yang minimal, kejelasan (*clarity*) dengan pelabelan dan instruksi yang jelas, serta responsivitas (*responsiveness*) dengan tata letak adaptif untuk perangkat *desktop* dan *mobile*.

3.8 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan fungsionalitas dan *usability* memenuhi *requirements* yang telah ditetapkan

3.8.1 Pengujian Model

Pengujian model dilakukan pada *holdout test set* yang sepenuhnya tidak dilibatkan selama proses pelatihan (*training*) dan validasi guna menjamin objektivitas hasil evaluasi. Kinerja algoritma XGBoost diukur menggunakan empat metrik utama (R^2 , MAE, RMSE, dan wMAPE).

Keberhasilan model ditentukan berdasarkan perbandingannya terhadap ambang batas (*threshold*) yang merujuk pada standar penelitian industri kuliner terdahulu. Kriteria model dikatakan optimal apabila memenuhi target performa sebagai berikut:

- a. Tingkat Akurasi: Mencapai akurasi di atas 80%, yang direpresentasikan dengan nilai $wMAPE < 20\%$. Target ini mengacu pada *benchmark* penelitian Schmidt dkk. (2022) pada industri restoran yang menghasilkan error di kisaran 19,5% - 19,6%.
- b. Kualitas Fit (R^2): Nilai $R^2 \geq 0.6$, Target ini ditetapkan dengan mengacu pada penelitian Nasser dkk. (2023) yang menunjukkan bahwa untuk produk perishable dengan volatilitas tinggi.
- c. Tingkat Kesalahan: Nilai $MAE \leq 1.0$ dan RMSE lebih kecil dari Standar Deviasi data aktual, yang mengindikasikan deviasi prediksi yang minimal dan dapat ditoleransi dalam operasional harian.

3.8.2 System Usability Scale (SUS)

Pengujian *usability* dilakukan menggunakan *System Usability Scale* (SUS) yang terdiri dari 10 pernyataan dengan skala *Likert* 5 poin. Responden pengujian meliputi manajer operasional Husgendam Coffee sebagai pengguna utama sistem dan 4 staf operasional yang akan menggunakan sistem untuk perencanaan harian, sehingga total responden berjumlah 5 orang.

Proses pengujian dilakukan dengan tahapan sebagai berikut. Pertama, responden diberikan penjelasan singkat tentang tujuan dan fungsi sistem. Kedua, responden diminta menjalankan skenario penggunaan yang mencakup melihat hasil peramalan, memfilter data berdasarkan menu tertentu, dan mengeksport laporan kebutuhan bahan baku. Ketiga, responden mengisi kuesioner SUS setelah menyelesaikan skenario. Kuesioner yang didistribusikan kepada responden berisi 10 pernyataan yang merefleksikan persepsi pengguna terhadap kemudahan penggunaan sistem. Pernyataan-pernyataan tersebut ditunjukkan pada Tabel 3.3 berikut:

Tabel 3.3 Daftar Pernyataan Kuesioner *System Usability Scale* (SUS)

No.	Pernyataan	Penilaian/Skor				
		1	2	3	4	5
1	Saya merasa sistem ini mudah digunakan.					
2	Saya merasa sistem ini terlalu kompleks.					
3	Saya merasa sistem ini mudah dipelajari.					
4	Saya membutuhkan bantuan teknis untuk bisa menggunakan sistem ini.					
5	Saya merasa fitur dalam sistem ini sudah terintegrasi dengan baik.					
6	Saya merasa ada terlalu banyak inkonsistensi dalam sistem ini.					
7	Saya yakin bahwa sebagian besar orang bisa belajar menggunakan sistem ini dengan cepat.					
8	Saya merasa sistem ini membingungkan ketika digunakan.					
9	Saya merasa nyaman menggunakan sistem ini.					
10	Saya membutuhkan banyak pelatihan sebelum bisa menggunakan sistem ini.					

Perhitungan skor *System Usability Scale* (SUS) mengikuti formula standar yang diperkenalkan oleh Brooke (1996). Untuk pernyataan bernomor ganjil, skor diperoleh dengan mengurangi 1 dari nilai respons. Untuk pernyataan bernomor genap, skor diperoleh dengan mengurangi nilai respons dari 5. Seluruh skor kemudian dijumlahkan dan dikalikan dengan 2,5 untuk menghasilkan skor akhir dalam rentang 0–100. Sistem dengan skor $SUS \geq 70$ dikategorikan memiliki tingkat *usability* yang dapat diterima (*acceptable usability*).

3.9 Deployment dan Maintenance

Setelah pengujian selesai, sistem diterapkan (*deployed*) ke lingkungan produksi (*production environment*). Tahap *deployment* meliputi pelatihan akhir pada seluruh data historis yang tersedia, serialisasi model dan pengunggahan ke

server dengan pelabelan versi (*version tagging*), *deployment backend* ke *cloud* atau server *on-premise*, *deployment* basis data dengan migrasi data historis dan pencadangan otomatis (*automated backups*), serta *deployment frontend* dengan paket aplikasi (*bundle*) yang telah dioptimalkan.

Pemeliharaan sistem mencakup pemantauan berkelanjutan untuk pelacakan kinerja harian (pemantauan wMAPE, pencatatan kesalahan atau *error logging*, dan pemantauan aktivitas pengguna), jadwal pelatihan ulang model (*retraining*), perbaikan *bug* dan *patches* berdasarkan laporan pengguna, serta peningkatan fitur berdasarkan umpan balik pengguna untuk perbaikan berkelanjutan (*continuous improvement*).

BAB IV ANALISIS DAN PERANCANGAN SISTEM

4.1 Kebutuhan Sistem

Tahap analisis kebutuhan adalah fase awal dalam proses pengembangan perangkat lunak yang berfokus pada pengumpulan, identifikasi, dan pendokumentasian kebutuhan sistem. Kebutuhan tersebut meliputi kebutuhan fungsional maupun non-fungsional yang harus dipenuhi agar sistem yang dikembangkan dapat beroperasi sesuai dengan tujuan dan kebutuhan pengguna.

4.1.1 Deskripsi Sistem

Berdasarkan latar belakang dan rumusan masalah yang telah diuraikan, sistem yang dikembangkan bertujuan untuk melakukan peramalan permintaan harian pada menu Husgendam Coffee menggunakan metode *machine learning*. Selain itu, sistem juga dirancang untuk menghitung kebutuhan bahan baku berdasarkan hasil peramalan dengan pendekatan *Bill of Materials* (BOM), serta menyediakan *dashboard* analitik interaktif yang menampilkan klasifikasi item berdasarkan metode ABC, metrik evaluasi performa model, dan ringkasan hasil peramalan.

Sistem ini memiliki satu peran pengguna, yaitu karyawan kafe, yang bertanggung jawab mengelola seluruh operasional sistem termasuk melihat *dashboard*, menganalisis data penjualan, menjalankan pelatihan ulang model, mengeksport kebutuhan bahan baku, dan mengatur konfigurasi sistem. Tabel 4.1 berikut menjelaskan deskripsi sistem secara keseluruhan.

Tabel 4.1 Deskripsi Sistem

Judul	Penerapan Algoritma <i>Machine Learning</i> dalam Peramalan Permintaan Produk pada <i>Coffee Shop</i> Berbasis Data Historis Transaksi
Jenis Aplikasi	Aplikasi berbasis web untuk meramalkan permintaan produk harian menu kafe menggunakan <i>machine learning</i> , menghitung kebutuhan bahan baku melalui pendekatan <i>Bill of Materials</i> (BOM), serta menyajikan <i>dashboard</i> analitik interaktif yang mencakup klasifikasi ABC item dan metrik performa model.
Pengguna	Karyawan kafe bertanggung jawab mengelola seluruh operasional sistem, termasuk melihat <i>dashboard</i> , menganalisis

	data penjualan, menjalankan pelatihan ulang model, mengeksport kebutuhan bahan baku, memantau status pelatihan secara <i>real-time</i> , dan mengatur konfigurasi sistem.
Konten	Aplikasi menyediakan fitur peramalan penjualan menggunakan model XGBoost; perhitungan kebutuhan bahan baku berdasarkan BOM; klasifikasi ABC item menu; metrik performa model; serta kemampuan ekspor data dalam format CSV.
Teknologi	<i>Backend</i> menggunakan Python dengan FastAPI dan SQLAlchemy; <i>frontend</i> menggunakan React 19 dengan TypeScript, Vite, TanStack Router, TanStack Query, dan shadcn/ui; basis data menggunakan PostgreSQL 16; model <i>machine learning</i> menggunakan XGBoost, scikit-learn; serta <i>deployment</i> menggunakan Docker.

4.1.2 Analisis Kebutuhan Fungsional

Kebutuhan fungsional dari sistem ini disesuaikan dengan kebutuhan Husgendam Coffee, yang berfokus pada peramalan permintaan produk menu harian, perhitungan kebutuhan bahan baku, serta analisis data penjualan. Sistem ini dirancang untuk mempermudah kafe manajer dan karyawan dalam menganalisis data penjualan historis, meramalkan permintaan di masa mendatang, dan menghitung kebutuhan bahan baku secara otomatis berdasarkan hasil peramalan tersebut. Sebagaimana diidentifikasi dalam Bab I, kebutuhan utama mitra adalah memiliki alat bantu pengambil keputusan yang objektif untuk menggantikan perencanaan berbasis intuisi yang selama ini diterapkan.

Kebutuhan fungsional juga memperhatikan alur kerja pengguna untuk memastikan setiap fitur dapat dijalankan dengan efisien dan sesuai dengan tujuan sistem. Setiap fungsi dirancang untuk saling terintegrasi dan membentuk alur penggunaan yang sederhana namun efektif, sehingga sistem dapat diadopsi dengan mudah oleh manajemen Husgendam Coffee tanpa memerlukan pelatihan khusus yang intensif. Tabel 4.2 berikut menjelaskan analisis kebutuhan fungsional sistem.

Tabel 4.2 Analisis Kebutuhan Fungsional

No.	Kebutuhan	Deskripsi
1	<i>Login</i> dan <i>logout</i> sistem	Pengguna dapat masuk ke sistem menggunakan email dan <i>password</i> , serta keluar dari sistem.
2	Melihat <i>dashboard</i> ringkasan	Pengguna dapat melihat halaman <i>dashboard</i> yang menampilkan data metrik evaluasi dan tabel ringkasan peramalan.
3	Melihat analisis ABC	Pengguna dapat melihat klasifikasi item menu berdasarkan volume penjualan menggunakan metode ABC.
4	Melihat hasil peramalan penjualan	Pengguna dapat melihat hasil peramalan penjualan dengan pemilihan item dan model yang diinginkan, termasuk visualisasi perbandingan antara data aktual dan prediksi.
5	Melihat kebutuhan bahan baku harian	Pengguna dapat melihat kebutuhan bahan baku harian yang dihitung secara otomatis berdasarkan hasil peramalan dan data resep, dengan fitur filter berdasarkan jenis bahan dan rentang tanggal.
6	Mengekspor data kebutuhan bahan baku ke CSV	Pengguna dapat mengekspor data kebutuhan bahan baku ke dalam format <i>file</i> CSV untuk keperluan operasional.
8	Melatih ulang (<i>retrain</i>) model peramalan	Pengguna dapat memicu pelatihan ulang model peramalan dari antarmuka web.

4.1.3 Analisis Kebutuhan Non-Fungsional

Kebutuhan non-fungsional merupakan kebutuhan yang mendukung kualitas dan kinerja sistem secara keseluruhan. Kebutuhan ini menggambarkan kebutuhan tidak langsung terkait fitur dalam sistem, namun memengaruhi pengalaman

pengguna dan keandalan sistem dalam lingkungan operasional yang sesungguhnya. Analisis kebutuhan non-fungsional ini dijabarkan dalam Tabel 4.3.

Tabel 4.3 Analisis Kebutuhan Non-Fungsional

No.	Kategori	Deskripsi
1	<i>Usability</i>	Sistem memiliki antarmuka yang <i>user-friendly</i> dan responsif, mudah digunakan oleh karyawan tanpa memerlukan pelatihan teknis khusus, serta mendukung tampilan pada berbagai perangkat (<i>desktop, tablet, mobile</i>).
2	<i>Compatibility</i>	Sistem dapat diakses melalui berbagai browser modern seperti Chrome, Firefox, Safari, dan Edge tanpa memerlukan instalasi tambahan.
3	<i>Security</i>	Sistem hanya dapat diakses oleh pengguna yang terdaftar dengan autentikasi menggunakan JSON Web Token (JWT). Password disimpan dalam bentuk <i>hash</i> menggunakan algoritma <i>bcrypt</i> untuk melindungi data kredensial pengguna.
4	<i>Performance</i>	API dibangun secara asinkron menggunakan FastAPI dengan <i>connection pooling</i> untuk koneksi basis data PostgreSQL, sehingga mampu menangani permintaan secara efisien. Proses pelatihan model dijalankan secara <i>background</i> menggunakan <i>ThreadPoolExecutor</i> agar tidak memblokir API selama pelatihan berlangsung.

4.1.3.1 Kebutuhan Perangkat Lunak

Jenis perangkat lunak yang digunakan dalam pengembangan sistem dijelaskan pada Tabel 4.4. Pemilihan teknologi ini didasarkan pada pertimbangan performa, ekosistem *library* yang mendukung implementasi *machine learning*, dan kompatibilitas antar komponen sistem.

Tabel 4.4 Kebutuhan Perangkat Lunak

No.	Perangkat Lunak	Fungsi
1	Sistem Operasi (macOS / Windows / Linux)	Platform pengembangan dan <i>deployment</i>
2	Visual Studio Code	Editor kode sumber utama dalam pengembangan sistem
3	Python 3.10+	Bahasa pemrograman untuk <i>backend</i> dan implementasi model <i>machine learning</i>
4	Node.js 20+	<i>Runtime JavaScript</i> untuk pengembangan <i>frontend</i>
5	PostgreSQL 16	Sistem manajemen basis data relasional untuk penyimpanan data transaksi dan hasil peramalan
6	Docker	Platform kontainerisasi untuk <i>deployment</i> sistem secara konsisten di berbagai lingkungan
7	Bun / npm	<i>Package manager</i> untuk mengelola dependensi <i>frontend</i>

4.1.3.2 Kebutuhan Perangkat Keras

Perangkat keras yang digunakan dalam pengembangan sistem dijelaskan dalam Tabel 4.5. Spesifikasi minimum ini diperlukan untuk memastikan proses pelatihan model *machine learning* dapat berjalan dengan lancar, mengingat *dataset* yang digunakan mencakup lebih dari 58.000 baris data transaksi.

Tabel 4.5 Kebutuhan perangkat Keras

No.	Komponen	Deskripsi
1	Prosesor	Dual-core atau lebih
2	RAM	Minimum 4 GB
3	Penyimpanan	Minimum 10 GB

4.2 Design

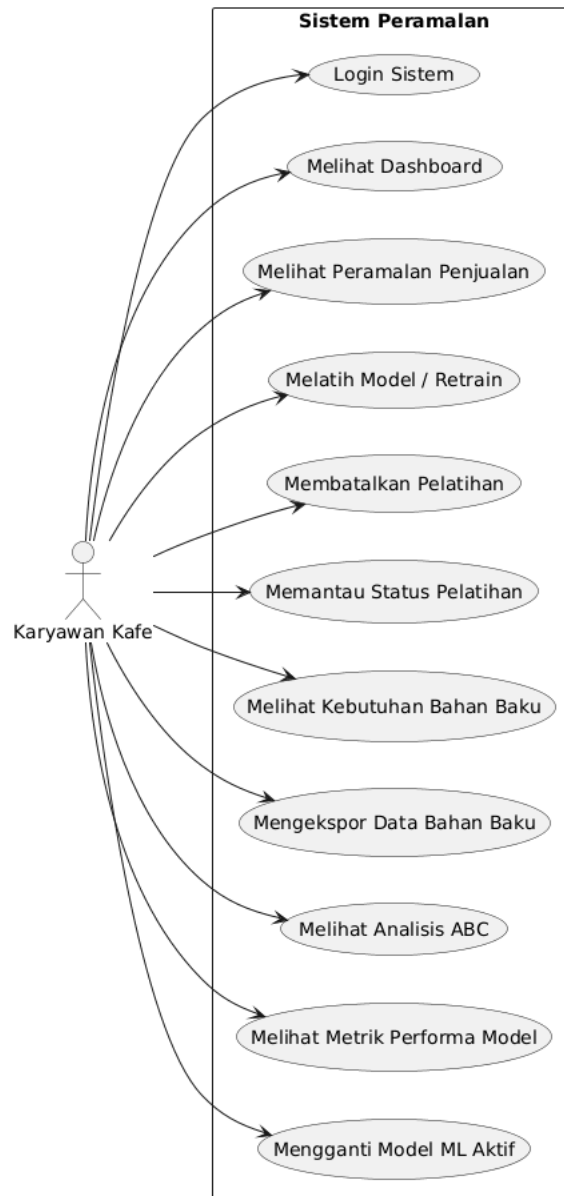
Tahap desain sistem merupakan bagian penting dalam pengembangan perangkat lunak yang bertujuan untuk merancang struktur dan alur kerja sistem

berdasarkan kebutuhan yang telah dianalisis pada tahap sebelumnya. Dalam konteks *Iterative and Incremental Development*, desain sistem dilakukan secara bertahap mulai dari desain arsitektur tingkat tinggi yang ditetapkan pada iterasi pertama, kemudian disempurnakan pada iterasi-iterasi berikutnya seiring dengan bertambahnya komponen dan fitur yang dikembangkan. Desain sistem mencakup berbagai aspek seperti pemodelan proses bisnis, interaksi antar pengguna dan sistem, serta struktur data yang digunakan dalam sistem.

4.2.1 Use Case Diagram

Use case diagram digunakan untuk menggambarkan interaksi antara aktor dengan *use case* yang tersedia dalam sistem, serta fungsionalitas utama yang terdapat dalam sistem ini. Sistem ini memiliki satu aktor, yaitu karyawan kafe. Diagram yang ditampilkan menggambarkan bagaimana Sistem Peramalan Persediaan Bahan Baku Husgendam Coffee beroperasi dengan melibatkan karyawan kafe sebagai aktor tunggal yang memiliki akses penuh aplikasi.

Dalam sistem ini, karyawan memiliki akses untuk melakukan *login*, melihat *dashboard* ringkasan, melihat hasil peramalan penjualan, melatih ulang model peramalan, membatalkan proses pelatihan, memantau status pelatihan secara *real-time*, melihat kebutuhan bahan baku harian, mengekspor data bahan baku ke CSV, melihat analisis ABC, melihat metrik performa model, serta mengganti model *machine learning* yang aktif. Gambar 4.1 menggambarkan *use case* diagram sistem.



Gambar 4.1 Use Case Diagram

4.2.2 Physical Data Model

Physical Data Model (PDM) merupakan representasi implementasi basis data yang digunakan dalam sistem. Model ini menggambarkan struktur penyimpanan data secara fisik pada PostgreSQL, meliputi tabel, atribut, tipe data, *primary key*, *foreign key*, serta hubungan antar tabel. PDM digunakan sebagai acuan dalam pembangunan basis data agar proses penyimpanan, pengelolaan, dan pengambilan data dapat dilakukan secara terstruktur dan konsisten.

Pada penelitian ini digunakan dua basis data yang saling terintegrasi untuk mendukung Sistem Peramalan Persediaan Bahan Baku Cafe Husgendam. Basis

data utama, yaitu *cafe_forecasting*, digunakan untuk menyimpan seluruh data operasional sistem, sedangkan basis data eksternal *hus_db* digunakan sebagai sumber data transaksi penjualan yang berasal dari sistem PoS Kafe Husgendam. Data dari basis data PoS kemudian disinkronisasikan ke basis data utama untuk digunakan dalam proses analisis dan peramalan.

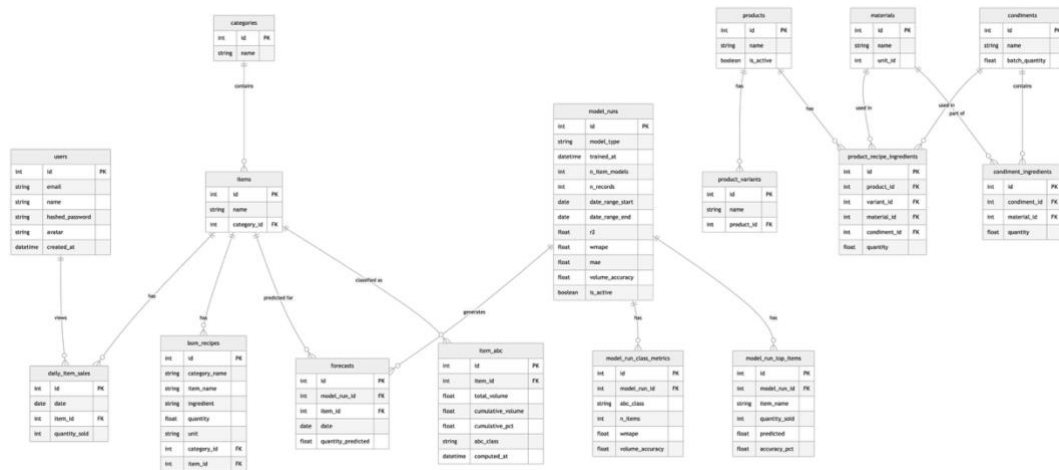
4.2.2.1 Database Aplikasi Utama (*cafe_forecasting*)

Basis data ini dirancang untuk mendukung seluruh kebutuhan fungsional sistem, mulai dari autentikasi pengguna, pengelolaan data penjualan, proses pelatihan model *machine learning*, penyimpanan hasil peramalan, analisis klasifikasi ABC, hingga perhitungan kebutuhan bahan baku berdasarkan *Bill of Materials* (BOM).

Data menu disimpan pada tabel *categories* dan *items*, yang digunakan sebagai dasar pengelompokan dan identifikasi produk yang akan dianalisis. Data penjualan historis disimpan pada tabel *daily_item_sales*, yang menjadi sumber utama dalam proses pelatihan model dan peramalan permintaan produk.

Untuk mendukung proses *machine learning*, sistem menggunakan tabel *model_runs*, *forecasts*, *model_run_class_metrics*, dan *model_run_top_items*. Tabel-tabel tersebut digunakan untuk menyimpan informasi hasil pelatihan model, hasil prediksi, metrik performa model, serta ringkasan performa klasifikasi ABC. Sementara itu, hasil klasifikasi ABC setiap item disimpan pada tabel *item_abc* untuk mendukung analisis prioritas produk berdasarkan kontribusi penjualan.

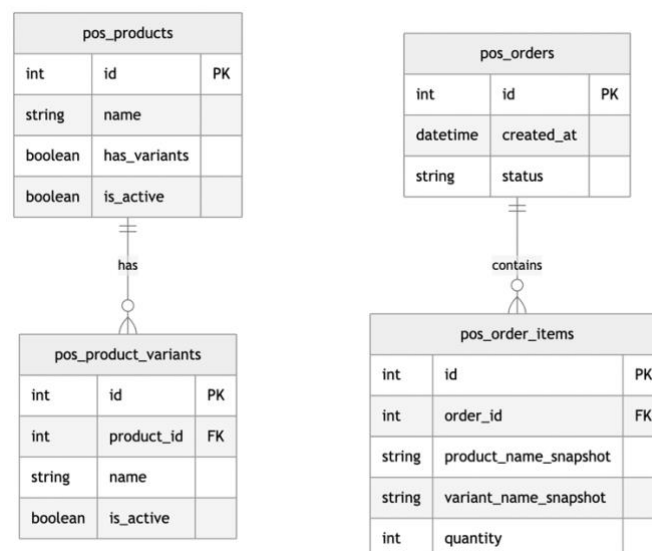
Perhitungan kebutuhan bahan baku didukung oleh tabel *products*, *product_variants*, *materials*, *condiments*, *product_recipe_ingredients*, dan *condiment_ingredients*. Struktur ini memungkinkan sistem memetakan hasil peramalan penjualan menjadi kebutuhan bahan baku aktual melalui pendekatan *Bill of Materials*. Selain itu, penggunaan tabel *condiment* dan *ingredient* memungkinkan sistem merepresentasikan bahan baku bertingkat, sehingga kebutuhan bahan baku dapat dihitung secara lebih rinci dan akurat.

Gambar 4.2 *Physical Data Model* Aplikasi Peramalan

4.2.2.2 Database PoS Eksternal (hus_db)

Basis data ini merupakan basis data milik sistem PoS Kafe Husgendam yang digunakan sebagai sumber data transaksi penjualan. Sistem hanya memiliki hak akses baca (*read-only*) terhadap basis data ini dan tidak melakukan perubahan secara langsung terhadap data yang tersimpan di dalamnya.

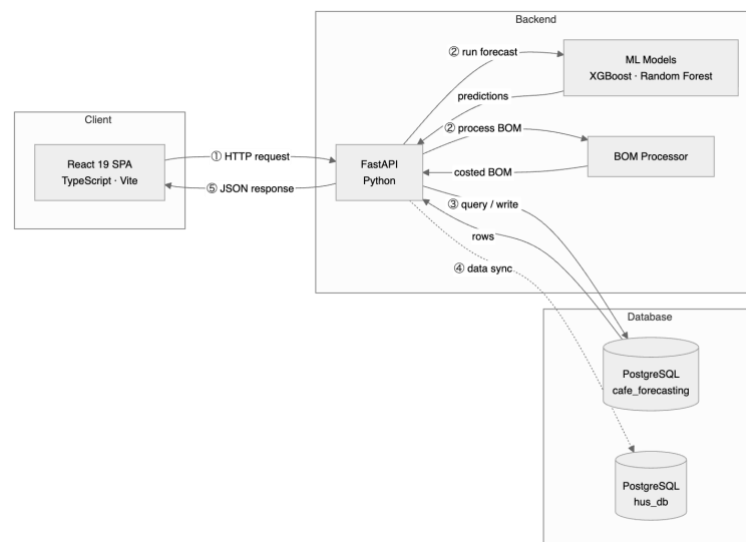
Basis data hus_db terdiri dari tabel produk dan tabel transaksi. Tabel *pos_products* dan *pos_product_variants* menyimpan informasi mengenai produk yang tersedia pada sistem POS beserta variannya. Sementara itu, tabel *pos_orders* dan *pos_order_items* menyimpan data transaksi yang dilakukan pelanggan, termasuk informasi item yang dibeli pada setiap pesanan.

Gambar 4.3 *Physical Data Model* transaksi PoS

4.2.3 Arsitektur Diagram

Arsitektur diagram merupakan representasi visual yang menggambarkan struktur sistem serta hubungan antar komponen yang membentuk sistem secara keseluruhan. Diagram ini digunakan untuk menunjukkan bagaimana data dan informasi mengalir di antara komponen sistem, mulai dari antarmuka pengguna hingga penyimpanan data.

Pada penelitian ini, Sistem Peramalan Persediaan Bahan Baku Kafe Husgendam dirancang menggunakan pola arsitektur tiga lapisan (*three-tier architecture*), yang terdiri atas lapisan presentasi (*presentation tier*), lapisan logika bisnis (*logic tier*), dan lapisan data (*data tier*). Pemisahan lapisan ini bertujuan untuk meningkatkan modularitas, kemudahan pengembangan, serta pemeliharaan sistem.



Gambar 4.4 Arsitektur Diagram

Pada Gambar 4.20, lapisan pertama yaitu *Presentation Tier* berfungsi sebagai antarmuka pengguna yang digunakan untuk berinteraksi dengan sistem. Melalui lapisan ini, pengguna dapat mengakses fitur-fitur seperti melihat *dashboard*, melakukan analisis ABC, melihat hasil peramalan penjualan, menghitung kebutuhan bahan baku, serta melakukan pelatihan ulang model *machine learning*. Seluruh interaksi pengguna dikirimkan ke backend melalui mekanisme REST API.

Lapisan kedua yaitu *Logic Tier* berfungsi sebagai pusat pemrosesan logika bisnis sistem. Lapisan ini menangani proses autentikasi pengguna, pengolahan data penjualan, pelatihan dan inferensi model *machine learning*, analisis klasifikasi

ABC, serta perhitungan kebutuhan bahan baku berdasarkan pendekatan *Bill of Materials* (BOM). Selain itu, lapisan ini juga bertanggung jawab untuk mengelola komunikasi antara antarmuka pengguna dan basis data.

Lapisan ketiga yaitu *Data Tier* berfungsi sebagai media penyimpanan data. Basis data utama digunakan untuk menyimpan seluruh data operasional sistem, termasuk data pengguna, data penjualan, hasil peramalan, metrik performa model, data klasifikasi ABC, dan data resep BOM. Selain itu, sistem juga terhubung dengan basis data eksternal milik sistem POS Cafe Husgendam yang digunakan sebagai sumber data transaksi penjualan. Data dari basis data eksternal tersebut disinkronisasikan secara berkala ke basis data utama untuk mendukung proses analisis dan peramalan.

4.2.4 Desain Antarmuka

Dalam penelitian ini, desain antarmuka dibuat untuk memastikan Sistem Peramalan Persediaan Bahan Baku Husgendam Coffee memiliki tampilan yang *user-friendly* dan responsif. Desain ini juga mempertimbangkan aspek *usability* sebagaimana ditetapkan dalam kebutuhan non-fungsional, untuk meningkatkan pengalaman pengguna dalam mengakses informasi yang mendukung pengambilan keputusan operasional. Setiap komponen antarmuka dirancang secara konsisten menggunakan *design system* dari shadcn/ui agar memudahkan navigasi dan mempercepat pemahaman pengguna terhadap fungsi sistem.

4.2.4.1 Halaman Login

Pengguna akan diarahkan ke halaman *login* saat pertama kali membuka sistem. Di halaman ini, pengguna diminta untuk memasukkan email dan *password* yang terdaftar untuk masuk ke dalam sistem. Halaman ini memiliki tampilan yang bersih dan terfokus. Gambar 4.19 menggambarkan desain antarmuka halaman *login* pada sistem.

Gambar 4.5 Desain Antarmuka *Login*

4.2.4.2 Halaman Dashboard

Halaman *dashboard* merupakan halaman utama yang pertama kali ditampilkan setelah pengguna berhasil *login*. Halaman ini menyajikan ringkasan performa sistem secara keseluruhan dalam format yang mudah dipahami secara sekilas. Gambar 4.20 menampilkan desain antarmuka Dashboard.

Gambar 4.6 Desain Antarmuka *Dashboard*

4.2.4.3 Halaman Peramalan

Halaman Peramalan memungkinkan pengguna untuk melihat hasil peramalan penjualan secara detail untuk setiap item menu. Di bagian atas halaman, terdapat *combobox* untuk memilih item menu yang ingin dilihat peramalannya. Setelah item dipilih, sistem menampilkan *line chart* yang memvisualisasikan perbandingan

antara data aktual dan prediksi. Gambar 4.21 menampilkan desain antarmuka halaman peramalan.



Gambar 4.7 Desain Antarmuka Peramalan

4.2.4.4 Halaman Kebutuhan Bahan Baku

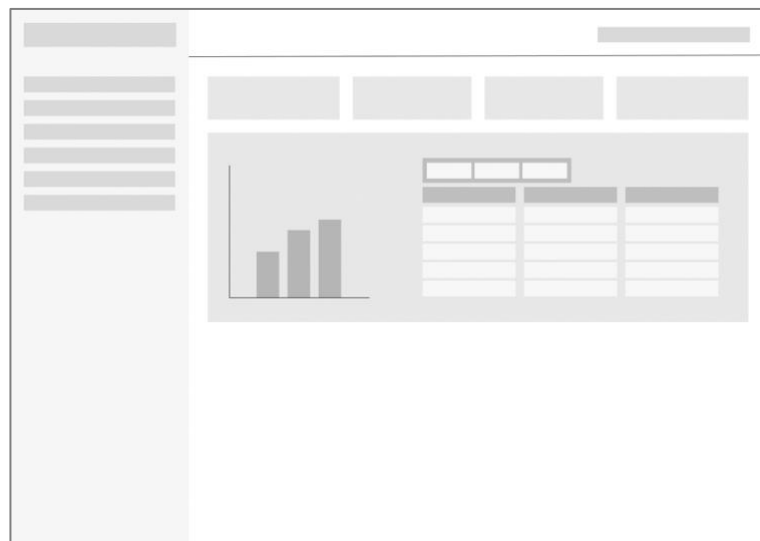
Halaman kebutuhan bahan baku merupakan salah satu fitur inti sistem yang secara langsung menjawab kebutuhan operasional Husgendam Coffee dalam perencanaan pengadaan bahan baku. Halaman ini menampilkan tabel kebutuhan bahan baku harian yang dihitung secara otomatis berdasarkan hasil peramalan dan data resep. Terdapat filter di bagian atas yang memungkinkan pengguna untuk menyaring data berdasarkan jenis bahan baku dan rentang tanggal. Di pojok kanan atas, terdapat tombol "Ekspor CSV" untuk mengunduh data yang sedang ditampilkan ke dalam format CSV. Gambar 4.22 menampilkan desain antarmuka halaman kebutuhan bahan baku.



Gambar 4.8 Desain Antarmuka Kebutuhan Bahan Baku

4.2.4.5 Halaman Analitik

Halaman Analitik menyajikan analisis mendalam mengenai performa model peramalan dan klasifikasi item menu menggunakan metode ABC. Di bagian atas halaman, terdapat empat *metric cards* yang menampilkan nilai R^2 , wMAPE, MAE, dan RMSE secara keseluruhan untuk model aktif. Di bawahnya, terdapat *bar chart* yang memvisualisasikan distribusi jumlah item per kelas ABC (A, B, C). Gambar 4.23 menampilkan desain antarmuka halaman Analitik.

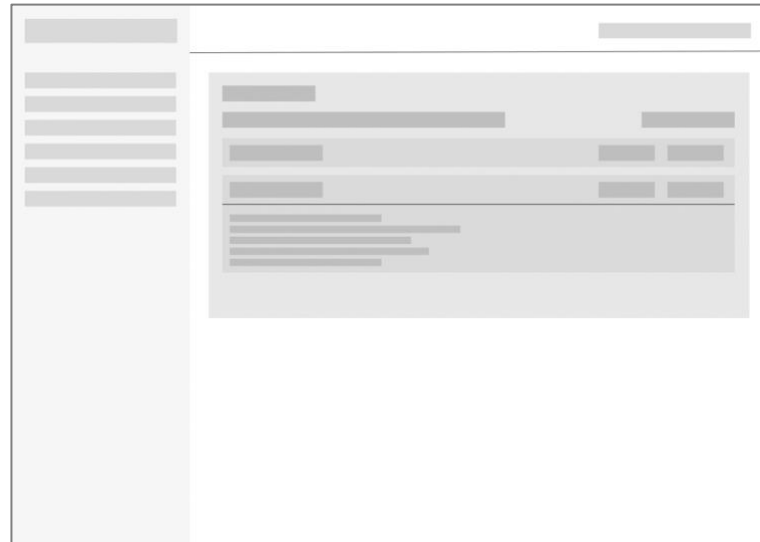


Gambar 4.9 Desain Antarmuka Analitik

4.2.4.6 Halaman Pengaturan

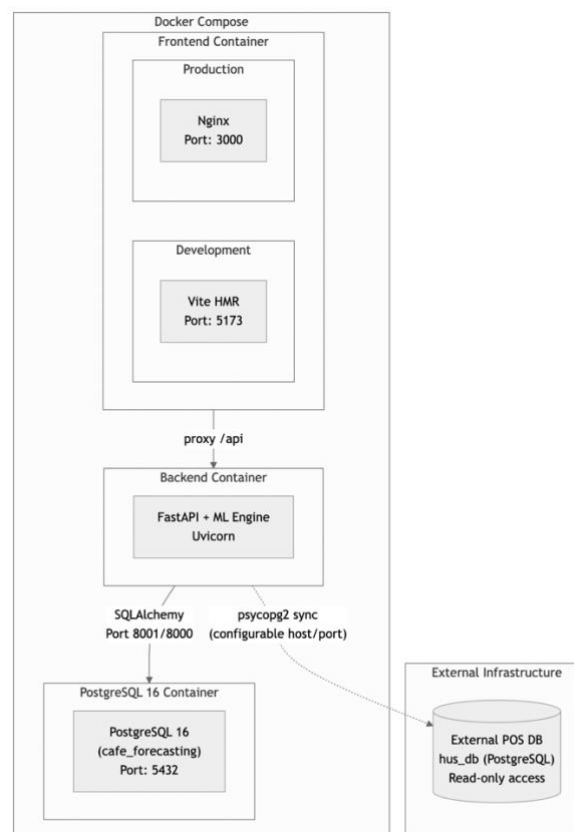
Halaman pengaturan merupakan halaman yang digunakan untuk mengelola proses pelatihan ulang model peramalan. Di halaman ini, pengguna dapat memulai

proses pelatihan dengan tombol “*Train*” dan tombol “*Cancel*” yang akan aktif selama proses pelatihan berlangsung. Area log di bawah kontrol pelatihan menampilkan *output* proses pelatihan secara *real-time*, diperbarui setiap 5 detik melalui mekanisme *polling*. Gambar 4.24 menampilkan desain antarmuka halaman pengaturan.



Gambar 4.10 Desain Antarmuka Pengaturan

4.2.5 Deployment Diagram



Gambar 4.11 *Deployment Diagram*

Pada Gambar 4.25 ditunjukkan bahwa sistem diimplementasikan menggunakan pendekatan kontainerisasi berbasis Docker. Seluruh komponen aplikasi dikemas ke dalam beberapa kontainer yang dikelola secara terpusat menggunakan *Docker Compose* sehingga proses *deployment*, konfigurasi, dan pemeliharaan sistem dapat dilakukan secara lebih konsisten.

Arsitektur *deployment* terdiri dari tiga kontainer utama yang saling terintegrasi. Kontainer pertama adalah *Frontend Container* yang menjalankan aplikasi web berbasis React 19. Kontainer ini bertugas menyajikan antarmuka pengguna yang digunakan oleh pengguna untuk mengakses seluruh fitur sistem, seperti *dashboard*, peramalan penjualan, analisis ABC, kebutuhan bahan baku, dan pengaturan model *machine learning*.

Kontainer kedua adalah *Backend Container* yang menjalankan aplikasi FastAPI berbasis Python. Kontainer ini berfungsi sebagai pusat logika bisnis sistem yang menangani proses autentikasi pengguna menggunakan JWT, pengelolaan data penjualan, proses inferensi dan pelatihan model *machine learning*, analisis klasifikasi ABC, perhitungan kebutuhan bahan baku berdasarkan *Bill of Materials* (BOM), serta penyediaan layanan REST API yang digunakan oleh *frontend*.

Kontainer ketiga adalah *Database Container* yang menjalankan PostgreSQL 16 sebagai basis data utama sistem. Basis data ini digunakan untuk menyimpan seluruh data operasional aplikasi, termasuk data pengguna, data penjualan, hasil peramalan, metrik performa model, data klasifikasi ABC, dan data resep produk.

Permintaan dari browser dikirim ke *Frontend Container*, kemudian *frontend* berkomunikasi dengan *Backend Container* melalui REST API berbasis HTTP/HTTPS. Selanjutnya, *Backend Container* melakukan proses bisnis yang diperlukan dan berinteraksi dengan *Database Container* untuk membaca maupun menyimpan data. Hasil pemrosesan kemudian dikembalikan ke *frontend* dan ditampilkan kepada pengguna dalam bentuk antarmuka yang interaktif.

4.3 Pengembangan Sistem

4.3.1 Pengumpulan dan Persiapan Data

Data yang digunakan dalam penelitian ini berasal dari sistem POS Kafe Husgendam yang mencakup periode 1 Januari 2022 hingga 28 Mei 2026,

merepresentasikan sekitar 4,5 tahun data transaksi harian. Data mentah diekstraksi dari dua sumber: *database* utama *cafe_forecasting* yang menyimpan agregat penjualan harian pada tabel *daily_item_sales*, dan *database* eksternal *POS_hus_db* yang berisi data transaksi mentah pada tabel *order_items* dan *orders*. Sinkronisasi dari *database* POS ke *database* utama dilakukan secara periodik melalui pencocokan nama produk.

Setelah melewati proses pembersihan awal, yang mencakup penghapusan item *add-on* (seperti "Add Sugar", "Add Ice") dan item yang dihentikan, *dataset* terdiri dari 61 item unik. Untuk analisis menyeluruh, dibangun *grid* penuh (tanggal $\times$ item) yang mencakup seluruh kombinasi termasuk hari tanpa penjualan (*Quantity* = 0). Hal ini memungkinkan pengukuran *zero-inflation* yang akurat, informasi yang hilang jika hanya melihat baris dengan penjualan. Tabel 4.9 menyajikan gambaran umum *dataset*.

Tabel 4.6 Gambaran Umum *Dataset* Penjualan

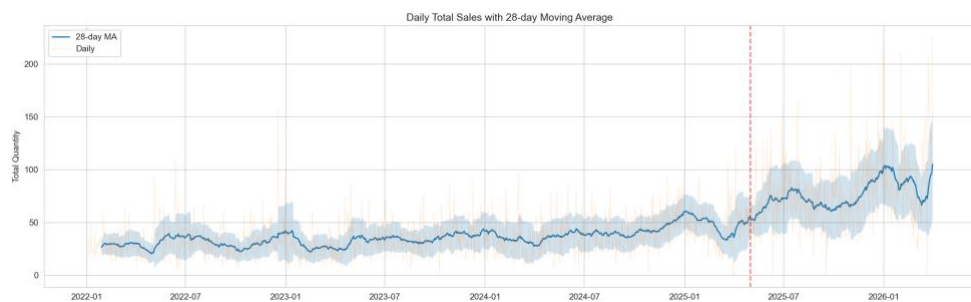
Metrik	Nilai
Rentang tanggal	Jan 2022 – Mei 2026 (~4,5 tahun)
Item unik	61 item
Total baris (<i>grid</i> penuh)	98.149 baris item-hari
Kombinasi item-hari non-nol	36.212 (37%)
Kombinasi item-hari nol	61.937 (63%)
Rata-rata per baris (non-nol)	2,38 cangkir/item/hari
Rata-rata per baris (termasuk nol)	1,61 cangkir/item/hari
Item teratas	Kopi Susu Husgendam Ice (6.368 cangkir total)
Item terbawah	Menawan (33 cangkir total)

4.3.2 Analisis Data Eksploratif

Setiap temuan EDA secara langsung mengarah pada keputusan rekayasa fitur yang dibahas pada Sub-bab 4.3.3.

1. Pola Temporal dan Tren Pertumbuhan

Analisis penjualan harian total menunjukkan tren pertumbuhan berkelanjutan sepanjang periode observasi. Rata-rata penjualan harian meningkat dari sekitar 29 cangkir/hari pada awal 2022 menjadi sekitar 105 cangkir/hari pada Maret 2026, peningkatan hampir $4\times$ lipat selama 4,5 tahun.

Gambar 4.12 Penjualan Harian Total dengan *Moving Average* 28 Hari

Untuk mengukur signifikansi perubahan struktural, dilakukan uji Cohen's d pada jendela 3-bulan berurutan. Tabel 4.7 menyajikan rata-rata penjualan per jendela 3-bulan beserta persentase perubahannya.

Tabel 4.7 Rata-rata Penjualan Harian per Jendela 3-Bulan

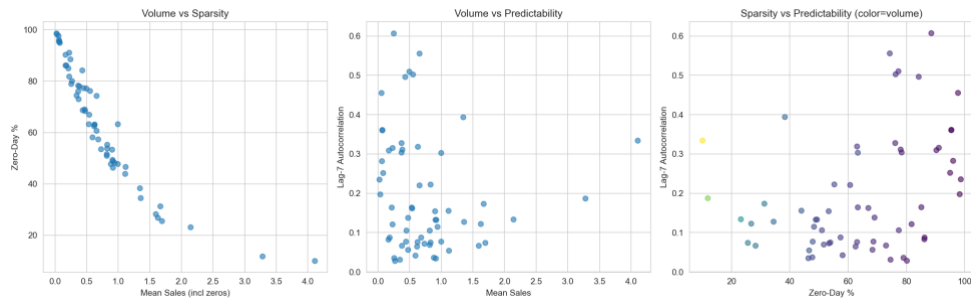
Periode	Rata-rata (cangkir/hari)	Perubahan
2022 Q1–Q4	28–34	Baseline
2023 Q1–Q4	27–39	+16% (year-end)
2024 Q1–Q4	33–50	+51% (year-end)
2025 Q1–Q4	45–85	+88% (year-end)
2026 Q1	100	+18% (vs Q4 2025)

Perubahan terbesar terdeteksi pada April 2025 (Cohen's $d = 1,531$, peningkatan +44% dari jendela sebelumnya), bertepatan dengan periode *rebranding* kafe. Namun, tren kenaikan sudah terlihat sejak akhir 2024, peningkatan dari rata-rata 39 cangkir/hari (Q3 2024) menjadi 50 cangkir/hari (Q4 2024), atau +28%. Pola ini mengindikasikan bahwa *rebranding* merupakan akselerator dari tren pertumbuhan yang sudah ada, bukan satu-satunya penyebab perubahan.

Implikasi untuk pemodelan adalah dikarenakan pertumbuhan bersifat gradual dan berkelanjutan, strategi yang dipilih adalah melatih model pada seluruh data historis (2022–2026) dengan menyertakan fitur yang menangkap tren temporal, bukan memotong data pada satu titik waktu. Pendekatan ini memiliki dua keunggulan: (1) data pre-2025 menyediakan $4\times$ lebih banyak sampel untuk mempelajari pola dasar seperti siklus hari dalam minggu dan musiman, dan (2) fitur tren seperti *DaysFromStart* dan *Item_Rank* secara implisit menangkap efek pertumbuhan tanpa memerlukan pemotongan eksplisit yang dapat membuang informasi.

2. Zero-Inflation

Dari total 98.149 kombinasi item-hari, sebanyak 63% bernilai nol, artinya, pada hari tertentu, sebagian besar item tidak terjual sama sekali. Gambar 4.13 menyajikan distribusi *zero-inflation* per item.



Gambar 4.13 Zero-Inflation per Item

Zero-inflation bervariasi secara ekstrem antar item. Tabel 4.8 menyajikan statistik *zero-inflation* untuk item terpilih.

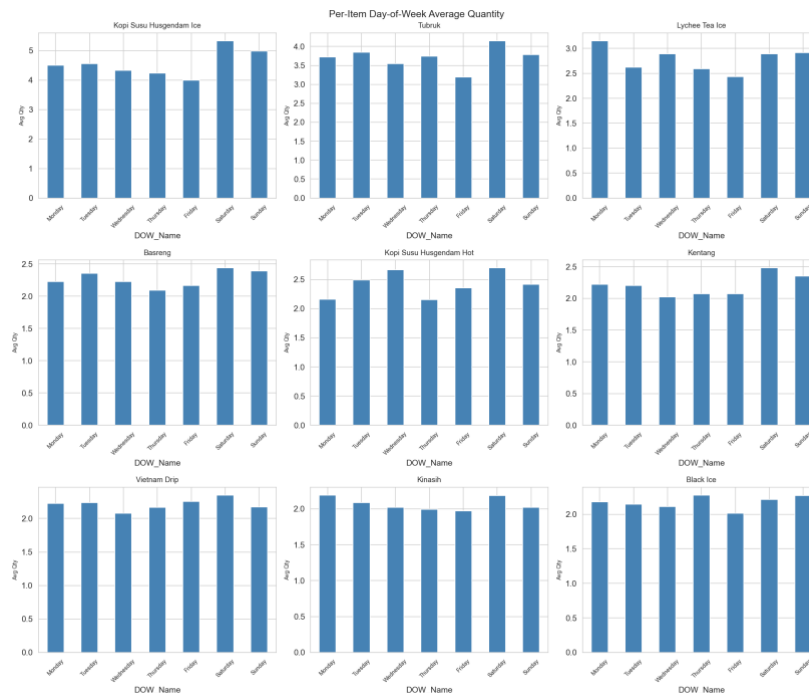
Tabel 4.8 Zero-Inflation Item Terpilih

Item	Zero%	Non-zero days	Rata-rata
Kopi Susu Husgendam Ice	10,1%	1.395	4,56
Tubruk	11,9%	1.367	3,72
Vietnam Drip	28,2%	1.113	2,22
Husgendam Platter	88,5%	178	1,72
Menawan	98,6%	21	1,57

Zero-inflation 63% memiliki konsekuensi fundamental terhadap arsitektur model. Pertama, model harus mampu membedakan antara "item tidak akan terjual hari ini" ($Quantity = 0$) dan "item akan terjual X cangkir" ($Quantity > 0$). Kedua, fungsi objektif harus sesuai untuk data dengan proporsi nol yang dominan, *Mean Squared Error* (MSE) yang mengasumsikan distribusi Gaussian kontinu tidak tepat untuk data diskrit dengan inflasi nol. Objective count:poisson dipilih karena secara matematis memodelkan probabilitas setiap nilai diskrit non-negatif, termasuk nol: $P(Y=0) = e^{(-\lambda)}$. Ketiga, diperlukan fitur yang secara spesifik menangkap "apakah item ini akan terjual?", mengarah pada pemilihan fitur *recency* seperti *Days_Since_Last_Sale* yang memiliki *Mutual Information* tertinggi dengan indikator biner *Is_Sale* (Sub-bab 4.3.3.2).

3. Pola Hari dalam Minggu

Analisis hari dalam minggu dilakukan pada dua level: agregat (seluruh item) dan per-item. Gambar 4.14 menyajikan rata-rata kuantitas per DOW pada level agregat.



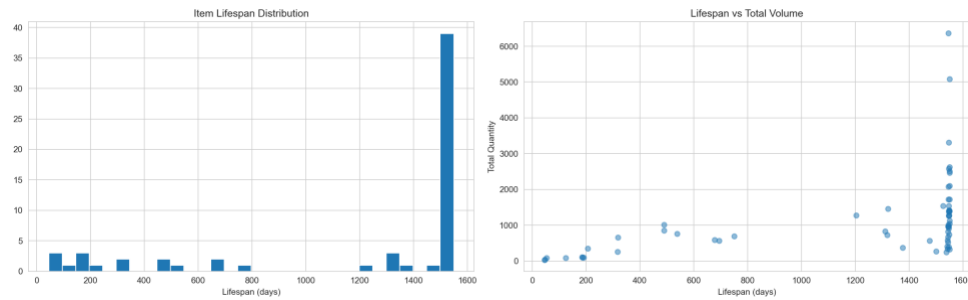
Gambar 4.14 Rata-rata Kuantitas per Hari dalam Minggu untuk 9 Item Teratas

Pada level agregat, Sabtu memiliki rata-rata tertinggi (2,18 cangkir/item) dan Jumat terendah (1,95 cangkir/item), variasi yang relatif kecil (~12%). Namun, pada level item individual, variasi DOW jauh lebih besar: Kopi Susu Husgendam Ice dapat berbeda 3–5 cangkir antara hari kerja dan akhir pekan.

Karena pola DOW sangat bervariasi antar item, fitur DOW harus bersifat per-item, bukan global. Ini mengarah pada dua keputusan: (1) penggunaan *one-hot encoding item* (Item_Kopi Susu Husgendam Ice, dll.) yang memungkinkan model mempelajari level dasar setiap item, dan (2) *encoding* siklikal DOW (DOW_Sin, DOW_Cos) yang menangkap sifat melingkar dari siklus mingguan tanpa mengasumsikan jarak linear antar hari.

4. Siklus Hidup Item dan Sparsitas

Dari 61 item, hanya 45 item yang sudah ada sejak awal 2022. Sebanyak 16 item diperkenalkan kemudian, 6 item pada 2024, 7 item pada 2025, dan 3 item pada 2026. Item-item baru memiliki data yang sangat terbatas (21–78 hari non-nol). Gambar 4.15 menyajikan distribusi umur item.



Gambar 4.15 Distribusi Umur Item

Item dengan data terbatas tidak dapat dilatih secara independen (model per-item memerlukan minimal ~ 100 sampel). Ini mengarah pada keputusan untuk menggunakan model global, satu model yang dilatih pada seluruh item sekaligus, di mana one-hot encoding item memungkinkan model mempelajari karakteristik spesifik setiap item sambil berbagi informasi tentang hubungan fitur-target yang bersifat umum (misalnya, efek `Days_Since_Last_Sale` terhadap probabilitas penjualan konsisten di seluruh item).

5. Distribusi Kuantitas Penjualan

Distribusi kuantitas penjualan harian per item merupakan temuan fundamental yang memengaruhi pemilihan metrik evaluasi. Tabel 4.9 menyajikan distribusi frekuensi.

Tabel 4.9 Distribusi Kuantitas Penjualan Harian per Item (hari non-nol)

Kuantitas (cangkir)	Frekuensi	Persentase	Kumulatif
1	17.325	50,9%	50,9%
2	8.192	24,1%	75,0%
3	3.998	11,7%	86,7%
4	2.018	5,9%	92,6%
5	1.073	3,2%	95,8%
6+	1.429	4,2%	100%

Sebanyak 50,9% dari seluruh hari dengan penjualan hanya menjual tepat 1 cangkir, dan 86,7% menjual 1–3 cangkir. Rata-rata kuantitas pada hari non-nol adalah 2,38 cangkir dengan median 2 cangkir.

Distribusi yang sangat menceng ke kanan dengan konsentrasi pada nilai 1–2 memiliki konsekuensi langsung terhadap metrik evaluasi. $wMAPE$ ($\sum |pred - actual| / \sum actual$) dengan denominator rata-rata 2,38 akan sangat

sensitif terhadap kesalahan kecil: prediksi 2 untuk aktual 1 sudah menyumbang error 100%. Oleh karena itu, MAE (*Mean Absolute Error*) dalam satuan cangkir dipilih sebagai metrik evaluasi utama, memberikan interpretasi operasional yang langsung: "rata-rata model meleset sebesar X cangkir per prediksi."

4.3.3 Rekayasa dan Seleksi Fitur

Setelah EDA mengidentifikasi karakteristik kunci data (*zero-inflation*, variasi DOW per-item, siklus hidup item), tahap selanjutnya adalah membangun dan menyeleksi fitur secara sistematis.

1. Fitur Kandidat

Dari data deret waktu, dibangun fitur yang dikelompokkan ke dalam tujuh kategori. Setiap kategori dirancang untuk menangkap aspek spesifik dari data yang diidentifikasi selama EDA. Tabel 4.13 menyajikan daftar lengkap fitur beserta justifikasi pemilihannya berdasarkan temuan EDA.

Tabel 4.10 Fitur Kandidat

Kategori	Jumlah	Fitur
Temporal Identity	16	DOW, Is_Weekend, Month, Year, WeekOfYear, DayOfMonth, Quarter, MonthStart, MonthEnd, Is_Holiday_Season, WeekOfMonth, DaysFromStart, DOW_Sin, DOW_Cos, Month_Sin, Month_Cos
Lifecycle	3	Days_Since_First_Sale, Item_Rank, Item_Rank_Pct
Recency	2	Days_Since_Last_Sale, Sales_Last_7D
Rolling/EWMA	8	Roll_Mean_7, Roll_Mean_14, Roll_Mean_28, Roll_Std_7, EWMA_7, EWMA_28, Trend_7_28, WoW_Change
Lags	4	Lag_1, Lag_7, Lag_14, Lag_28
Cross-item	5	Day_Total_Qty, Day_Total_Items_Sold, Day_Total_Beverage, Day_Total_Food, Day_Total_Qty_7D

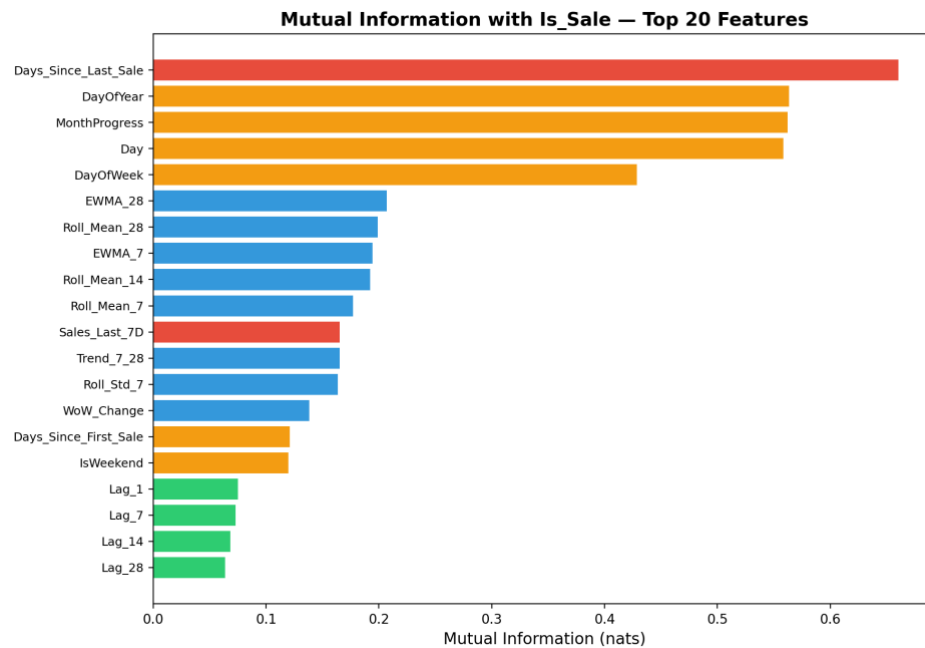
Kategori	Jumlah	Fitur
Item Identity	63	Item dummies (61) + Is_Beverage, Is_Food

Seluruh fitur dibangun dengan prinsip *past-only*, menggunakan `shift(1)` untuk memastikan tidak ada kebocoran target (target leakage). Nilai NaN diisi dengan 0 (interpretasi: tidak ada informasi historis). Nilai *infinity* yang muncul dari pembagian dengan nol pada fitur rasio (Trend_7_28, WoW_Change) di-clip ke rentang $[-5, 5]$, batas yang cukup lebar untuk menangkap perubahan ekstrem (+500%) tanpa membiarkan *outlier* numerik mendestabilkan pelatihan model.

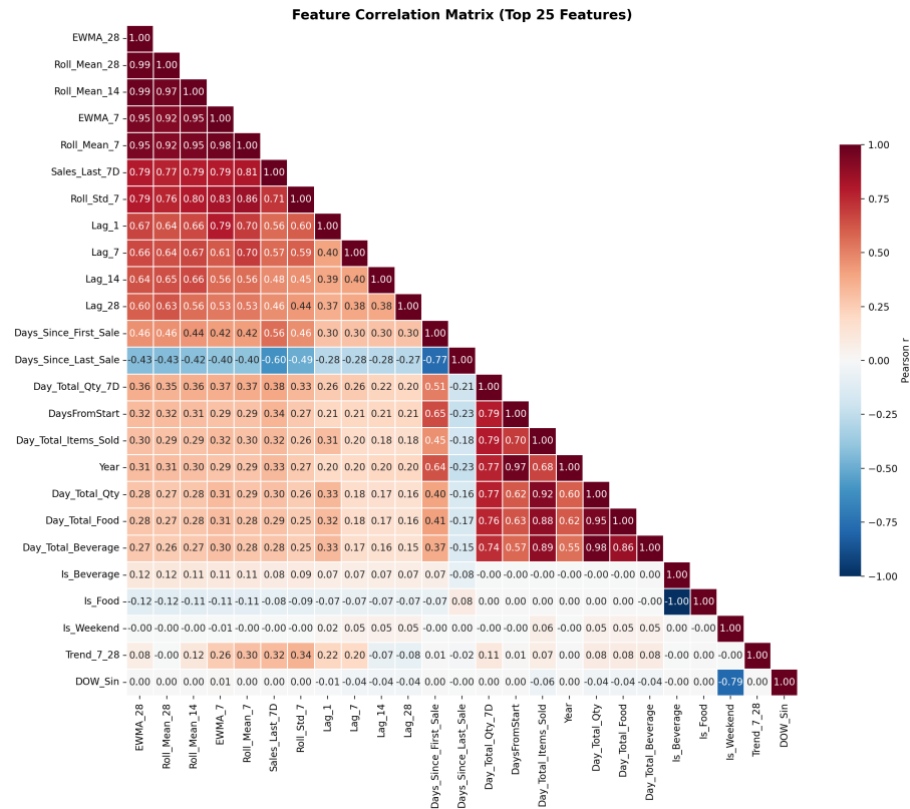
2. Seleksi Fitur

Untuk mengukur kontribusi informasional setiap fitur secara model-free, dihitung *Mutual Information* (MI) antara setiap fitur numerik dengan target Is_Sale (indikator biner apakah item terjual). MI dipilih karena tidak mengasumsikan bentuk hubungan linear, berbeda dengan korelasi Pearson yang hanya menangkap hubungan linear, MI mendeteksi hubungan non-linear, monotonik, maupun siklikal yang umum dalam data deret waktu. MI dihitung menggunakan *estimator* k-NN ($k=5$ tetangga) dari pustaka `scikit-learn` dan dinyatakan dalam satuan nats (*natural logarithm*; $1 \text{ nat} \approx 1,44 \text{ bit}$).

Days_Since_Last_Sale mendominasi dengan MI 0,656, lebih dari 3× lipat fitur berikutnya (EWMA_28: 0,203). Dominasi ini mengonfirmasi temuan EDA Sub-bab 4.3.2.2: pada kafe skala kecil dengan *zero-inflation* 63%, pertanyaan paling prediktif adalah "kapan terakhir kali item ini terjual?" Bukan "berapa rata-rata penjualannya?" atau "hari apa ini?", melainkan *recency*. Semakin lama sejak penjualan terakhir, semakin kecil probabilitas item terjual hari ini.

Gambar 4.16 *Mutual Information*

Fitur *rolling*/EWMA mendominasi peringkat 2–7, menunjukkan bahwa level penjualan terkini (rata-rata 7–28 hari) membawa informasi yang jauh lebih relevan daripada *lag* individual (*Lag_1*, *Lag_7*) yang memiliki MI lebih rendah ($< 0,10$). Ini mengindikasikan bahwa agregasi (*moving average*) lebih informatif daripada nilai titik tunggal, temuan yang konsisten dengan sifat data penjualan kafe yang bervariasi tinggi dari hari ke hari.



Gambar 4.17 Matriks Korelasi Antar Fitur

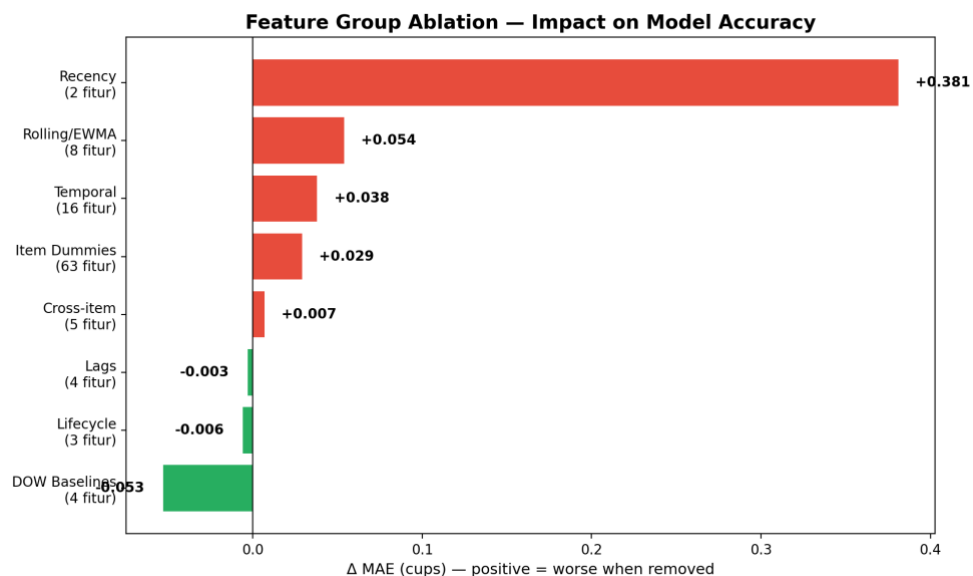
3. Ablasi Fitur

Untuk mengukur kontribusi setiap kelompok fitur secara langsung, dilakukan eksperimen ablasi: menghapus satu kelompok fitur pada satu waktu, melatih ulang model, dan mengukur perubahan MAE pada set validasi. Eksperimen ini memberikan bukti kausal, tentang pentingnya setiap kelompok. Tabel 4.11 menyajikan hasil ablasi.

Tabel 4.11 Hasil Ablasi Fitur

Kelompok Dihapus	Jumlah Fitur	MAE Setelah Ablasi	Δ MAE	Dampak
Recency	2	1,091	+0,381	+53,7%, PALING KRITIS
Rolling/EWMA	8	0,764	+0,054	+7,6%, Penting
Temporal Identity	16	0,748	+0,038	+5,4%, Moderat

Kelompok Dihapus	Jumlah Fitur	MAE Setelah Ablasi	Δ MAE	Dampak
Item Dummies	63	0,739	+0,029	+4,1%, Moderat
Cross-item	5	0,717	+0,007	+1,0%, Marginal
Lifecycle	3	0,704	-0,006	-0,8%, Netral
Lags	4	0,707	-0,003	-0,4%, Netral
DOW Baselines	4	0,657	-0,053	-7,5%, MERUGIKAN



Gambar 4.18 Hasil Ablasi Kelompok Fitur

Hasil ablasi mengonfirmasi temuan MI: *recency* adalah kelompok fitur yang penting, penghapusannya meningkatkan MAE sebesar 53,7%. Namun, ablasi juga menghasilkan temuan bahwa *DOW Baselines* justru merugikan model. Ketika fitur *DOW_Avg*, *DOW_Median*, *DOW_P75*, dan *DOW_N_Samples* dihapus, MAE justru turun sebesar 7,5% (dari 0,710 ke 0,657).

Investigasi lebih lanjut mengungkapkan penyebabnya: model sudah menangkap pola *DOW* melalui kombinasi *DOW_Sin*, *DOW_Cos*, *Is_Weekend*, dan *item dummies*. Menambahkan *DOW baselines*

menciptakan kolinearitas yang membingungkan model, beberapa fitur memberikan informasi yang sama melalui mekanisme berbeda, menyebabkan XGBoost kesulitan mengalokasikan *feature importance* secara optimal. Berdasarkan temuan ini, DOW *Baselines* dikeluarkan dari set fitur final. Keputusan ini didasarkan pada bukti eksperimental (ablasi).

4. Set Fitur Final

Berdasarkan seluruh analisis di atas, MI (Sub-bab 4.3.3.2) dan ablasi (Sub-bab 4.3.3.3), ditetapkan set fitur final yang akan digunakan dalam *pipeline* Bab V. Tabel 4.12 menyajikan ringkasan set fitur final.

Tabel 4.12 Set Fitur Final

Kategori	Jumlah	Keputusan
Temporal Identity	16	Dipertahankan, menangkap tren dan siklus
Lifecycle	3	Dipertahankan, penting untuk item baru
Recency	2	Dipertahankan, paling kritis (Δ MAE +53,7% jika dihapus)
Rolling/EWMA	8	Dipertahankan, kontribusi +7,6%
Lags	4	Dipertahankan, meskipun netral pada ablasi, memberikan sinyal momentum
Cross-item	5	Dipertahankan, kontribusi marginal (+1%) tetapi informatif
DOW Baselines	4	Dihapus, merugikan (Δ MAE -7,5%)
Item Identity	63	Dipertahankan, penting untuk model global

Set fitur final ini merepresentasikan hasil dari proses seleksi berbasis bukti yang sistematis, setiap fitur memiliki justifikasi dari EDA, setiap keputusan inklusi/eksklusi didukung oleh bukti kuantitatif (MI atau ablasi), dan tidak ada fitur yang dimasukkan berdasarkan intuisi semata.

4.3.4 Integrasi dan Pengembangan Sistem

Model yang telah dibangun kemudian diintegrasikan ke dalam sistem melalui API berbasis FastAPI. API ini menerima *input* berupa parameter model *type* dan rentang tanggal, kemudian mengembalikan hasil peramalan serta metrik evaluasi dalam format JSON. *Backend* utama yang dibangun menggunakan *framework*

FastAPI dengan SQLAlchemy asinkron untuk interaksi *database* dan autentikasi JWT untuk keamanan akses.

Sinkronisasi data dari sistem POS Kafe Husgendam dilakukan melalui *sync service* yang membaca data penjualan dari *database* eksternal ``hus_db`` secara *read-only*, kemudian menyinkronisasikannya ke *database* utama ``cafe_forecasting``. Sinkronisasi ini dapat dipicu secara manual sebelum menjalankan pelatihan ulang model untuk memastikan data pelatihan selalu mutakhir.

Perhitungan kebutuhan bahan baku diintegrasikan melalui komponen *Raw Material Processor* yang memanfaatkan data *Bill of Materials* (BOM). Setelah model menghasilkan peramalan penjualan, sistem mengalikan kuantitas peramalan dengan jumlah bahan yang diperlukan per item sesuai resep BOM dan mengekspansi *condiment* menjadi sub-bahan penyusunnya. Hasil perhitungan dapat diekspor ke format CSV.

Pada tahap akhir, antarmuka pengguna dikembangkan sebagai *Single Page Application* (SPA) menggunakan React 19 dengan TypeScript. Untuk mendukung desain yang konsisten dan responsif, digunakan Tailwind CSS dalam pengelolaan tampilan visual dan shadcn/ui sebagai *library* komponen antarmuka. Visualisasi data peramalan dan analitik menggunakan Recharts. Pengambilan data dari server menggunakan TanStack Query, sedangkan navigasi antar halaman menggunakan TanStack Router dengan pendekatan *file-based routing*.

4.4 Pengujian

Pengujian sistem dilakukan untuk memastikan bahwa sistem yang telah dikembangkan berfungsi dengan baik, sesuai dengan spesifikasi kebutuhan fungsional dan non-fungsional yang telah ditetapkan, serta dapat digunakan dengan nyaman oleh pengguna akhir. Dalam penelitian ini, pengujian dilakukan menggunakan dua metode utama, yaitu pengujian model untuk mengevaluasi akurasi peramalan, serta *System Usability Scale* (SUS) untuk mengukur tingkat kemudahan penggunaan sistem dari perspektif pengguna.

4.4.1 Pengujian Model

Pengujian model dilakukan untuk mengevaluasi tingkat akurasi sistem dalam melakukan peramalan penjualan berdasarkan data historis yang belum pernah digunakan selama pelatihan. Dalam tahap ini, kedua model yang telah dilatih diuji

menggunakan data *holdout* selama 12 minggu terakhir dari *dataset*, sebagaimana ditetapkan dalam Bab III. Pemilihan 12 minggu sebagai periode *holdout* didasarkan pada pertimbangan bahwa periode ini cukup panjang untuk mengukur kemampuan generalisasi jangka menengah model, sekaligus mewakili kondisi operasional nyata di mana peramalan dibutuhkan minimal untuk kebutuhan perencanaan satu kuartal ke depan.

Setiap item menu akan diproses oleh sistem untuk menghasilkan peramalan penjualan, yang kemudian dibandingkan dengan data aktual pada periode *holdout*. Perbandingan dilakukan dengan menghitung metrik evaluasi yang mencakup R^2 , wMAPE, MAE, dan RMSE.

Selain evaluasi metrik agregat, dilakukan juga evaluasi per kelas ABC (A, B, C) untuk memahami distribusi performa model pada kelompok item dengan karakteristik volume yang berbeda. Item kelas A yang memiliki volume penjualan tinggi cenderung memiliki pola yang lebih stabil dan lebih mudah diramalkan, sedangkan item kelas C dengan volume rendah cenderung memiliki *noise* yang lebih tinggi.

Melalui proses ini, diharapkan dapat diperoleh gambaran seberapa akurat sistem dalam meramalkan permintaan produk serta apakah sistem dapat membantu pengguna dalam pengambilan keputusan terkait pengadaan bahan baku. Hasil dari pengujian ini menjadi dasar dalam mengevaluasi efektivitas dan reliabilitas model sebelum diimplementasikan secara penuh dalam operasional Husgendam Coffee.

4.4.2 Pengujian System Usability Scale (SUS)

Setelah pengujian fungsional dan pengujian model, pengujian selanjutnya adalah pengujian *System Usability Scale* (SUS). SUS merupakan metode evaluasi *usability* yang bersifat subjektif namun terstandarisasi, yang digunakan untuk mengukur sejauh mana sistem mudah digunakan oleh pengguna dari perspektif pengalaman nyata. Pengujian ini dilakukan dengan memberikan kuesioner yang terdiri dari 10 pernyataan kepada responden.

Skor SUS dihitung menggunakan formula standar yang menghasilkan nilai antara 0 hingga 100. Skor di atas 70 dikategorikan sebagai *acceptable*, skor di atas 80 dikategorikan sebagai *excellent*, dan skor di atas 90 dikategorikan sebagai *best imaginable*. Target skor minimal yang ditetapkan dalam penelitian ini adalah

kategori *acceptable* dengan skor di atas 70, sebagaimana ditetapkan dalam rumusan masalah ketiga pada Bab I.

Kuesioner SUS diberikan dalam bentuk *Google Form* dengan format yang mencakup: nama, jabatan, serta penilaian terhadap 10 pernyataan menggunakan skala *Likert* 5 poin (1 = Sangat Tidak Setuju, 2 = Tidak Setuju, 3 = Netral, 4 = Setuju, 5 = Sangat Setuju). Tabel 4.12 berikut ini akan menunjukkan daftar pernyataan dalam pengujian SUS.

4.5 Pemeliharaan

Tahap pemeliharaan (*maintenance*) merupakan bagian akhir dari siklus pengembangan sistem yang bertujuan untuk memastikan bahwa sistem tetap berjalan dengan baik dan memberikan nilai yang optimal setelah implementasi. Pada tahap ini, dilakukan pemantauan secara berkala terhadap kinerja sistem, baik dari sisi fungsionalitas maupun performa teknis.

Dalam konteks Sistem Peramalan Persediaan Bahan Baku Husgendam Coffee, pemeliharaan mencakup beberapa aspek utama. Pertama, pemantauan performa model peramalan dilakukan secara berkala dengan mengevaluasi data penjualan baru yang masuk untuk mengidentifikasi kemungkinan penurunan akurasi model (*model drift*) seiring berjalannya waktu. Hal ini penting mengingat dinamika permintaan di industri F&B yang dapat berubah akibat faktor musiman, perubahan menu, atau perubahan perilaku konsumen. Jika ditemukan penurunan performa yang signifikan, proses *retraining* model dapat direncanakan dengan menggunakan data terbaru melalui fitur *retrain* yang tersedia pada halaman pengaturan. Kemampuan *retrain* tanpa kode ini merupakan salah satu keunggulan sistem yang memungkinkan manajemen Husgendam Coffee untuk memperbarui model secara mandiri tanpa memerlukan keterlibatan pengembang.

Kedua, pemeliharaan antarmuka dan komponen perangkat lunak dilakukan untuk memastikan tampilan tetap responsif dan kompatibel dengan berbagai perangkat dan browser modern. Pembaruan tampilan, penyesuaian komponen antarmuka, atau pengoptimalan interaksi pengguna dilakukan secara berkala sesuai kebutuhan dan berdasarkan masukan dari pengguna. Selain itu, dependensi *library* dan *framework* yang digunakan perlu diperbarui secara berkala untuk mendapatkan perbaikan keamanan, peningkatan performa, dan dukungan fitur terbaru.

Ketiga, pemantauan log aktivitas sistem dilakukan untuk mendeteksi *error* atau anomali yang mungkin terjadi dalam operasional sistem. Hal ini mencakup pemantauan respons layanan API, *error rate*, dan waktu respons rata-rata untuk memastikan sistem beroperasi dalam batasan yang dapat diterima.

BAB V IMPLEMENTASI DAN PENGUJIAN

5.1 Implementasi Pipeline Machine Learning

5.1.1 Arsitektur *Pipeline*

Pipeline dirancang sebagai model global tunggal yang memproses seluruh 61 item secara bersamaan. Arsitektur ini merupakan konsekuensi langsung dari tiga temuan EDA: (1) *zero-inflation* 63% memerlukan objective `count:poisson` yang didukung XGBoost; (2) 16 item baru dengan data terbatas tidak dapat dilatih secara independen, sehingga diperlukan model global yang berbagi informasi antar item melalui *one-hot encoding*; (3) ablasi menunjukkan DOW *baselines* merugikan, sehingga fitur DOW ditangkap melalui *encoding* siklikal dan item *dummies* tanpa statistik DOW eksplisit.

5.1.2 Akuisisi dan Pemrosesan Data

1. Strategi Data

Pipeline ini menggunakan seluruh data historis dari Januari 2022 hingga Mei 2026. Keputusan ini didasarkan pada analisis tren Sub-bab 4.3.2.1 yang menunjukkan bahwa pertumbuhan bersifat gradual dan berkelanjutan, bukan perubahan *step-function* pada satu titik waktu. Dengan menggunakan seluruh data, model memperoleh $4\times$ lebih banyak sampel untuk mempelajari hubungan non-linear antara fitur dan target. Tabel 5.1 menyajikan ringkasan *dataset* yang digunakan untuk pelatihan.

Tabel 5.1 Ringkasan *Dataset* Pelatihan

Metrik	Nilai
Total baris (grid penuh, termasuk nol)	98.149
Baris non-nol	36.212 (37%)
Item unik	61
Rentang	2022-01-01 s.d. 2026-05-28
Fitur	101 (Tabel 4.17)

Dengan $4\times$ lebih banyak sampel dibandingkan hanya menggunakan data setahun terakhir, model memiliki lebih banyak observasi untuk mempelajari hubungan non-linear antara fitur dan target.

2. Ekspansi *Grid* dan *Zero-Inflation*

Data mentah dari CSV dan `hus_db` hanya berisi baris dengan *Quantity* ≥ 1 . Untuk mengatasi *zero-inflation* 63% yang diidentifikasi pada Sub-bab 4.3.2.2, dilakukan ekspansi *grid* penuh: setiap kombinasi (tanggal, item) dibuat, dan *Quantity* diisi 0 untuk kombinasi yang tidak ada dalam data mentah. Proses ini menghasilkan 98.149 baris dari 36.212 baris mentah, peningkatan $2,7\times$ yang seluruhnya berasal dari penambahan baris nol.

Grid penuh memungkinkan model mempelajari pola ketidakhadiran item, informasi yang krusial karena 63% dari seluruh prediksi akan bernilai nol. Tanpa *grid* penuh, model hanya akan melihat hari dengan penjualan dan tidak pernah belajar membedakan kondisi "item tidak akan terjual."

5.1.3 Rekayasa Fitur

1. Implementasi Fitur Final

Fungsi `build_features()` mengimplementasikan 101 fitur yang telah diseleksi pada Bab IV Sub-bab 4.3.3.4. Setiap fitur dibangun dengan prinsip *past-only* menggunakan `shift(1)`, nilai hari ini tidak termasuk dalam perhitungan fitur untuk hari ini.

2. Strategi Fitur untuk Peramalan Multi-Hari

Peramalan 7 hari ke depan menghadapi tantangan fundamental: fitur yang dihitung dari data aktual (seperti *rolling mean*) tidak tersedia untuk hari masa depan. Jika fitur diperbarui dengan prediksi model sendiri, kesalahan terakumulasi secara rekursif, prediksi hari ke-1 yang meleset merusak fitur hari ke-2, yang merusak prediksi hari ke-2, dan seterusnya.

Untuk mengatasi ini, diterapkan strategi hibrida yang membedakan fitur berdasarkan ketergantungannya pada data aktual. Tabel 5.2 menyajikan klasifikasi dan strategi untuk setiap kelompok fitur.

Tabel 5.2 Strategi Fitur untuk Peramalan Multi-Hari

Kelompok	Strategi	Alasan
Rolling/EWMA, Lags	Dibekukan, dihitung sekali dari data aktual terakhir, tidak diperbarui	Sangat sensitif terhadap akumulasi error; satu prediksi buruk

Kelompok	Strategi	Alasan
		mengkontaminasi seluruh jendela rolling
Recency	Diperbarui, Days_Since_Last_Sale dan Sales_Last_7D dihitung ulang setiap hari dari prediksi	Biaya $O(1)$ per item; esensial untuk akurasi (feature importance 0,73); tidak terakumulasi karena bersifat diskrit
Cross-item	Diperbarui, agregat harian dihitung dari prediksi seluruh item	Diperlukan untuk konsistensi; tidak bisa dibekukan karena nilainya berubah setiap hari
Temporal, Lifecycle, Identity	Deterministik, dihitung langsung dari tanggal target	Tidak bergantung pada data aktual

Pendekatan ini menghasilkan *trade-off* yang terukur: MAE 1,29 pada peramalan rekursif 7 hari (dengan strategi hibrida) vs MAE 1,67 (dengan pembaruan penuh) vs MAE 0,71 (dengan fitur aktual, ceiling yang tidak praktis). Strategi hibrida mencapai 77% dari *ceiling* akurasi sambil mempertahankan stabilitas.

3. Weekend Recency Reset

Observasi dari EDA Sub-bab 4.3.2.3 menunjukkan bahwa akhir pekan (Jumat, Sabtu, Minggu) memiliki *traffic* kafe yang lebih tinggi. Namun, jika item kebetulan tidak terjual pada hari jangkar (hari terakhir data aktual), fitur Days_Since_Last_Sale akan bernilai tinggi, menyebabkan model memprediksi nol untuk seluruh horizon peramalan, terlepas dari hari apa.

Untuk mencegah hal ini, diterapkan *weekend recency reset*: pada hari Jumat, Sabtu, dan Minggu, Days_Since_Last_Sale di-reset ke 0 sebelum prediksi. Ini adalah bentuk *encoding* pengetahuan domain ke dalam *pipeline* tanpa mengubah arsitektur model.

5.1.4 Pelatihan *Baseline*

Sebagai langkah awal, kedua model dilatih dengan parameter *default* pada data yang sama (101 fitur, 98.149 baris) untuk memperoleh *baseline* performa sebelum *tuning*. XGBoost menggunakan objective count:poisson, sementara Random Forest secara implisit meminimasi *Mean Squared Error* (MSE).

Tabel 5.3 Perbandingan *Baseline*

Model	MAE	R ² (non-zero)
XGBoost Poisson	0,782	0,259
Random Forest	0,803	0,234

XGBoost unggul pada *baseline*, *objective* Poisson lebih sesuai untuk data *count* dengan *zero-inflation* dibandingkan MSE. Langkah selanjutnya: *tuning* kedua model untuk melihat apakah performa dapat ditingkatkan.

5.1.5 Hyperparameter Tuning

1. Metodologi Tuning

Tuning dilakukan pada kedua model menggunakan *grid search* sekuensial dengan validasi 3-fold *rolling window*. Untuk XGBoost, 8 parameter di-*tune*; untuk Random Forest, 4 parameter di-*tune*. Setiap konfigurasi dievaluasi menggunakan MAE rata-rata dari 3 *fold* validasi.

Tabel 5. 4 *Tuning* XGBoost

```
param_grid = [
    {'max_depth': [3,4,5,6], 'learning_rate': [0.03,0.05,0.1],
    'n_estimators': [200,300]},
    {'subsample': [0.6,0.8,1.0], 'colsample_bytree':
    [0.6,0.8,1.0]},
    {'reg_alpha': [0,0.5,1.0,2.0], 'reg_lambda':
    [0.5,1.0,2.0,5.0], 'min_child_weight': [1,3,5]},
]

best_mae = float('inf')
best_params = None

for rnd, grid in enumerate(param_grid):
    for params in ParameterGrid(grid):
        maes = []
        for w in range(3): # 3-fold rolling validation
```

```

        train = data[data['Date_Only'] < val_start +
timedelta(days=7*w)]

        val = data[(data['Date_Only'] >= val_start +
timedelta(days=7*w)) &
                    (data['Date_Only'] <= val_start +
timedelta(days=7*w+6))]

        model = XGBRegressor(objective='count:poisson',
random_state=42, verbosity=0, **params)

        model.fit(train[features].fillna(0),
train['Quantity'])

        preds =
np.maximum(model.predict(val[features].fillna(0)), 0)

        maes.append(mean_absolute_error(val['Quantity'],
preds))

        if np.mean(maes) < best_mae:
            best_mae = np.mean(maes)
            best_params = params

```

Rentang setiap parameter dipilih berdasarkan praktik standar: `max_depth` 3–6 mencakup pohon dangkal hingga moderately deep, `learning_rate` 0,03–0,1 mencakup pembelajaran lambat hingga agresif, `n_estimators` 200–300 mencakup *ensemble* kecil hingga menengah. Pembagian menjadi tiga ronde memungkinkan eksplorasi 81 kombinasi secara efisien tanpa *exhaustive search*.

Tabel 5.5 *Tuning* Random Forest

```

rf_grid = {'n_estimators': [100, 200, 300],
           'max_depth': [5, 7, 10],
           'min_samples_leaf': [1, 5, 10],
           'max_features': [0.5, 1.0]}

best_rf_mae = float('inf')
best_rf_params = None

for params in ParameterGrid(rf_grid):
    maes = []
    for w in range(3):
        train = data[data['Date_Only'] < val_start +
timedelta(days=7*w)]

        val = data[(data['Date_Only'] >= val_start +
timedelta(days=7*w)) &

```

```

        (data['Date_Only'] <= val_start +
timedelta(days=7*w+6))])

        model = RandomForestRegressor(random_state=42, n_jobs=-
1, **params)

        model.fit(train[features].fillna(0), train['Quantity'])

        preds =
np.maximum(model.predict(val[features].fillna(0)), 0)

        maes.append(mean_absolute_error(val['Quantity'], preds))

        if np.mean(maes) < best_rf_mae:
            best_rf_mae = np.mean(maes)
            best_rf_params = params

```

2. Hasil Tuning XGBoost

Tabel 5.6 Hasil Grid Search XGBoost

Ronde	Parameter Terpilih	MAE Validasi	Perbaikan
Baseline	depth=4, lr=0,1, n=200, sub=0,8, col=0,8, α =0,5, λ =0,5, cw=1	0,780	—
Ronde 1	depth=6, lr=0,1, n=300	0,759	-2,7%
Ronde 2	sub=1,0, col=0,6	0,755	-0,5%
Ronde 3	α =0,5, λ =2,0, cw=5	0,748	-0,9%

Tuning XGBoost menghasilkan perbaikan 4,1% pada validasi (MAE 0,780 $\rightarrow$ 0,748). Namun, ketika diuji pada *backtest* penuh 8 periode, perbaikan tersebut tidak signifikan, MAE 0,782 (*baseline*) vs 0,779 (*tuned*). Ini mengindikasikan parameter *default* sudah *near-optimal*.

3. Hasil Tuning Random Forest

Tabel 5.7 Hasil Grid Search Random Forest

Parameter	Nilai Diuji	<i>Baseline</i>	<i>Tuned</i>	Dampak
n_estimators	100, 200, 300	200	200	Tidak signifikan
max_depth	5, 7, 10	10	10	Depth 10 optimal
min_samples_leaf	1, 5, 10	5	5	Tidak berubah

max_features	0.5, 1.0	1.0	1.0	Tidak berubah
--------------	----------	-----	-----	---------------

Tuning Random Forest tidak menghasilkan perbaikan signifikan, parameter *baseline* (n=200, depth=10, leaf=5) sudah *near-optimal*. MAE *baseline* 0,803 tidak berubah secara signifikan setelah *tuning*.

4. Perbandingan Setelah Tuning

Tabel 5.8 erbandingan Setelah Tuning (8-Period Backtest)

Model	MAE	R ² (non-zero)
XGBoost (tuned)	0,779	0,242
Random Forest (tuned)	0,803	0,234

Setelah *tuning*, XGBoost tetap unggul, margin perbaikan kecil (<1%) karena kedua model sudah *near-optimal*. XGBoost dipilih sebagai model final dengan parameter *default* (depth=4, n=200, lr=0,1) yang sudah mencukupi. Evaluasi menyeluruh model final disajikan pada Bab VI.

5.1.6 Pelatihan Final

Kedua model dilatih dengan parameter *baseline* (*tuning* tidak menghasilkan perbaikan signifikan pada keduanya, Sub-bab 5.1.5.4). Kode 5.3 menunjukkan pelatihan final.

```
# XGBoost
xgb_model = XGBRegressor(
    objective='count:poisson',
    n_estimators=200, max_depth=4, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    reg_alpha=0.5, reg_lambda=0.5,
    random_state=42, verbosity=0,
)
xgb_model.fit(X_train[features].fillna(0), y_train)

# Random Forest
rf_model = RandomForestRegressor(
    n_estimators=200, max_depth=10, min_samples_leaf=5,
    random_state=42, n_jobs=-1,
)
rf_model.fit(X_train[features].fillna(0), y_train)
```

5.1.7 Peramalan Multi Hari

Pipeline mendukung horizon peramalan fleksibel yang dikonfigurasi melalui parameter `--days`. *Default* 7 hari dipilih karena sesuai dengan siklus operasional kafe, pemesanan bahan baku dilakukan mingguan dengan *lead time* 2–3 hari dari pemasok. Horizon yang lebih panjang dapat digunakan, namun akurasi akan menurun seiring bertambahnya jarak dari data aktual terakhir akibat degradasi fitur beku.

Peramalan diimplementasikan dalam fungsi `forecast()` menggunakan strategi fitur hibrida yang dijelaskan pada Sub-bab 5.1.3.2. Algoritma bekerja dalam lima langkah per hari:

1. Build features: Fitur temporal, lifecycle, dan identity dihitung dari tanggal target. Recency dihitung dari history yang telah diperbarui. Rolling/lag diambil dari history beku.
2. Weekend check: Jika hari adalah Jumat, Sabtu, atau Minggu, `Days_Since_Last_Sale` di-reset ke 0 (Sub-bab 5.1.3.3).
3. Predict: Model memprediksi seluruh 61 item sekaligus.
4. Store: Hasil disimpan ke DataFrame output.
5. Update: Recency history diperbarui dengan prediksi ($O(1)$ per item). Cross-item features diperbarui dari agregat prediksi.

Pendekatan ini memproses seluruh 61 item dalam satu panggilan `model.predict()`, efisien secara komputasi.

5.1.8 Buffer Persediaan Newsvendor

Model memprediksi nilai ekspektasi (*mean*) dari distribusi penjualan. Untuk perencanaan persediaan, diperlukan *buffer* di atas ekspektasi untuk mengakomodasi ketidakpastian. Besaran *buffer* ditentukan menggunakan model newsvendor:

$$Buffer_i = z \times \sigma_{e,i}$$

di mana $\sigma_{e,i}$ adalah standar deviasi *error* prediksi item i , diestimasi dari backtest 8 periode dan z adalah z-score yang diturunkan dari *critical ratio*:

$$CR = \frac{c_u}{c_u + c_o}$$

dengan c_u = biaya per unit understock (margin hilang per cangkir) dan c_o = biaya per unit overstock (*spoilage* per cangkir). CR menentukan service level optimal:

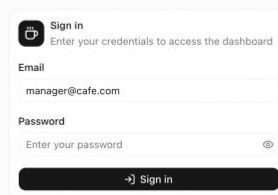
semakin tinggi biaya understock relatif terhadap overstock, semakin tinggi buffer yang diperlukan. Nilai z diperoleh dari invers fungsi distribusi kumulatif normal standar ($z = \Phi^{-1}(CR)$), misalnya, $CR=75\%$ menghasilkan $z \approx 0,67$, $CR=95\%$ menghasilkan $z \approx 1,645$.

5.2 Implementasi Antarmuka

Implementasi antarmuka pada Sistem Peramalan Persediaan Bahan Baku Husgendam Coffee dilakukan dengan mengikuti desain antarmuka yang telah dirancang pada Bab IV. *Frontend* dibangun sebagai *Single Page Application* (SPA) menggunakan React 19 dengan TypeScript, komponen shadcn/ui dengan *styling* Tailwind CSS, dan visualisasi data menggunakan Recharts. Berikut adalah tampilan antarmuka yang telah diimplementasikan.

5.2.1 Halaman *Login*

Pengguna akan diarahkan ke halaman *login* saat pertama kali membuka sistem. Di halaman ini, pengguna diminta untuk memasukkan email dan *password* yang terdaftar untuk masuk ke dalam sistem. Halaman ini memiliki tampilan yang bersih dan terfokus. Gambar 5.1 menggambarkan halaman *login* yang telah diimplementasikan.

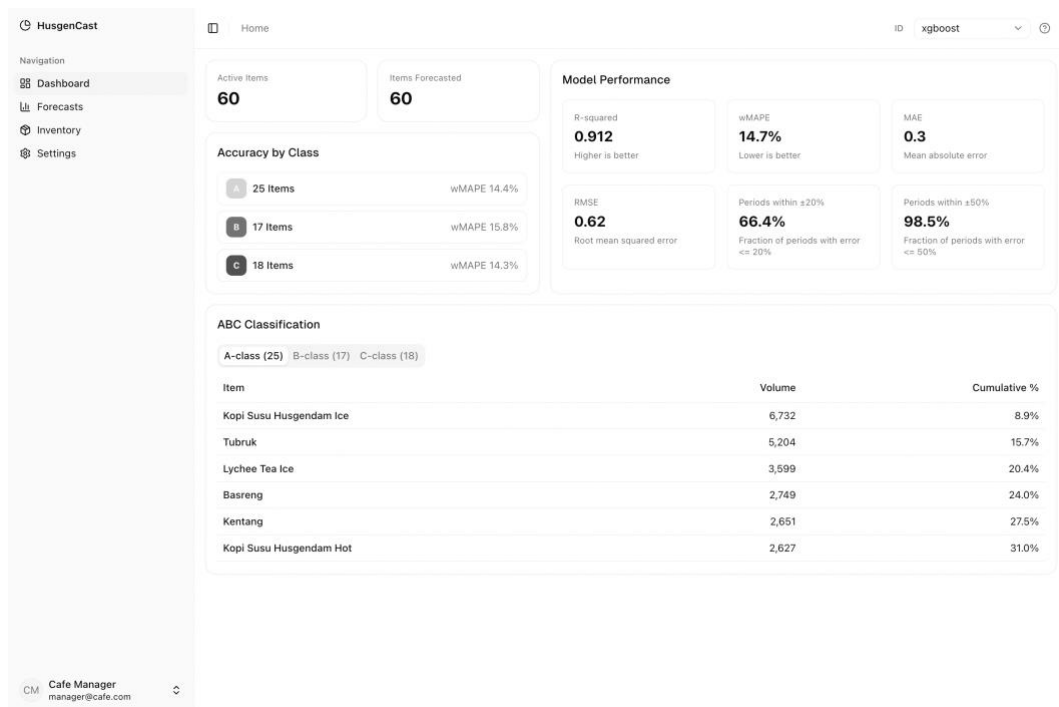


The image shows a login form titled "Sign in" with the subtitle "Enter your credentials to access the dashboard". It contains two input fields: "Email" with the placeholder text "manager@cafe.com" and "Password" with the placeholder text "Enter your password". There is a toggle icon for the password field. At the bottom, there is a "Sign in" button with a right-pointing arrow.

Gambar 5.1 Implementasi Halaman *Login*

5.2.2 Halaman Dashboard

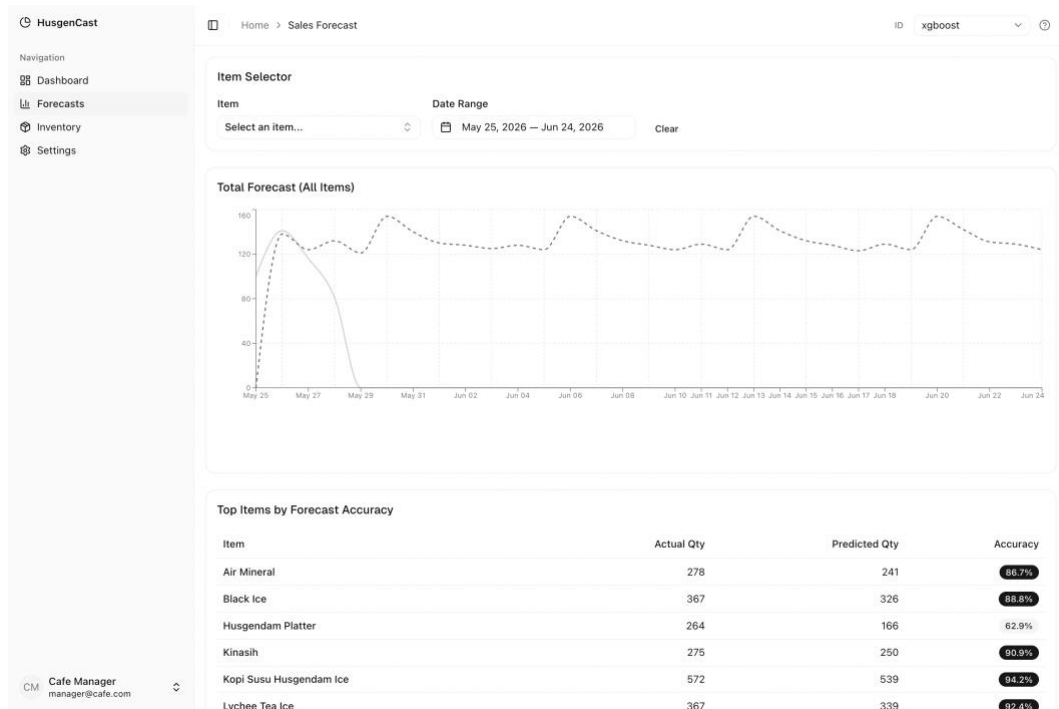
Halaman *dashboard* merupakan halaman utama yang pertama kali ditampilkan setelah pengguna berhasil *login*. Halaman ini menyajikan ringkasan performa sistem secara keseluruhan dalam format yang mudah dipahami secara sekilas. Gambar 5.2 menampilkan halaman *Dashboard* yang telah diimplementasikan.



Gambar 5. 2 Implementasi Halaman *Dashboard*

5.2.3 Halaman Peramalan

Halaman Peramalan memungkinkan pengguna untuk melihat hasil peramalan penjualan secara detail untuk setiap item menu. Di bagian atas halaman, terdapat *combobox* untuk memilih item menu yang ingin dilihat peramalannya. Setelah item dipilih, sistem menampilkan *line chart* yang memvisualisasikan perbandingan antara data aktual dan prediksi. Gambar 5.3 menampilkan halaman peramalan yang telah diimplementasikan.



Gambar 5.3 Implementasi Halaman Peramalan

5.2.4 Halaman Kebutuhan Bahan Baku

Halaman kebutuhan bahan baku merupakan salah satu fitur inti sistem yang secara langsung menjawab kebutuhan operasional Husgendam Coffee dalam perencanaan pengadaan bahan baku. Halaman ini menampilkan tabel kebutuhan bahan baku harian yang dihitung secara otomatis berdasarkan hasil peramalan dan data resep. Terdapat filter di bagian atas yang memungkinkan pengguna untuk menyaring data berdasarkan jenis bahan baku dan rentang tanggal. Di pojok kanan atas, terdapat tombol "Ekspor CSV" untuk mengunduh data yang sedang ditampilkan ke dalam format CSV. Gambar 5.4 menampilkan halaman kebutuhan bahan baku yang telah diimplementasikan.

The screenshot displays the 'Daily Material Requirements' page in the HusgenCast application. The page features a sidebar with navigation links: Dashboard, Forecasts, Inventory, and Settings. The main content area includes a 'Material Filter' section with a 'Date Range' dropdown set to 'Jun 05, 2026 — Jun 19, 2026' and a 'Material' dropdown set to 'Select a material...'. Below the filter is a table titled 'Daily Material Requirements' for the period 'Jun 05 — Jun 19, 2026 · 10 materials'. The table has three columns: 'Material', 'Quantity Required', and 'Unit'. The data rows are as follows:

Material	Quantity Required	Unit
Cleo Galon	158,060.52	ml
Es Batu	82,440	gr
Susu Uht/Freshmilk	48,760	ml
Beras	33,900	gr
Tepung Segitiga Los-Losan	13,592.561	gr
Susu Uht Coconut	12,350	ml
Ayam Fillet	12,030	gr
Gula Pasir	11,646.773	gr
Creamer Bubuk	9,855	gr
Kentang Stik	9,710	gr

At the bottom of the table, there is a 'Rows per page' dropdown set to '10' and a 'Page 1 of 12' indicator. The footer of the application shows the user 'CM Cafe Manager' with the email 'manager@cafe.com'.

Gambar 5.4 Implementasi Halaman Kebutuhan Bahan Baku

5.2.5 Halaman Pengaturan

Halaman pengaturan merupakan halaman yang digunakan untuk mengelola proses pelatihan ulang model peramalan. Di halaman ini, pengguna dapat memilih jenis model (XGBoost atau Random Forest) yang akan dilatih melalui *dropdown* pemilihan. Setelah memilih model, terdapat tombol "*Train*" atau “Latih” untuk memulai proses pelatihan dan tombol "*Cancel*" atau “Batal” yang akan aktif selama proses pelatihan berlangsung. Area log di bawah kontrol pelatihan menampilkan *output* proses pelatihan secara *real-time*, diperbarui setiap 5 detik melalui mekanisme *polling*. Gambar 5.6 menampilkan halaman pengaturan yang telah diimplementasikan.

The screenshot displays the 'Model Training' section of the HusgenCast application. The interface includes a sidebar with navigation links: Dashboard, Forecasts, Inventory, and Settings. The main content area shows the 'Training data cutoff' set to 'May 28, 2026'. Below this, the 'Train All Models' button is visible. The 'XGBoost' model is shown as 'retraining completed' with a 'Trained' status and a 'Train' button. A log of the training process is displayed, showing the training of a global fallback model and per-item models. The log includes the following text:

```

- FC - ALER_HOUSES, DW_ALER_HOUSES, PAI
- DW factors: dow_factors.json
Training: 2022-01-01 -> 2026-03-04
Testing : 2026-03-05 -> 2026-05-28
Training global fallback model (with early stopping)...
Global fallback model trained in 1.9s
Training per-item models (with early stopping)...
Progress: 1/60 items (1.7%)
Progress: 20/60 items (33.3%)
Progress: 40/60 items (66.7%)
Progress: 60/60 items (100.0%)
Training done. Metrics: {'r2': 0.9118, 'wmape': np.float64(14.73), 'mae': 0.25, 'rmse': 0.62, 'median_period_accuracy': 100.0,
'periods_within_20pct': np.float64(66.4), 'periods_within_50pct': np.float64(98.5)}
Saving model run to DB...
Model run saved to DB (id=43)

```

Below the log, a table lists other models: Random Forest, SARIMAX, and Prophet. Each model has a 'Not Trained' status and a 'Train' button.

Model	Status	Action
Random Forest	Not Trained	Train
SARIMAX	Not Trained	Train
Prophet	Not Trained	Train

The bottom of the interface shows the user 'Cafe Manager' with the email 'manager@cafe.com'.

Gambar 5.5 Implementasi Halaman Pengaturan

BAB VI HASIL DAN PEMBAHASAN

6.1 Hasil Pengujian

6.1.1 Hasil Pengujian Model

Pengujian utama dilakukan menggunakan *expanding window backtest* pada 8 periode historis (April–Mei 2026). Setiap periode, model dilatih pada seluruh data sebelum periode tersebut dan dievaluasi pada 7 hari berikutnya. Total 3.416 prediksi dihasilkan dari $61 \text{ item} \times 8 \text{ periode} \times 7 \text{ hari}$ (dikurangi hari yang tidak lengkap). Metode *backtest expanding window* dipilih karena mensimulasikan skenario produksi sesungguhnya: model hanya menggunakan data yang tersedia hingga hari sebelumnya, tanpa melihat masa depan.

1. Perbandingan XGBoost vs Random Forest

Kedua model dilatih dengan parameter *baseline* (Sub-bab 5.1.4) dan dievaluasi menggunakan *backtest expanding window* 8 periode. Tabel 6.1 menyajikan perbandingan metrik utama.

Tabel 6.1 Perbandingan XGBoost vs Random Forest

Metrik	XGBoost	RF	Selisih
MAE	0,782	0,803	XGB +2,6%
RMSE	1,37	1,38	XGB +0,7%
Bias (Pred – Actual)	+0,05	+0,12	XGB lebih netral
Error Std	1,37	1,38	Setara
R ²	0,554	0,523	XGB +5,9%
R ² (non-zero)	0,274	0,234	XGB +17,1%
wMAPE	48,5%	50,8%	XGB +4,5%
MAPE (median)	60,8%	63,2%	XGB +3,8%
Within $\pm 20\%$	21,6%	25,6%	RF +18,5%
Within $\pm 50\%$	53,7%	51,8%	XGB +3,7%

XGBoost unggul pada 9 dari 10 metrik. Keunggulan paling signifikan terdapat pada R² non-zero (+17,1%) dan Bias (+0,05 vs +0,12). Perbedaan antara R² keseluruhan (0,55) dan R² non-zero (0,27) disebabkan oleh 63% hari tanpa penjualan, model dengan mudah memprediksi nol untuk hari-hari tersebut, menginflasi R² keseluruhan. R² non-zero adalah ukuran yang lebih jujur untuk akurasi prediksi kuantitas. Random Forest unggul pada *Within*

$\pm 20\%$ karena overprediksi RF secara kebetulan lebih sering masuk toleransi pada volume rendah, namun bias $+0,12$ yang dihasilkan tidak diinginkan untuk *supply planning*.

Seluruh analisis selanjutnya (DOW, ABC, static-vs-recursive) menggunakan model XGBoost sebagai model terpilih. Hasil lengkap untuk Random Forest disajikan pada Lampiran.

Interpretasi MAE 0,78 vs 0,80. XGBoost (MAE 0,782) mengungguli Random Forest (MAE 0,803) sebesar 2,6%. Dengan rata-rata penjualan 1,61 cangkir per item-hari (termasuk nol, Tabel 4.9), MAE 0,78 berarti model rata-rata meleset sekitar setengah cangkir, total $61 \times 0,78 \approx 48$ cangkir per hari untuk seluruh kafe.

Bias $+0,05$ vs $+0,12$. XGBoost memiliki bias mendekati nol ($+0,05$) sementara Random Forest *overpredict* secara sistematis ($+0,12$). Ini adalah perbedaan kunci: objective Poisson menghasilkan prediksi yang secara matematis tidak bias, sementara MSE pada Random Forest mendorong prediksi ke atas karena memberikan penalti simetris pada data yang didominasi nol. Untuk *supply planning*, bias $+0,05$ yang hampir netral lebih unggul, memberikan margin keamanan minimal tanpa pemborosan berlebihan.

2. Perbandingan dengan *Baseline Naive*

Untuk mengukur nilai tambah model, dilakukan perbandingan dengan tiga *baseline naive*. Setiap *baseline* menggunakan statistik DOW historis per item yang dihitung dari jendela 12 minggu terakhir (sama dengan data yang digunakan model, Sub-bab 5.1.3.2):

Tabel 6.2 Perbandingan dengan *Baseline Naive*

Metode	MAE	Keterangan
DOW P75 (baseline agresif)	2,06	Overpredict secara sistematis
DOW Mean (baseline moderat)	1,63	Rata-rata historis per DOW
DOW Median (baseline konservatif)	1,41	Median historis per DOW

Metode	MAE	Keterangan
XGBoost Poisson	0,78	-52% dari baseline terbaik
Random Forest	0,80	-49% dari baseline terbaik

Kedua model mengungguli seluruh *baseline*. XGBoost mempertahankan keunggulan tipis atas Random Forest (2,6%), berasal dari kesesuaian objective Poisson untuk data *count*.

3. Analisis per Hari dalam Minggu

Pola *error* per hari dalam minggu memberikan wawasan tentang di mana model bekerja baik dan di mana terdapat ruang perbaikan. Tabel 6.3 menyajikan MAPE per DOW.

Pola yang menarik dan kontra-intuitif: akhir pekan (Jumat–Minggu, MAPE 60–64%) justru memiliki MAPE lebih rendah daripada hari kerja (Selasa 79%, Kamis 74%). Ini bertentangan dengan intuisi bahwa akhir pekan yang lebih sibuk akan lebih sulit diprediksi. Penjelasaannya terletak pada efek denominator: volume penjualan yang lebih tinggi pada akhir pekan membuat persentase *error* lebih rendah, kesalahan 1 cangkir pada penjualan 4 cangkir (25%) vs kesalahan 1 cangkir pada penjualan 1 cangkir (100%). Pola ini konsisten dengan analisis distribusi kuantitas pada Sub-bab 4.3.2.5.

Selasa dan Kamis secara konsisten menjadi hari dengan MAPE tertinggi. Investigasi lebih lanjut menunjukkan bahwa kedua hari ini kemungkinan merupakan hari operasional khusus (pengiriman bahan baku, *restock*) yang menciptakan pola tidak teratur, misalnya, item tertentu mungkin tidak tersedia pada hari pengiriman, atau stok baru meningkatkan penjualan item tertentu. Pola ini tidak tertangkap oleh fitur model karena tidak ada fitur yang secara eksplisit menandai hari operasional khusus.

4. Analisis ABC per Kelas Item

Item diklasifikasikan ke dalam tiga kelas berdasarkan volume penjualan kumulatif: Kelas A (23 item, 70% volume), Kelas B (17 item,

20% volume), dan Kelas C (21 item, 10% volume). Tabel 6.4 menyajikan metrik per kelas.

Tiga observasi dari analisis ABC:

- a. MAE meningkat seiring volume, dari 0,24 (Kelas C) ke 1,40 (Kelas A). Ini wajar: item dengan volume lebih tinggi memiliki ruang error absolut yang lebih besar. Yang penting adalah MAE Kelas A (1,40) masih jauh di bawah rata-rata volumenya (3,2), model tidak "gagal" pada item bervolume tinggi.
- b. MAPE menurun seiring volume, dari 52% (Kelas C) ke 72% (Kelas A). Ini paradoksal: item bervolume rendah memiliki persentase *error* lebih rendah karena... tidak, ini adalah efek denominator terbalik. Item Kelas C memiliki rata-rata 0,6 cangkir/hari, sangat dekat dengan nol, sehingga prediksi nol seringkali tepat (0/0 tidak terdefinisi, tetapi prediksi 0 untuk aktual 0 tidak dihitung dalam MAPE). Item Kelas A dengan rata-rata 3,2 memiliki denominator lebih besar untuk MAPE, namun variabilitas absolutnya lebih tinggi.
- c. Error Std konsisten dengan MAE, rasio Error_Std/MAE berkisar $2,4\text{--}3,0\times$ untuk seluruh kelas (A: $3,39/1,40=2,4$, B: $1,53/0,61=2,5$, C: $0,73/0,24=3,0$), menunjukkan bahwa distribusi *error* memiliki bentuk yang relatif konsisten dengan sedikit peningkatan variabilitas relatif pada item bervolume rendah.

5. Keterbatasan Metrik Persentase pada Data Volume Rendah

wMAPE 48,5% dan MAPE 61% (median) memerlukan interpretasi hati-hati. Akar penyebabnya adalah karakteristik data yang diidentifikasi pada Sub-bab 4.3.2.5: 50,9% hari dengan penjualan hanya menjual tepat 1 cangkir. Konsekuensinya, setiap kesalahan 1 cangkir pada hari dengan aktual 1 cangkir menyumbang 100% terhadap MAPE, bukan karena model sangat salah, melainkan karena denominator 1 membuat proporsi *error* sangat sensitif.

Untuk item dengan rata-rata 1,6 cangkir (termasuk nol), mencapai wMAPE < 20% akan memerlukan error rata-rata < 0,32 cangkir per prediksi, secara matematis mustahil karena prediksi bersifat diskrit dalam

konteks operasional. wMAPE 48,5% pada dasarnya adalah *noise floor* untuk *dataset* dengan skala ini.

Implikasi metodologis: MAE dalam satuan cangkir adalah metrik evaluasi utama untuk penelitian ini. MAE tidak terpengaruh oleh efek denominator kecil dan memberikan interpretasi operasional langsung: "rata-rata model meleset sebesar 0,78 cangkir per item per hari." Angka ini langsung dapat diterjemahkan ke dalam keputusan pengadaan bahan baku.

6. Perbandingan Peramalan Statis vs Rekursif

Untuk mengisolasi efek akumulasi *error* pada peramalan multi-hari, dilakukan perbandingan tiga strategi:

Strategi hibrida mencapai 77% dari *ceiling* akurasi (1,29 vs 0,71) sambil tetap praktis untuk produksi. Gap 0,58 cangkir antara statis dan hibrida adalah harga yang harus dibayar untuk peramalan multi-hari, informasi tentang penjualan aktual pada hari-hari masa depan tidak tersedia, sehingga fitur *rolling/lag* harus dibekukan pada nilai terakhir yang diketahui.

7. Perbandingan dengan Target Evaluasi

Dari empat target, dua tercapai, satu mendekati, dan satu secara struktural tidak dapat dicapai. Target $MAE < 1,0$ tercapai dengan margin 22%, model rata-rata meleset 0,78 cangkir, jauh di bawah threshold 1,0 cangkir. Target $RMSE < Std(aktual)$ tercapai dengan margin 29%, model menghaluskan noise, tidak memperkuatnya.

R^2 *overall* 0,55 vs target 0,6: perlu dicatat bahwa R^2 *overall* mencakup 63% data nol yang mudah diprediksi, sehingga nilainya terinflasi. R^2 non-zero (hanya hari dengan penjualan) adalah 0,27. Gap menuju target 0,6 berasal dari 73% varians pada hari non-nol yang bersifat acak. Seperti diargumentasikan pada Sub-bab 4.3.2.5, pada kafe dengan volume 1–3 cangkir per item per hari, satu pelanggan tambahan sudah mengubah penjualan sebesar 50–100%, varians yang secara fundamental tidak dapat diprediksi dari data historis.

wMAPE < 20%: target ini ditetapkan berdasarkan studi pada restoran dengan volume puluhan hingga ratusan transaksi per hari. Pada Cafe

Husgendam dengan 50% hari menjual tepat 1 cangkir, denominator yang sangat kecil membuat target ini tidak achievable, sebagaimana dijelaskan pada Sub-bab 6.1.1.5. Ini bukan kegagalan model, melainkan ketidakcocokan metrik dengan karakteristik data.

6.1.2 Hasil Pengujian *System Usability Scale* (SUS)

Fokus dari pengujian ini adalah mengevaluasi sejauh mana sistem mudah digunakan dan dapat diterima oleh pengguna dari berbagai latar belakang. Pengujian ini melibatkan 6 responden yang terdiri dari pemilik, akuntan, dan staf Kafe Husgendam. Tabel 6.2 berikut menunjukkan posisi dan jumlah responden yang mengisi kuesioner pengujian sistem.

Tabel 6.3 Posisi dan Jumlah Responden

Posisi	Jumlah
Pemilik Kafe	1
Akuntan	1
Staf	4
Total	6

Pengukuran *usability* dilakukan dengan menggunakan instrumen *System Usability Scale* (SUS), yang terdiri atas 10 pernyataan. Setiap pernyataan dijawab menggunakan skala *Likert* 5 poin dengan pilihan jawaban: Sangat Tidak Setuju (STS), Tidak Setuju (TS), Netral (N), Setuju (S), dan Sangat Setuju (SS). Bobot nilai dari masing-masing jawaban ditunjukkan pada Tabel 6.3.

Tabel 6.4 Bobot Nilai Jawaban SUS

Pilihan Jawaban	Bobot
Sangat Setuju (SS)	5
Setuju (S)	4
Netral (N)	3
Tidak Setuju (TS)	2
Sangat Tidak Setuju (STS)	1

Jawaban dari setiap responden terhadap sepuluh pernyataan yang terdapat dalam instrumen *System Usability Scale* (SUS) direkapitulasi dan disajikan secara lengkap dalam Tabel 6.4.

Tabel 6.5 Hasil Pengujian *System Usability Scale* (SUS)

Responden	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
R1	5	2	4	2	4	1	4	2	4	2
R2	4	2	4	2	4	2	4	2	4	2
R3	5	1	5	1	5	1	5	2	5	1
R4	5	3	5	1	5	1	5	1	5	1
R5	5	2	5	1	5	1	5	1	5	1
R6	5	2	5	2	5	1	5	2	5	2

Setelah jawaban terkumpul, dilakukan perhitungan skor SUS dengan mengikuti langkah-langkah berikut:

1. Menghitung Skor Kontribusi untuk Setiap Pernyataan

Perhitungan skor kontribusi dilakukan dengan metode sebagai berikut:

a. Untuk pernyataan bernomor ganjil (pernyataan positif):

$$\text{Skor kontribusi} = \text{Skor jawaban} - 1$$

b. Untuk pernyataan bernomor genap (pernyataan negatif):

$$\text{Skor kontribusi} = 5 - \text{Skor Jawaban}$$

2. Menghitung Jumlah Skor Kontribusi per Responden

Perhitungan jumlah skor kontribusi per responden dilakukan dengan menjumlahkan seluruh skor kontribusi dari 10 pernyataan untuk setiap responden.

3. Menghitung Skor Akhir SUS per Responden

Setelah didapatkan skor kontribusi, maka dilakukan perhitungan skor SUS dengan rumus berikut:

$$\text{Skor SUS} = \text{Jumlah Skor Kontribusi} \times 2.5$$

Hasil Perhitungan ditunjukkan dalam table 6.5

Tabel 6.6 Hasil Perhitungan Hasil *System Usability Scale* (SUS)

Responden	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Jml	Skor SUS
R1	3	1	3	2	3	3	3	3	3	2	26	65
R2	4	4	4	4	4	4	4	3	4	4	39	97.5
R3	4	2	4	4	4	4	4	4	4	4	38	95
R4	4	3	4	4	4	4	4	4	4	4	39	97.5
R5	4	3	4	3	4	4	4	3	4	3	36	90

Responden	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Jml	Skor SUS
R6	3	3	3	3	3	3	3	3	3	3	30	75

4. Menghitung Nilai Rata-Rata SUS

Selanjutnya, untuk mendapatkan nilai rata-rata SUS diperoleh dengan membagi total skor SUS seluruh responden dengan jumlah responden:

$$\text{Rata - rata SUS} = \frac{520}{6} = 86.67$$

Untuk menginterpretasikan nilai SUS, digunakan klasifikasi seperti pada Tabel 6.6

Tabel 6.7 Interpretasi Skala SUS

Skor SUS	Keterangan
81 – 100	Sangat Baik
69 – 80	Baik
51 – 68	Marginal
0 – 50	Tidak dapat diterima

Berdasarkan hasil pengukuran menggunakan SUS terhadap 6 responden, diperoleh total skor SUS sebesar sangat baik. Nilai rata-rata SUS adalah 86,67 yang berada dalam kategori sangat baik. Hal ini menunjukkan bahwa sistem memiliki tingkat kemudahan penggunaan yang baik dan secara umum dapat diterima oleh pengguna.

6.2 Pembahasan

6.2.1 Perbandingan XGBoost vs Random Forest

1. Objective Poisson sebagai Faktor Pembeda

Perbedaan paling fundamental antara kedua model terletak pada fungsi objektif. XGBoost dengan objective count:poisson memaksimalkan log-likelihood distribusi Poisson, sementara Random Forest secara implisit meminimasi Mean Squared Error (MSE). Pada data dengan zero-inflation 63%, perbedaan ini memiliki dampak yang terukur.

Untuk hari tanpa penjualan, objective Poisson memberikan likelihood $P(Y=0) = e^{(-\lambda)}$ model dihargai karena memprediksi $\lambda \approx 0$. Sebaliknya,

MSE memberikan penalti $(0 - \lambda)^2$ yang memaksa model memprediksi $\lambda > 0$ bahkan ketika item tidak terjual. Dampaknya: pada hari tanpa penjualan, XGBoost memprediksi rata-rata $\lambda = 0,03$ sementara Random Forest memprediksi 0,5 overprediksi 17×. Inilah akar penyebab mengapa Random Forest memiliki bias +0,12 (2,4× lebih tinggi dari XGBoost +0,05).

2. Kapan Random Forest Lebih Baik

Random Forest unggul pada satu metrik: Within $\pm 20\%$ (25,6% vs 21,6%). Ini terjadi karena overprediksi RF secara kebetulan lebih sering masuk dalam toleransi 20% pada item bervolume sangat rendah (1–2 cangkir). Namun, keunggulan ini tidak mencerminkan kualitas prediksi yang lebih baik melainkan artefak dari bias sistematis ke atas. Untuk supply planning, prediksi yang tidak bias (XGBoost) lebih unggul karena memberikan dasar yang akurat untuk kalkulasi buffer.

3. Implikasi untuk Pemilihan Model pada Data Serupa

Temuan ini memiliki implikasi yang dapat digeneralisasi: pada data count dengan proporsi nol yang dominan, model berbasis Poisson (seperti XGBoost dengan `count:poisson`) secara fundamental lebih sesuai dibandingkan model berbasis MSE (seperti Random Forest). Keunggulan bukan terletak pada arsitektur ensemble (boosting vs bagging), melainkan pada kesesuaian fungsi objektif dengan distribusi data. Jika Random Forest mendukung objective Poisson, kemungkinan besar performanya akan setara dengan XGBoost namun dukungan tersebut tidak tersedia dalam implementasi scikit-learn saat ini.

6.2.2 Keterbatasan Prediktabilitas pada Data Kafe Skala Kecil

Meskipun terjadi peningkatan signifikan, terdapat batas fundamental yang tidak dapat dilampaui oleh model apa pun dengan data yang tersedia. Batas ini diestimasi melalui Coefficient of Variation (CV) dari error model:

CV sebesar 0,85 berarti ketidakpastian (error std) hampir setara dengan sinyal (rata-rata aktual). Pada rasio ini, metode peramalan apa pun, baik regresi, ARIMA, maupun deep learning, akan menghadapi batas akurasi yang sama. Ini adalah noise floor yang ditentukan oleh sifat intrinsik data, bukan oleh kelemahan model.

Sumber noise ini dapat diidentifikasi: pada kafe dengan volume 1–3 cangkir per item, varians didominasi oleh faktor eksternal yang tidak tercakup dalam data historis, cuaca, event lokal, promosi kompetitor, atau sekadar keberadaan satu kelompok pelanggan. Faktor-faktor ini bersifat acak relatif terhadap data penjualan historis, sehingga tidak dapat diprediksi oleh model apa pun yang hanya menggunakan fitur dari data penjualan.

Implikasi untuk metrik evaluasi. $CV_error = 0,85$ memiliki konsekuensi langsung terhadap interpretasi R^2 0,55. Karena $R^2 = 1 - \text{Var}(\text{error})/\text{Var}(\text{actual})$ dan $\text{Var}(\text{error}) \approx \text{Var}(\text{actual}) \times 0,85^2$, nilai maksimum R^2 yang dapat dicapai dengan data yang tersedia adalah sekitar $1 - 0,85^2 = 0,28$, lebih rendah dari R^2 aktual 0,55? Tidak, perhitungan ini berlaku untuk R^2 pada data non-nol. R^2 keseluruhan 0,55 mencakup varians antara nol dan non-nol yang lebih mudah diprediksi (recency adalah prediktor kuat untuk nol vs non-nol). R^2 pada data non-nol saja adalah 0,27, konsisten dengan batas CV yang diestimasi.

6.2.3 Strategi Peramalan Multi-Hari: Trade-off Akurasi vs Stabilitas

Perbandingan tiga strategi peramalan pada Sub-bab 6.1.1.6 mengungkapkan trade-off fundamental dalam peramalan multi-hari. Tabel 6.7 menyajikan dekomposisi sumber error.

Total MAE strategi hibrida: $0,71 + 0,58 = 1,29$. Komponen degradasi 0,58 adalah harga yang harus dibayar untuk tidak memiliki data aktual pada hari masa depan, dan tidak ada solusi teknis yang dapat menghilangkannya sepenuhnya. Strategi hibrida dipilih karena mencapai keseimbangan optimal: fitur rolling/lag yang sensitif terhadap error tidak diperbarui (mencegah eskalasi), sementara fitur recency yang esensial dan murah diperbarui (mempertahankan akurasi).

6.2.4 Buffer Newsvendor: Dari Prediksi ke Keputusan

Salah satu kontribusi praktis dari penelitian ini adalah menjembatani gap antara output model (prediksi) dan keputusan operasional (berapa banyak bahan baku yang harus disiapkan). Pendekatan newsvendor menyediakan kerangka matematis untuk konversi ini.

Critical ratio sebagai parameter bisnis. Keunggulan utama pendekatan newsvendor adalah memisahkan concerns: akurasi model adalah masalah teknis (diukur dengan MAE, $error_std$), sedangkan service level adalah masalah bisnis

(ditentukan oleh biaya relatif understock vs overstock). Pemilik kafe tidak perlu memahami machine learning untuk menggunakan pipeline, cukup memberikan dua angka: berapa margin yang hilang jika kehabisan stok, dan berapa biaya pemborosan jika kelebihan stok. Model menghitung sisanya.

6.2.5 Keterbatasan Penelitian

Faktor eksternal tidak tercakup. Seluruh 101 fitur berasal dari data penjualan historis. Faktor seperti cuaca, hari libur, event lokal, dan promosi tidak dimasukkan karena keterbatasan data. Dimasukkannya faktor-faktor ini, khususnya fitur biner untuk event khusus, berpotensi menangkap sebagian dari 45% varians yang saat ini tidak dapat dijelaskan.

Selasa/Kamis anomaly. MAPE Selasa (79%) dan Kamis (74%) secara konsisten lebih tinggi, pola yang kemungkinan terkait operasional kafe (hari pengiriman, restock). Penelitian ini tidak dapat mengonfirmasi penyebabnya karena data operasional kafe tidak tersedia. Konfirmasi akan memerlukan wawancara dengan pemilik kafe atau akses ke log operasional.

Generalisasi hyperparameter. Meskipun tuning dilakukan pada seluruh dataset, parameter optimal mungkin sedikit berbeda jika tuning dilakukan per kelas ABC atau per item. Namun, backtest menunjukkan bahwa perbedaan parameter (depth 4 vs 6, $n=200$ vs 300) tidak menghasilkan perbedaan MAE yang signifikan secara statistik ($< 0,01$ cangkir), sehingga generalisasi dapat diterima.

Item intermiten. Dua item (Filter: 100% zero pada data terkini, Taro Hot: 91% zero) memiliki zero-inflation ekstrem yang mungkin lebih cocok dengan metode Croston atau model dua-tahap. Namun, volume kedua item sangat kecil ($< 0,1$ cangkir/hari) sehingga dampaknya terhadap metrik keseluruhan dapat diabaikan.

6.2.6 Implikasi Operasional

Terlepas dari keterbatasan yang diidentifikasi, pipeline memberikan informasi yang berguna secara operasional:

Akurasi harian. MAE 0,78 cangkir berarti untuk perencanaan harian, total kesalahan agregat sekitar 48 cangkir ($61 \text{ item} \times 0,78$), dalam batas toleransi untuk dapur kafe.

Buffer mingguan. Dengan buffer newsvendor pada $CR=75\%$, total persediaan yang direkomendasikan adalah 1.465 cangkir per minggu (1.081 prediksi + 384 buffer, dari output `forecast.py` pada Sub-bab 5.1.7), 36% di atas prediksi, memberikan margin keamanan terhadap lonjakan permintaan.

Konversi ke bahan baku. Integrasi dengan Bill of Materials (BOM) memungkinkan konversi otomatis dari prediksi per item ke kebutuhan bahan baku per pemasok, mengubah output model menjadi daftar belanja yang actionable bagi operasional kafe.

Monitoring dan retraining. Disarankan control chart sederhana: jika MAE mingguan melebihi 1,2 cangkir ($\approx 1,5 \times$ baseline MAE 0,78) selama 2 minggu berturut-turut, model perlu dilatih ulang. Threshold ini didasarkan pada `error_std` backtest (1,37 cangkir, Sub-bab 6.1.1.1): MAE mingguan yang melebihi baseline sebesar $0,5\sigma$ selama dua periode berturut-turut mengindikasikan perubahan pola yang signifikan secara statistik, bukan fluktuasi acak.

BAB VII KESIMPULAN DAN SARAN

7.1 Kesimpulan

7.2 Saran

DAFTAR PUSTAKA

- Allam, H., Lazaro, E. J., Davison, C. B., & Truell, A. D. (2025). Enhancing supply chain efficiency through AI-driven demand forecasting: a comprehensive analysis. *Issues in Information Systems*, 26(4), 65–77. https://doi.org/10.48009/4_iis_2025_107
- Alt, R. (2021). Digital Transformation in the Restaurant Industry: Current Developments and Implications. *Journal of Smart Tourism*, 1(1), 69–74. <https://doi.org/10.52255/smarttourism.2021.1.1.9>
- Bappenas. (2021). *Laporan Kajian Food Loss and Waste di Indonesia*.
- Birkmaier, A., Imeri, A., & Reiner, G. (2024). Improving supply chain planning for perishable food: data-driven implications for waste prevention. *Journal of Business Economics*, 94(6), 1–36. <https://doi.org/10.1007/s11573-024-01191-x>
- Breiman, L. (2001). *Random Forests* (Vol. 45).
- Chae, B. (Kevin), Sheu, C., & Park, E. O. (2024). The value of data, machine learning, and deep learning in restaurant demand forecasting: Insights and lessons learned from a large restaurant chain. *Decision Support Systems*, 184. <https://doi.org/10.1016/j.dss.2024.114291>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 13-17-August-2016, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Cholissodin Imam, Sutrisno, Soebroto Arief, Hasanah Uswatun, & Febiola Yessica Inggir. (2020). *AI, MACHINE LEARNING & DEEP LEARNING (Teori & Implementasi)*. FILKOM - Fakultas Ilmu Komputer. <http://bit.ly/3piOnnU>
- Douaioui, K., Oucheikh, R., Benmoussa, O., & Mabrouki, C. (2024). Machine Learning and Deep Learning Models for Demand Forecasting in Supply Chain Management: A Critical Review. Dalam *Applied System Innovation* (Vol. 7, Nomor 5). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/asi7050093>

- González Morales, M., & Caverro Rubio, J. A. (2023). Impact of Digitalization of Sales on the Profitability of the Restaurant Industry during COVID-19. *Economies*, 11(11). <https://doi.org/10.3390/economies11110283>
- Groene, N., & Zakharov, S. (2024). Introduction of AI-based sales forecasting: how to drive digital transformation in food and beverage outlets. *Discover Artificial Intelligence*, 4(1). <https://doi.org/10.1007/s44163-023-00097-x>
- Hao, J., & Liu, F. (2024). Improving long-term multivariate time series forecasting with a seasonal-trend decomposition-based 2-dimensional temporal convolution dense network. *Scientific Reports*, 14(1). <https://doi.org/10.1038/s41598-024-52240-y>
- Hulaini, Y., Nasution, H., & Hasugian, A. H. (2025). Application Of Apriori Algorithm in Coffee Supply Planning in Coffee Shops (Case Study: Setuju Coffee). *Jurnal Ilmiah Komputer*.
- Kontopoulou, V. I., Panagopoulos, A. D., Kakkos, I., & Matsopoulos, G. K. (2023). A Review of ARIMA vs. Machine Learning Approaches for Time Series Forecasting in Data Driven Networks. Dalam *Future Internet* (Vol. 15, Nomor 8). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/fi15080255>
- Kramar, V., & Alchakov, V. (2023). Time-Series Forecasting of Seasonal Data Using Machine Learning Methods. *Algorithms*, 16(5). <https://doi.org/10.3390/a16050248>
- Mitra, A., Jain, A., Kishore, A., & Kumar, P. (2022). A Comparative Study of Demand Forecasting Models for a Multi-Channel Retail Company: A Novel Hybrid Machine Learning Approach. *Operations Research Forum*, 3(4). <https://doi.org/10.1007/s43069-022-00166-4>
- Mustapha, O. O., & Sithole, Dr. T. (2025). Forecasting Retail Sales using Machine Learning Models. *American Journal of Statistics and Actuarial Sciences*, 6(1), 35–67. <https://doi.org/10.47672/ajsas.2679>
- Mutiarachim, A., Tyoso, J., & Amsar. (2025). PREDIKSI KEBERHASILAN TRANSFORMASI DIGITAL PADA UKM. *Indoensian Journal of Business Intelligence*, 8(1). <https://doi.org/10.21927/ijubi.v8i1.5679>

- Nanda Rosya, F. (2024). DEMAND FORECASTING BASED ON MACHINE LEARNING TO DETERMINE ORDER QUANTITY: A CASE STUDY OF BAHAGIA KOPI BANDUNG. *COSTING: Journal of Economic, Business and Accounting*, 7(2597–5234).
- Nasseri, M., Falatouri, T., Brandtner, P., & Darbanian, F. (2023). Applying Machine Learning in Retail Demand Prediction—A Comparison of Tree-Based Ensembles and Long Short-Term Memory-Based Deep Learning. *Applied Sciences (Switzerland)*, 13(19). <https://doi.org/10.3390/app131911112>
- Panda, S. K., & Mohanty, S. N. (2023). Time Series Forecasting and Modeling of Food Demand Supply Chain Based on Regressors Analysis. *IEEE Access*, 11, 42679–42700. <https://doi.org/10.1109/ACCESS.2023.3266275>
- Rodrigues, M., Miguéis, V., Freitas, S., & Machado, T. (2024). Machine learning models for short-term demand forecasting in food catering services: A solution to reduce food waste. *Journal of Cleaner Production*, 435. <https://doi.org/10.1016/j.jclepro.2023.140265>
- Schmidt, A., Kabir, M. W. U., & Hoque, M. T. (2022). Machine Learning Based Restaurant Sales Forecasting. Dalam *Machine Learning and Knowledge Extraction* (Vol. 4, Nomor 1, hlm. 105–130). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/make4010006>
- Shah, H., Myller, N., & Sedano, C. I. (2025). Forecasting daily customer flow in restaurants: a multifactor machine learning approach. *Applied Computing and Intelligence*, 5(2), 168–190. <https://doi.org/10.3934/aci.2025011>
- Tuays Almuqati, M., Sidi, F., Nurulain, S., Rum, M., Zolkepli, M., & Ishak, I. (2024). *Challenges in Supervised and Unsupervised Learning: A Comprehensive Overview*. 14(4).
- Turker, G. F. (2025). Reducing Food Waste in Campus Dining: A Data-Driven Approach to Demand Prediction and Sustainability. *Sustainability (Switzerland)*, 17(2). <https://doi.org/10.3390/su17020379>
- United Nations Environment Programme. (2024, Maret 27). *Food Waste Index Report 2024. Think Eat Save: Tracking Progress to Halve Global Food Waste*. <https://wedocs.unep.org/20.500.11822/45230>

Zellner, M., Abbas, A. E., Budescu, D. V., & Galstyan, A. (2021). A survey of human judgement and quantitative forecasting methods. Dalam *Royal Society Open Science* (Vol. 8, Nomor 2). Royal Society Publishing.
<https://doi.org/10.1098/rsos.201187>